RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution | Affiliated to Anna University, Chennai

Thandalam, Chennai - 602 105

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23532 - COMPUTER NETWORKS LABORATORY

LABORATORY MANUAL & OBSERVATION RECORD

SUBMITTED BY

THANIKAIVEL S

REGISTER NUMBER: 2116240701562

Degree & Branch: B.E. Computer Science and Engineering

Year & Semester: III Year / V Semester (Odd Semester 2026 - 2027)

Rajalakshmi Engineering College, Chennai

COURSE OVERVIEW & LIST OF EXPERIMENTS

Course Code: CS23532 | Course Name: Computer Networks Laboratory
Degree & Department: B.E. Computer Science and Engineering | Regulation: Autonomous
Total Practical Hours: 60 Hours | Academic Year: 2026 - 2027 (V Semester)

Exp. No.	Title of the Experiment	Domain / Layer	Hours
1	Study of Various Network Commands in Linux and Windows	Network Utilities & Diagnostics	4
2	Study of Network Cables & Hands-on Cable Crimping	Physical Layer (Cabling & Standards)	4
3	Network Device Simulation using Cisco Packet Tracer (Hub vs. Switch)	Data Link Layer & Topologies	4
4	Local Area Network (LAN) Setup & Resource Sharing using Switch	LAN Infrastructure & Addressing	2
5	Packet Capture & Protocol Stack Inspection using Wireshark	Multi-layer Packet Analysis	4
6	Error Detection and Correction at Data Link Layer (Hamming Code)	Data Link Layer (Error Control)	4
7	Flow Control at Data Link Layer using Sliding Window Protocol	Data Link Layer (Flow Control)	4
8	Live Host Discovery using Nmap Scans on TryHackMe Platform	Network Security & Reconnaissance	4
8(b)	Wireless LAN Configuration using Cisco Packet Tracer	IEEE 802.11 Infrastructure	2
9	Classless Subnetting Implementation in Cisco Packet Tracer	Network Layer (CIDR / VLSM)	4
10(a)	Internetworking with Routers in Cisco Packet Tracer	Network Layer (Routing Fundamentals)	4
10(b)	Internetworking using Wireless Router, DHCP Server & Cloud	Tiered Enterprise Internetwork	4
11(a)	Static Routing Protocol Configuration in Cisco Packet Tracer	Network Layer (Static Routing)	4
11(b)	Dynamic Routing Simulation using RIP in Cisco Packet Tracer	Network Layer (Distance Vector)	4
12(a)	Transport Layer Communication: TCP & UDP Echo Client-	Transport Layer (Socket	2

	Server	Programming)	
12(b)	Transport Layer Communication: Bidirectional Chat Client-Server	Transport Layer (Socket Programming)	2
13	Implementation of Custom Ping Utility using ICMP Sockets	Network / Transport Diagnostics	2
14	Network Packet Sniffing using RAW Sockets in Python	Network Monitoring & Promiscuous Mode	4
15	Web Server Log Analysis and Traffic Auditing using Webalizer	Application Layer & Analytics	4

EXPERIMENT 1: STUDY OF VARIOUS NETWORK COMMANDS IN LINUX AND WINDOWS

AIM: To gain comprehensive hands-on practice with fundamental networking command-line utilities across both Windows and Linux operating systems for host identification, interface configuration, routing analysis, reachability verification, and protocol troubleshooting.

1.1 Theoretical Background

Network troubleshooting and diagnostic commands form the bedrock of systems and network administration. These utilities operate across various tiers of the TCP/IP protocol suite, facilitating real-time queries into host addressing, physical address resolution, routing topologies, and active transport sessions. Proficiency across both Windows NT Command Prompt / PowerShell and Linux POSIX shells enables administrators to rapidly diagnose connectivity outages, DNS misconfigurations, routing blackholes, and interface bottlenecks.

1.2 Windows Networking Command-Line Utilities

Command	Syntax & Common Options	Operational Purpose & Description
arp	arp -a arp -d <ip> arp -s <ip> <mac>	Displays and modifies the Address Resolution Protocol cache mapping IP addresses to physical MAC addresses.
hostname	hostname	Returns the NetBIOS and DNS computer name of the currently logged-in workstation.
ipconfig	ipconfig ipconfig /all ipconfig /release ipconfig /renew ipconfig /flushdns	Queries interface configurations including IPv4/IPv6 addresses, subnet masks, default gateways, DHCP leases, and DNS caches.
nslookup	nslookup <domain> nslookup -type=mx <domain>	Queries Domain Name System (DNS) servers to resolve hostnames to IP addresses or extract specific Resource Records (A, AAAA, MX, NS).
netstat	netstat -a netstat -b netstat -e netstat -n netstat -r netstat -s	Displays active TCP connections, listening ports, Ethernet interface statistics, IPv4 routing tables, and protocol-specific metrics.

ping	ping <host> ping -t <host> ping -n <count> <host>	Tests IP-level reachability by transmitting ICMP Echo Request messages and measuring round-trip latency and packet loss.
pathping	pathping <host>	Combines ping and tracert functionalities, analyzing packet loss across intermediate router hops over a sustained evaluation period.
route	route print route add <dest> mask <mask> <gw>	Displays and manipulates the local IP routing table, identifying default gateways and interface routing metrics.
tracert	tracert <target> tracert -d <target>	Determines the sequential path packets take to reach a destination by incrementing the IP Time-To-Live (TTL) field.

1.3 Linux Networking Command-Line Utilities

Modern Linux distributions leverage the unified 'iproute2' toolset alongside legacy diagnostics to manage network interfaces, routing tables, and packet inspection:

Command	Key Usage & Options	Operational Capability
ip	ip addr show ip link set dev <if> up/down ip route show	Modern replacement for ifconfig and route. Configures network devices, IP addresses, routing policies, and tunnels.
ifconfig	ifconfig -a ifconfig eth0 <ip> netmask <mask>	Classic Unix interface configuration utility used to query MAC addresses, MTU, RX/TX packet counts, and interface status.
mtr	mtr <target> mtr --report <target>	My Traceroute: Combines traceroute and continuous ping into a dynamic, real-time diagnostic reporting hop-by-hop latency and packet loss.
tcpdump	tcpdump -i eth0 -c 10 tcpdump -i any port 80 -n	Command-line packet analyzer. Captures raw packets traversing an interface and filters based on protocol, port, or host.
traceroute	traceroute <target> traceroute -I <target>	Traces the routed path to a remote host using UDP probes (default in Linux) or ICMP Echo packets (-I option).
ss	ss -tuln ss -t -a	Socket Statistics: Modern replacement for netstat, quickly dumping socket information directly from Linux kernel space.

dig	dig <domain> ANY dig +trace <domain>	Domain Information Groper: Flexible DNS exploration tool querying name servers and displaying root-level delegation traces.
nmcli	nmcli device status nmcli connection show	NetworkManager Command-Line Interface. Used to inspect and alter network profiles and physical interfaces in enterprise Linux.

CORE DIAGNOSTIC INSIGHT: Key Difference between arp and route: The 'arp' utility operates at the Data Link Layer (Layer 2) to resolve next-hop MAC addresses on the local broadcast segment. In contrast, 'route' operates at the Network Layer (Layer 3) to choose the appropriate interface and next-hop gateway IP based on destination IP subnet matching.

1.4 Practical Verification & Diagnostic Checklist

When troubleshooting a host experiencing complete loss of network connectivity, follow this systematic diagnostic sequence:

1. Physical & Link Verification: Check physical cable seating and verify link status with 'ip link' or Windows Network Connections.
2. Loopback Self-Test: Execute 'ping 127.0.0.1' to confirm the local TCP/IP protocol stack is initialized and functional.
3. Local Address Verification: Run 'ipconfig /all' (Windows) or 'ip addr' (Linux) to ensure a valid IP is assigned (detecting APIPA 169.254.x.x anomalies).
4. Gateway Reachability: Execute 'ping <Default_Gateway_IP>' to verify communication across the local switch fabric.
5. External Reachability: Execute 'ping 8.8.8.8' to determine whether Internet routing beyond the edge router is intact.
6. Name Resolution: Execute 'nslookup google.com' or 'dig google.com' to isolate DNS resolution failures from pure IP connectivity.

EXPERIMENT 2: STUDY OF NETWORK CABLES & HANDS-ON CABLE CRIMPING

AIM: To explore physical layer transmission media including Unshielded Twisted Pair (UTP) categories, understand TIA/EIA 568A and 568B pinout standards, and fabricate functional Straight-Through and Crossover Ethernet patch cables using professional modular crimping tools.

2.1 Classification of Network Cables

Data transmission at the Physical Layer depends on physical cabling media. The three dominant classes are:

1. Twisted Pair Cables: Consist of color-coded copper wire pairs twisted around each other to cancel out electromagnetic interference (EMI) and radio frequency interference (RFI) through differential signaling.
   - UTP (Unshielded Twisted Pair): Most common in local area networks, highly flexible and cost-effective.
   - STP (Shielded Twisted Pair): Encased in metallic foil or braided shielding for industrial, high-noise environments.
2. Coaxial Cable: Features a central copper core surrounded by dielectric insulation, metallic braided shield, and an outer jacket (common in legacy 10BASE2/10BASE5 networks and cable broadband).
3. Fiber Optic Cable: Uses silica glass or plastic cores to transmit data as modulated optical pulses, offering immunity to EMI, ultra-high bandwidth, and transmission distances exceeding tens of kilometers.

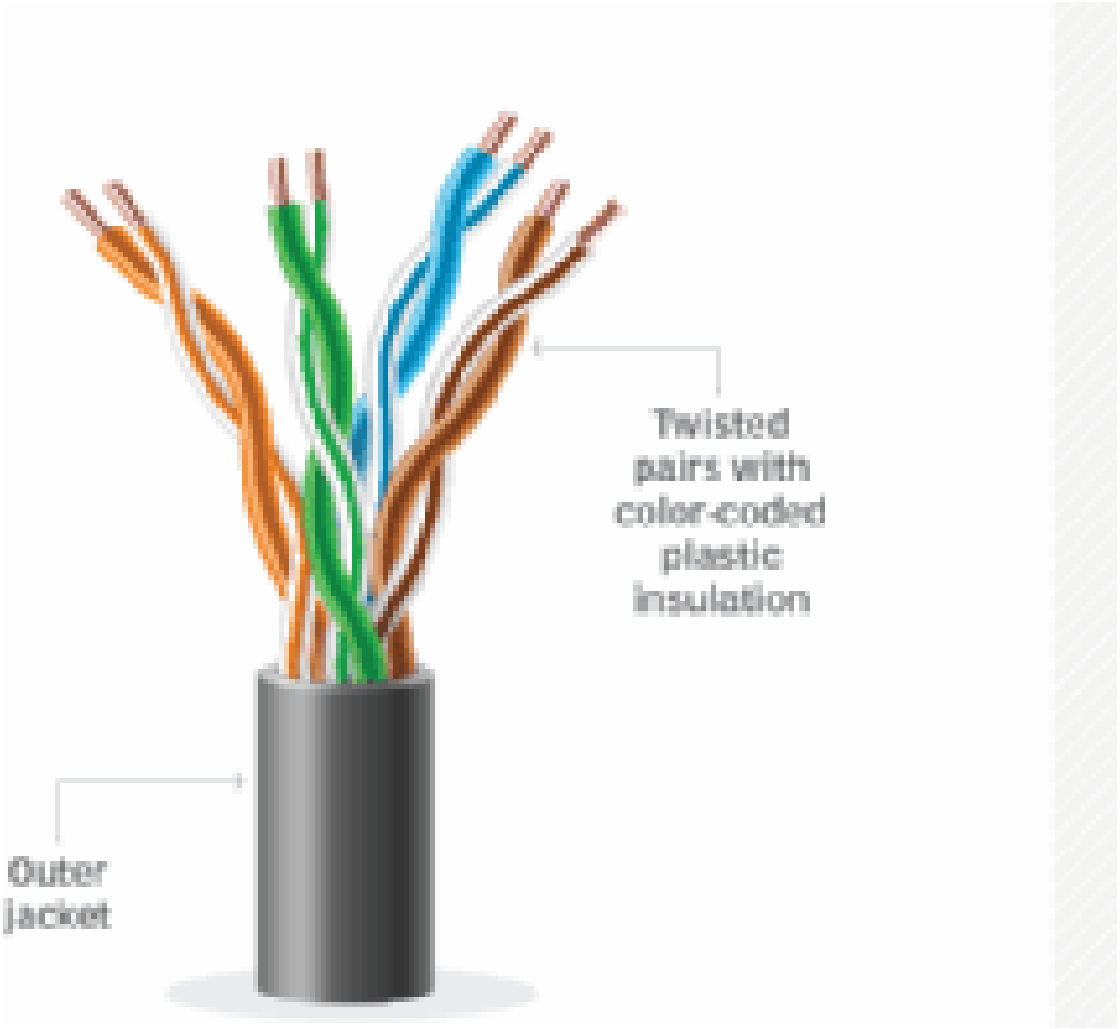


Figure 2.1: Structural Comparison of Twisted Pair and Coaxial Cables

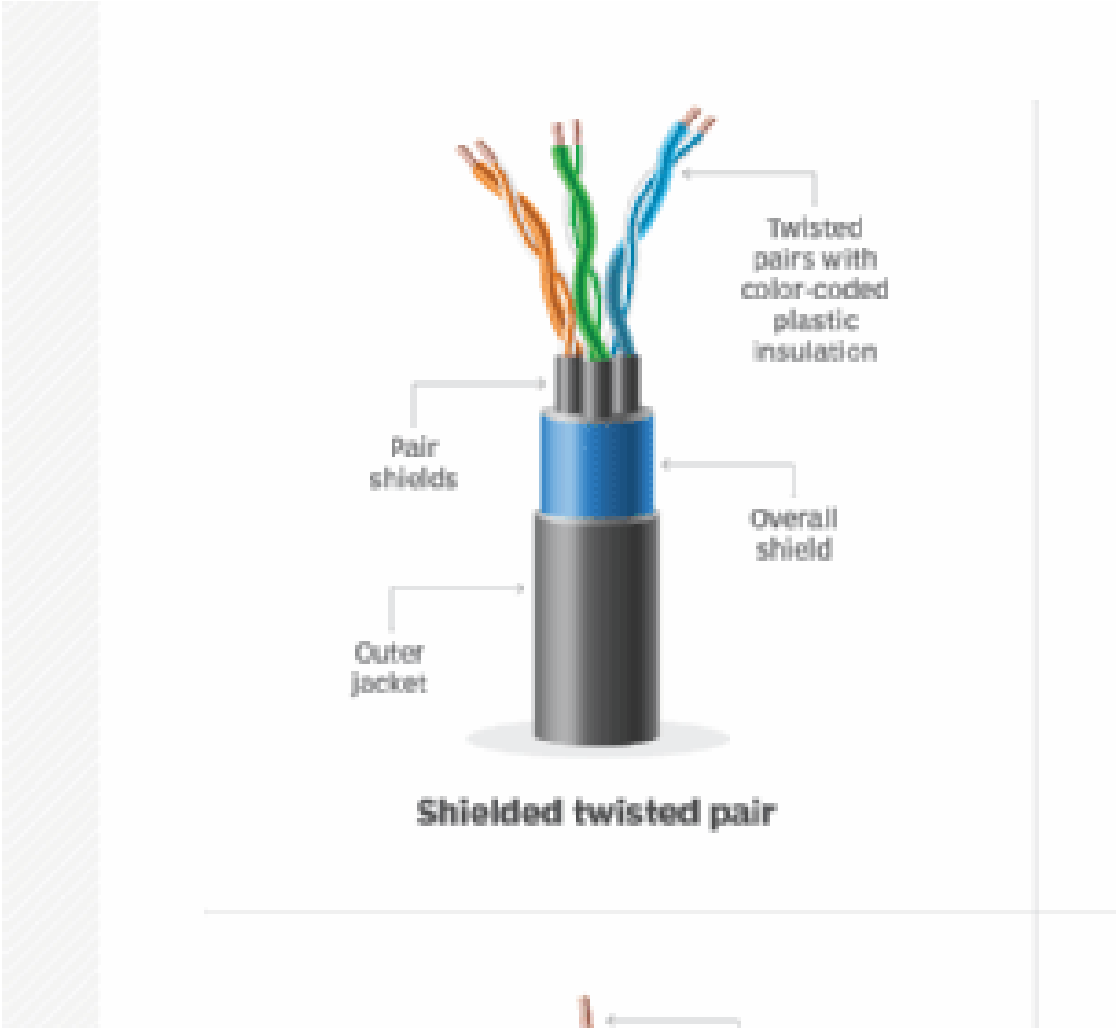


Figure 2.2: Shielded Twisted Pair (STP) Internal Layering

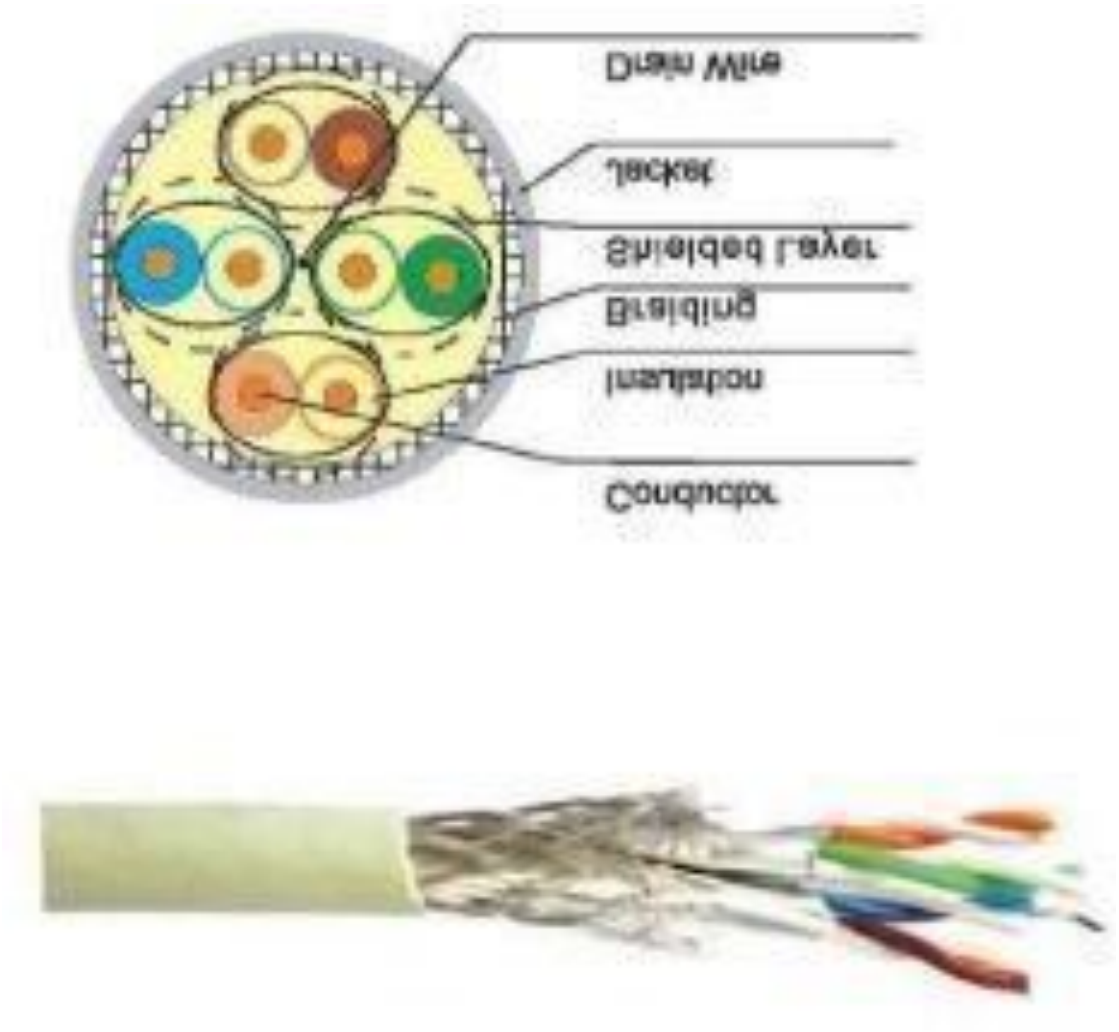


Figure 2.3: Unshielded Twisted Pair (UTP) Multi-Pair Bundles

2.2 Twisted Pair Performance Categories

Category	Max Data Rate	Bandwidth	Standard Application / Use Case
Cat 3	10 Mbps	16 MHz	Legacy 10BASE-T Ethernet and analog PBX telephony.
Cat 5	100 Mbps	100 MHz	Legacy Fast Ethernet (100BASE-TX).
Cat 5e	1000 Mbps (1 Gbps)	100 MHz	Standard Gigabit Ethernet (1000BASE-T) with enhanced crosstalk suppression.
Cat 6	1 Gbps / 10 Gbps (<55m)	250 MHz	Modern enterprise Gigabit LANs and short-distance 10G deployments.
Cat 6a	10 Gbps	500 MHz	Full-distance (100m) 10 Gigabit Ethernet

			with strict alien crosstalk resistance.
Cat 7 / Cat 8	25 Gbps - 40 Gbps	600 - 2000 MHz	Ultra-high-speed datacenter top-of-rack switching and backbone interconnections.

2.3 TIA/EIA 568A vs. TIA/EIA 568B Standards

The Telecommunications Industry Association defines two standardized color codes for terminating 8-pin RJ-45 modular connectors. The difference lies in the swapping of Pair 2 (Orange) and Pair 3 (Green):

Pin No.	TIA/EIA 568A Color Scheme	TIA/EIA 568B Color Scheme	Ethernet Signal Function
Pin 1	White / Green	White / Orange	Transmit Data + (Tx+)
Pin 2	Green	Orange	Transmit Data - (Tx-)
Pin 3	White / Orange	White / Green	Receive Data + (Rx+)
Pin 4	Blue	Blue	Unused in 10/100 Mbps (Bi-directional in 1000BASE-T)
Pin 5	White / Blue	White / Blue	Unused in 10/100 Mbps (Bi-directional in 1000BASE-T)
Pin 6	Orange	Green	Receive Data - (Rx-)
Pin 7	White / Brown	White / Brown	Unused in 10/100 Mbps (Bi-directional in 1000BASE-T)
Pin 8	Brown	Brown	Unused in 10/100 Mbps (Bi-directional in 1000BASE-T)



Figure 2.4: RJ-45 Modular Connector Pinout Numbering (Left to Right, Pins 1 to 8)



Figure 2.5: Color Wiring Layout: T568A Standard (Left) and T568B Standard (Right)

2.4 Straight-Through vs. Crossover Patch Cables

1. Straight-Through Cable:
   - Terminated with the identical standard on both ends (either 568A-568A or 568B-568B; standard practice is 568B on both ends).
   - Used to interconnect DTE (Data Terminal Equipment) to DCE (Data Circuit-terminating Equipment) devices, such as PC to Switch, Switch to Router, or Laptop to Hub.

2. Crossover Cable:
   - Terminated with T568A on one end and T568B on the opposite end.
   - Used to connect similar devices without an intervening switch fabric, such as PC to PC, Switch to Switch, or Router to Router.

- Note: Modern NICs implement Auto-MDI/MDIX, which dynamically detects pinout orientation and automatically crosses over Transmit and Receive pairs in hardware.

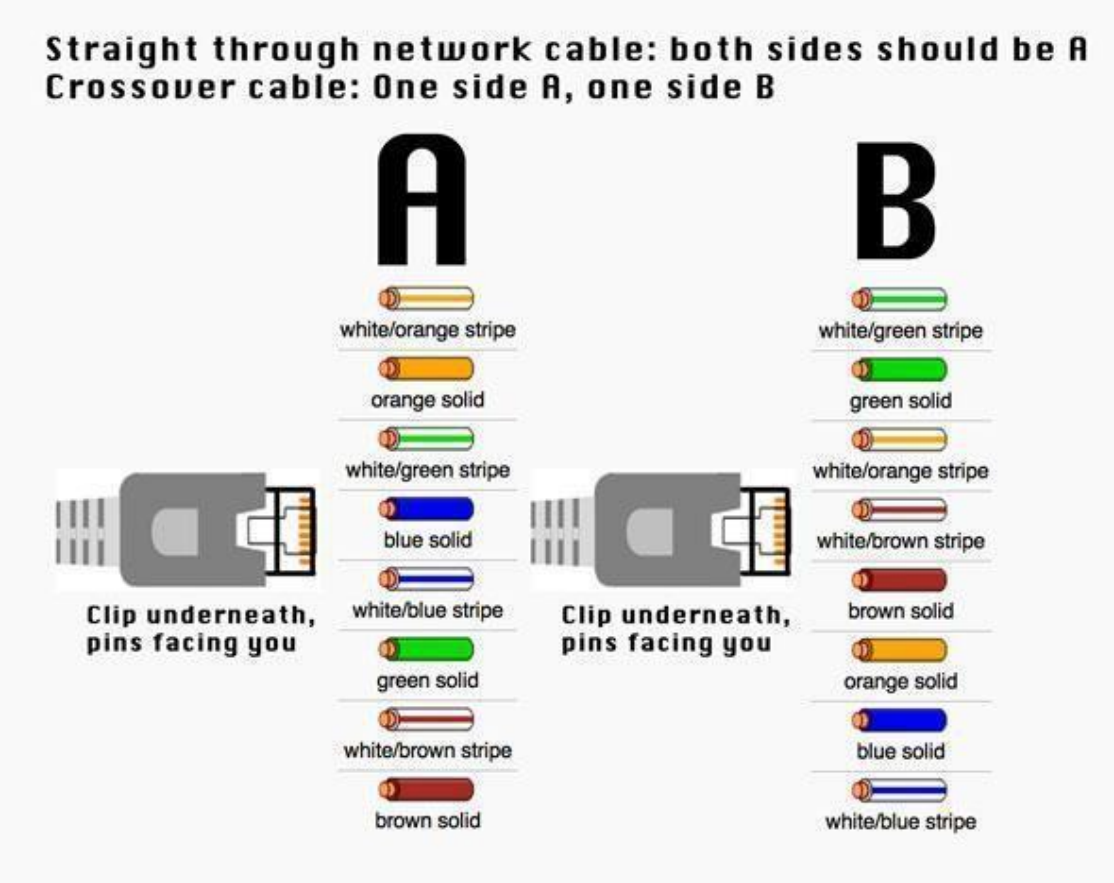


Figure 2.6: Straight-Through Cable Pin Configuration

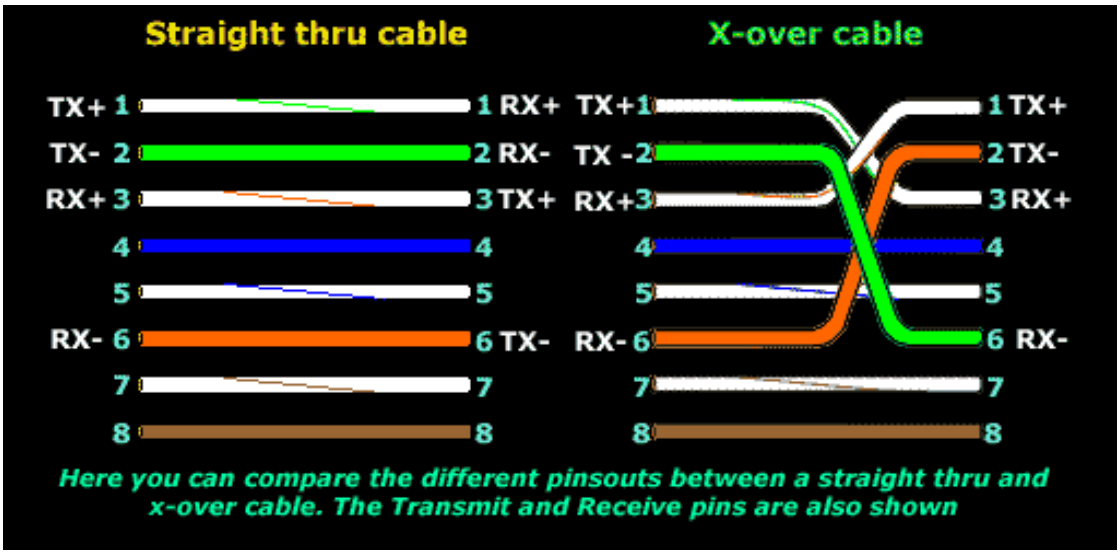


Figure 2.7: Crossover Cable Wiring Pin Assignment

2.5 Step-by-Step Cable Crimping Procedure

Tools Required: Category 5e/6 UTP Cable roll, RJ-45 8P8C Modular Connectors, Wire Stripper, Professional Ratcheting Crimping Tool, and an RJ-45 Digital Cable Tester.

Step 1: Strip Cable Jacket: Strip approximately 1 to 1.5 inches of outer PVC jacket using the stripper blade, ensuring individual inner wire insulation is not nicked.
Step 2: Untwist and Straighten: Untwist the 4 color pairs and fan out each conductor straight.
Step 3: Align According to Standard: Arrange conductors in exact T568B order: White/Orange, Orange, White/Green, Blue, White/Blue, Green, White/Brown, Brown.
Step 4: Trim Conductors Evenly: Use the cutting blade to trim all conductors flush at a 90-degree angle, leaving approximately 0.5 inches exposed.
Step 5: Insert into RJ-45 Plug: Carefully slide the aligned wires into the RJ-45 connector with the retention tab facing downward. Ensure all conductors reach the end of the plug channels under the gold-plated pins, and the outer jacket extends past the internal strain-relief wedge.
Step 6: Crimp the Modular Connector: Insert the plug firmly into the 8P cavity of the crimping tool and squeeze the handles firmly until the ratchet mechanism releases.
Step 7: Continuity & Pinout Verification: Insert both ends into the master and remote units of an RJ-45 cable tester. Observe LEDs 1 through 8 cycling sequentially in green.

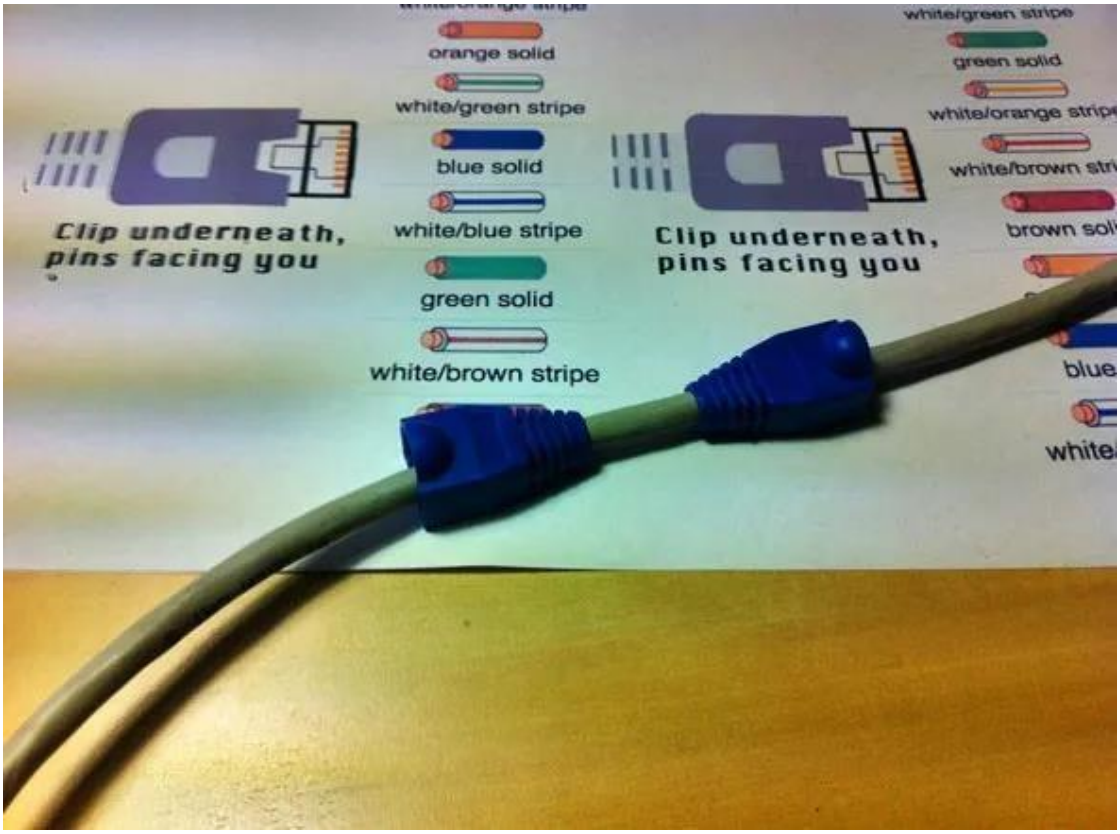


Figure 2.8: Stripping the Outer Protective Cable Jacket

Figure 2.9: Aligning and Straightening the Color-Coded Pairs

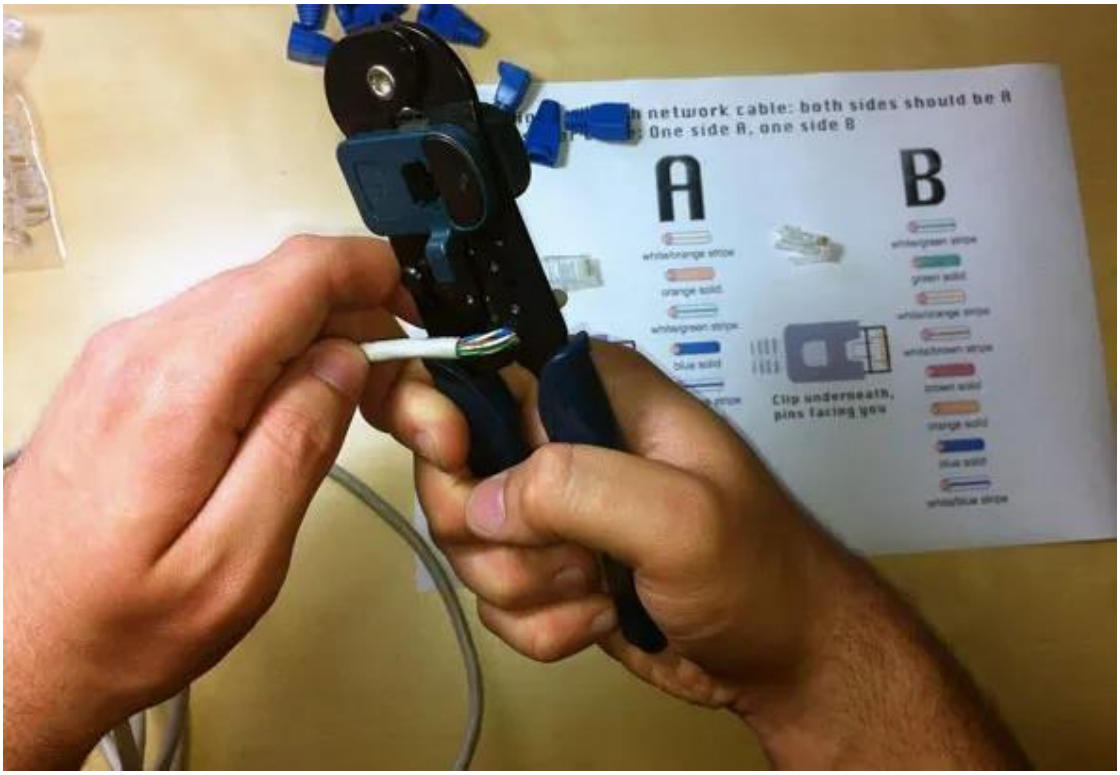


Figure 2.10: Trimming Conductors Flush at 90 Degrees

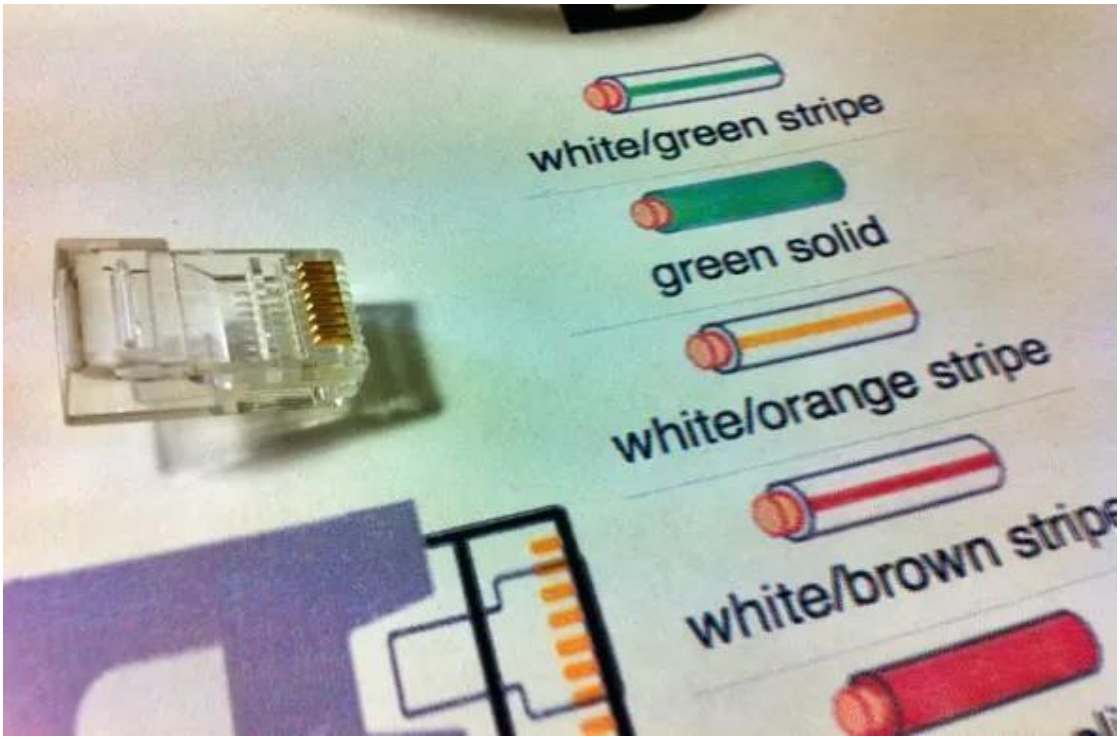


Figure 2.11: Fully Inserting Wires into RJ-45 Connector Channels

Figure 2.12: Modular Crimping Tool Alignment

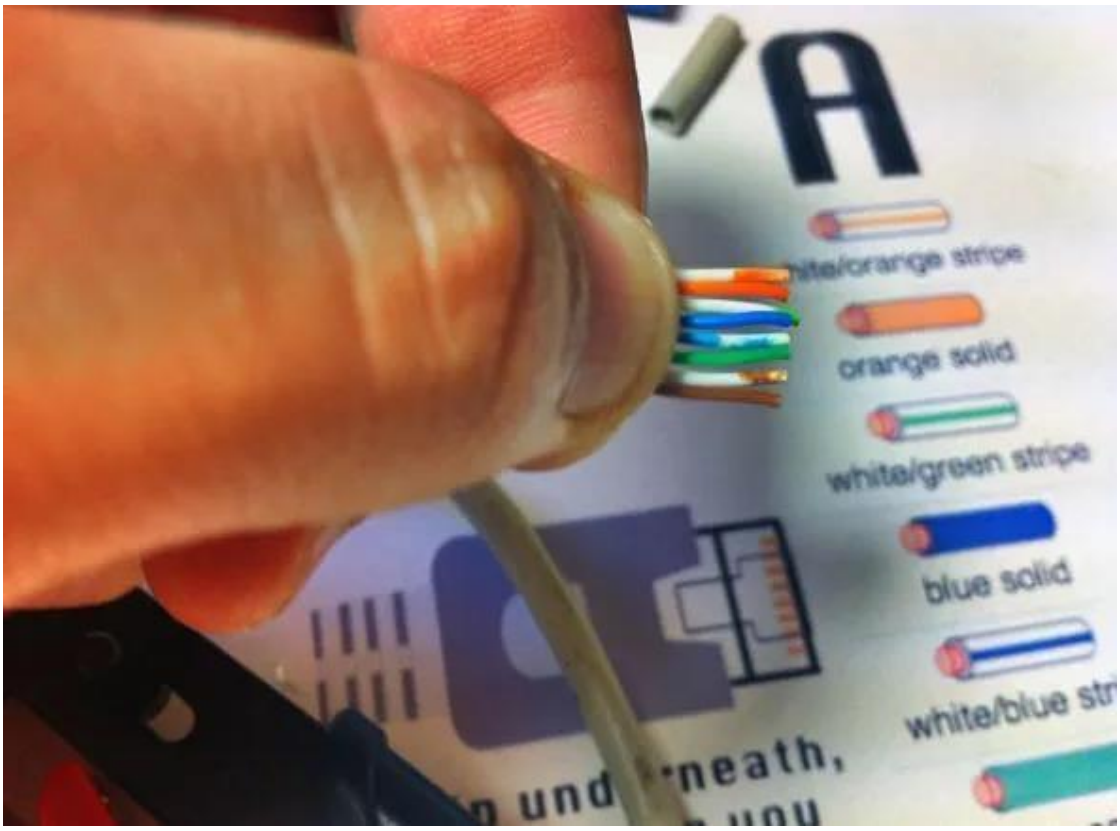


Figure 2.13: Squeezing the Crimping Tool to Drive Gold Pins into Conductors

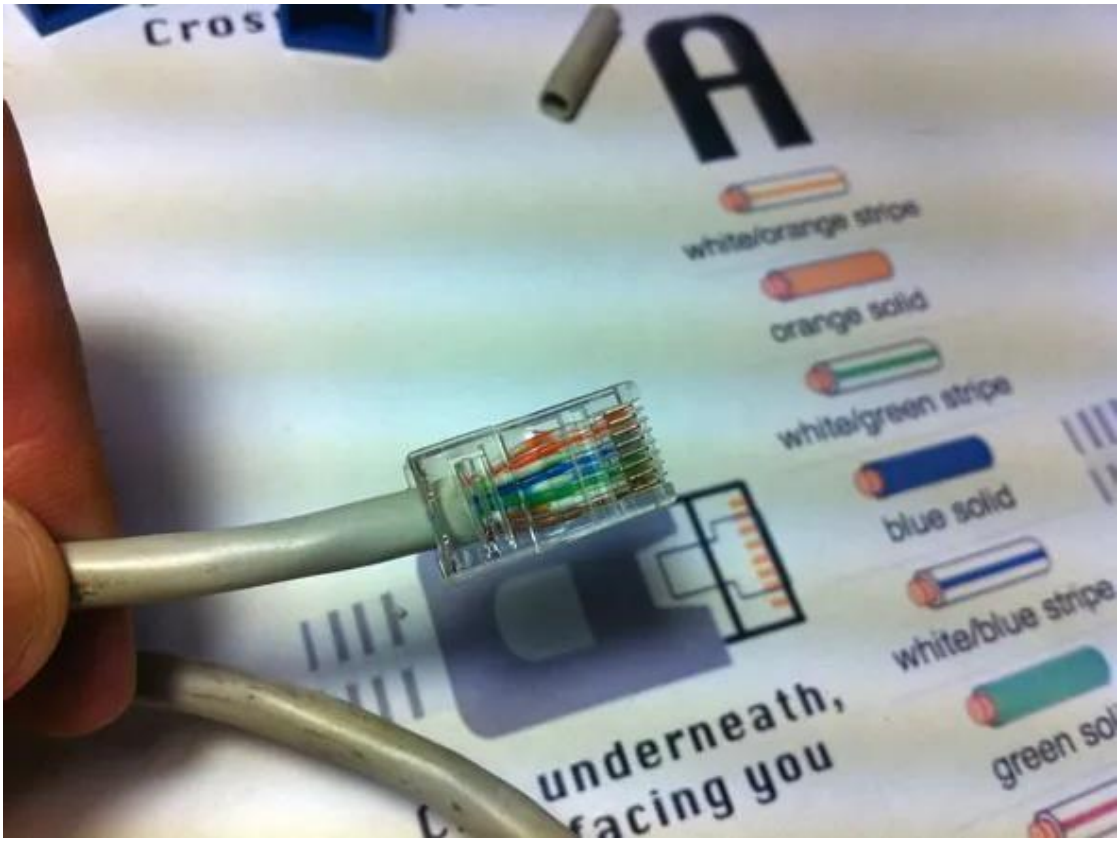


Figure 2.14: Completed Straight-Through and Crossover Patch Cables

EXPERIMENT 3: NETWORK DEVICE SIMULATION USING CISCO PACKET TRACER (HUB VS. SWITCH)

AIM: To explore the Cisco Packet Tracer simulation environment, design small multi-node local networks using Hubs and Switches, and systematically observe the fundamental differences in Protocol Data Unit (PDU) forwarding, collision handling, and address learning.

3.1 Cisco Packet Tracer Environment Overview

Cisco Packet Tracer is a visual network simulation platform developed by Cisco Systems. It provides a comprehensive workspace with simulated networking hardware (routers, switches, hubs, wireless access points, end devices) and dual operational modes:

1. Realtime Mode: Network protocols exchange messages and update routing/switching state tables in normal wall-clock time.
2. Simulation Mode: Freezes time and allows frame-by-frame inspection of Protocol Data Units (PDUs) traversing links, enabling granular packet header inspection at Layers 1 through 7.

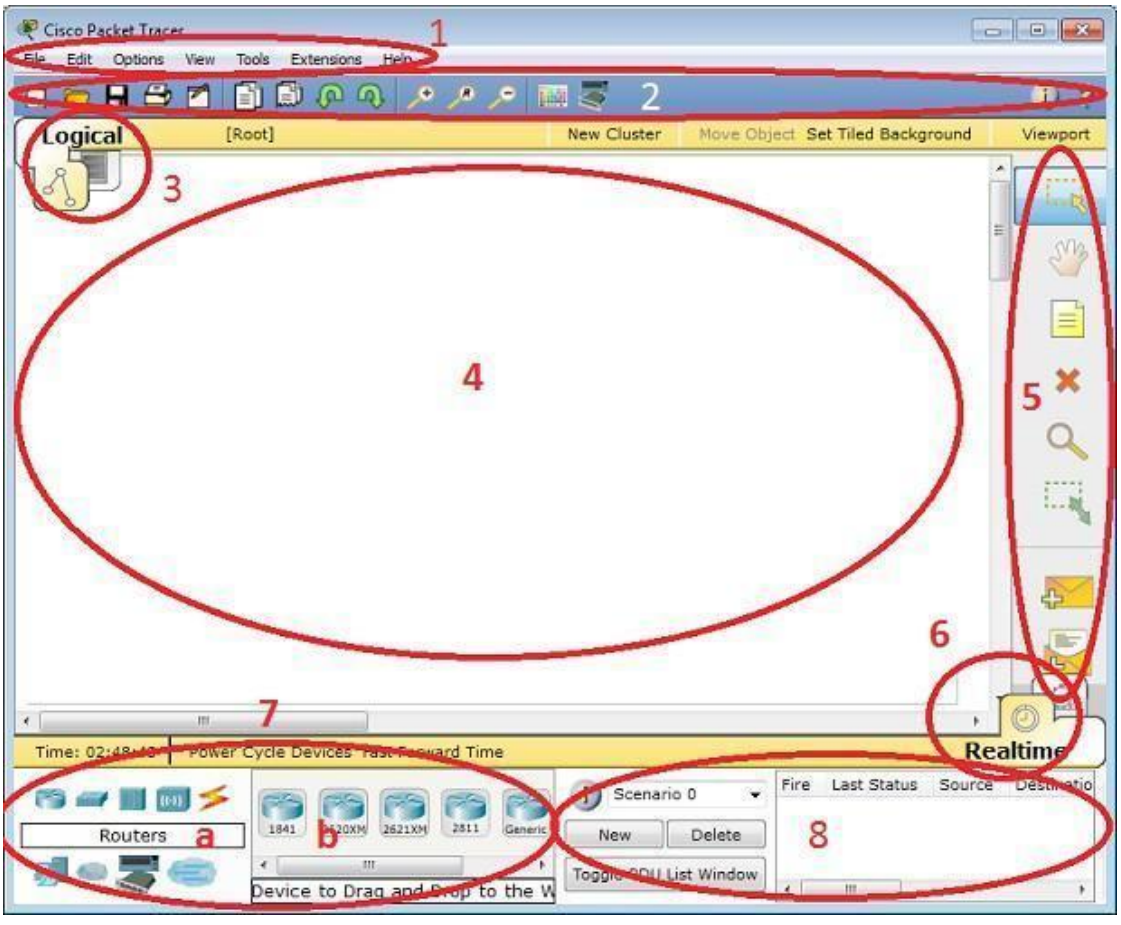


Figure 3.1: Cisco Packet Tracer Main Workspace and Device Selector Pane

3.2 Network Setup 1: Hub-Based Star Topology

Procedure:

1. Place a generic 4-port/6-port Ethernet Hub (PT-HUB) in the center of the logical workspace.
2. Connect 4 to 6 PC workstations (PC0, PC1, PC2, PC3) to the Hub using Copper Straight-Through cables.
3. Assign static IP addresses within the subnet 192.168.1.0/24:
   - PC0: 192.168.1.10 | Subnet Mask: 255.255.255.0
   - PC1: 192.168.1.11 | Subnet Mask: 255.255.255.0
   - PC2: 192.168.1.12 | Subnet Mask: 255.255.255.0
   - PC3: 192.168.1.13 | Subnet Mask: 255.255.255.0
4. Switch to Simulation Mode. Using the 'Add Simple PDU' tool, inject an ICMP ping from PC0 (source) to PC1 (destination).
5. Advance simulation: The hub receives the bitstream on port 0 and blindly regenerates/floods the frame out of every other connected port (ports 1, 2, and 3). PC2 and PC3 receive the frame, inspect the destination MAC address, determine it does not match their own MAC, and drop the frame (indicated by a red cross in simulation). Only PC1 processes the ICMP Echo Request and generates an ICMP Echo Reply, which the hub once again broadcasts to all connected hosts.

Figure 3.2: Hub-Based Local Network Topology in Packet Tracer

3.3 Network Setup 2: Switch-Based Star Topology

Procedure:

1. Replace the Hub with a Cisco Catalyst 2960 24-Port FastEthernet Switch.
2. Connect the workstations to switch ports FastEthernet0/1 through FastEthernet0/4.
3. In Simulation Mode, transmit an ICMP PDU from PC0 to PC1.
4. Observation: Initially, if the switch MAC address table is empty, the switch broadcasts the ARP request frame to learn which port holds the target MAC. Upon receiving the unicast ARP reply, the switch records PC1's MAC address mapped to port Fa0/2. Subsequent ICMP Echo Request packets from PC0 are forwarded *strictly* out of Fa0/2. PC2 and PC3 receive zero unnecessary traffic, eliminating idle line collisions.

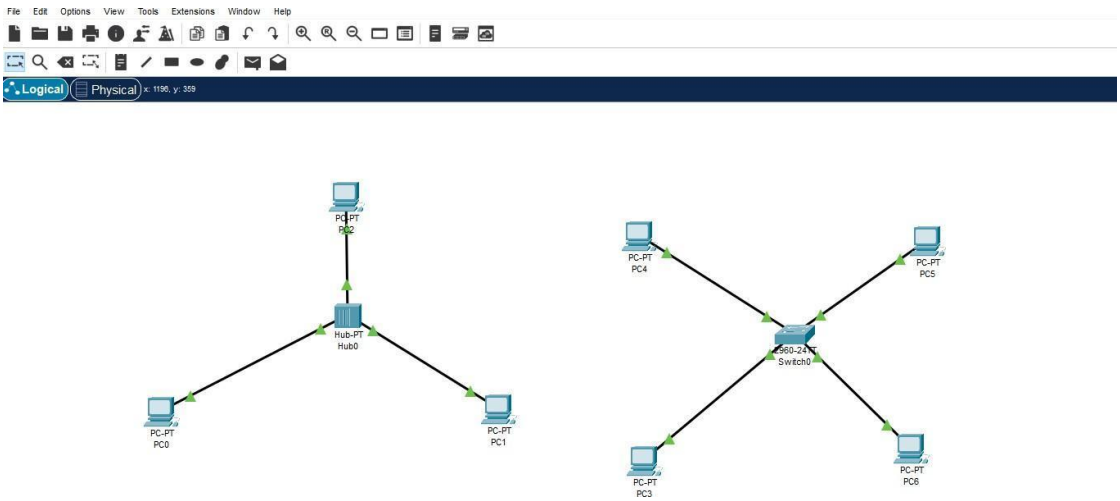


Figure 3.3: Switch-Based Local Network Topology in Packet Tracer

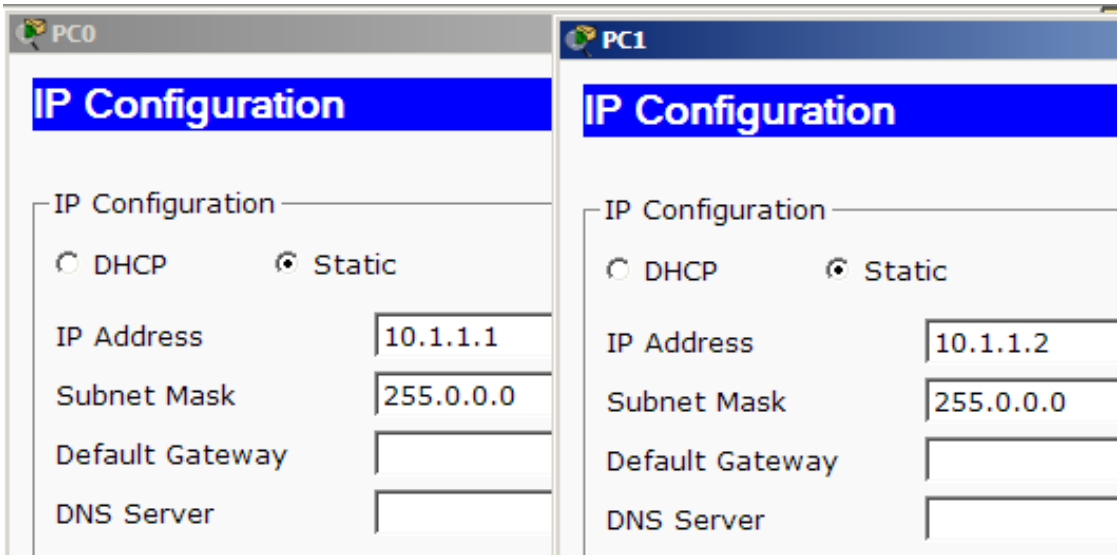


Figure 3.4: PDU Simulation Trace Demonstrating Unicast Forwarding on Switch vs Flooding on Hub

3.4 Comparative Analysis: Hub vs. Switch

Evaluation Metric	Network Hub (Multiport Repeater)	Network Switch (Bridge)
OSI Layer Operation	Layer 1 (Physical Layer)	Layer 2 (Data Link Layer)
Data Forwarding Mechanism	Electrical signal flooding across all active ports	Frame inspection and selective unicast via MAC Table
Collision Domains	Single shared collision domain across all ports	Each individual port constitutes an isolated collision domain
Broadcast Domains	Single broadcast domain	Single broadcast domain (unless partitioned by VLANs)

Duplex Mode Support	Half-duplex only (CSMA/CD arbitration)	Full-duplex dedicated throughput without collisions
Bandwidth Allocation	Bandwidth shared among all active transmitting nodes	Dedicated full-wire speed bandwidth per port interface

EXPERIMENT 4: LOCAL AREA NETWORK (LAN) SETUP & RESOURCE SHARING USING SWITCH

AIM: To establish a functional physical Local Area Network (LAN) using a multi-port Ethernet switch and UTP cables, configure consistent IPv4 addressing, verify bidirectional reachability via ping, and configure secure peer-to-peer file and folder sharing across workstations.

4.1 LAN Architecture & Addressing Scheme

A Local Area Network (LAN) interconnects computers and network resources within a bounded physical area (laboratory, office suite, or classroom). In a modern star-wired switch topology, each host connects via a dedicated Cat 5e/Cat 6 patch cord to a centralized network switch. To enable communication without a router, all participating machines must reside within the identical IPv4 logical subnet.

Host Designation	Physical Interface	Assigned IPv4 Address	Subnet Mask	Assigned Workgroup
Lab PC 1 (Host A)	GigabitEthernet 0	192.168.10.1	255.255.255.0	CN_LAB_GROUP
Lab PC 2 (Host B)	GigabitEthernet 0	192.168.10.2	255.255.255.0	CN_LAB_GROUP
Lab PC 3 (Host C)	GigabitEthernet 0	192.168.10.3	255.255.255.0	CN_LAB_GROUP
Lab PC 4 (Host D)	GigabitEthernet 0	192.168.10.4	255.255.255.0	CN_LAB_GROUP

4.2 Step-by-Step Configuration Steps

1. Physical Interconnection: Connect RJ-45 patch cables from the Ethernet port of each PC into ports 1 through 4 of the laboratory switch.
2. IP Address Assignment: Open Network and Sharing Center -> Change adapter settings -> Right-click Ethernet -> Properties -> Internet Protocol Version 4 (TCP/IPv4) -> Properties. Select 'Use the following IP address' and assign the static parameters specified in Table 4.1.
3. Connectivity Testing: From Host A's command prompt, execute ping sweeps to test reachability:
   > ping 192.168.10.2
   > ping 192.168.10.3
   Verify 0% packet loss and low latency (<1ms).
4. Advanced Sharing Settings: Under Network and Sharing Center -> Advanced sharing settings, enable 'Turn on network discovery' and 'Turn on file and printer sharing'.
5. Folder Sharing Configuration: Create a folder named 'CN_Lab_Share' on Host A. Right-click -> Properties -> Sharing tab -> Advanced Sharing -> Check 'Share this folder' -> Permissions -> Grant Read/Write access to 'Everyone'.

6. Remote Access: On Host B, press Windows+R, enter '\\192.168.10.1\CN_Lab_Share', and confirm seamless file reading and writing across the local network.

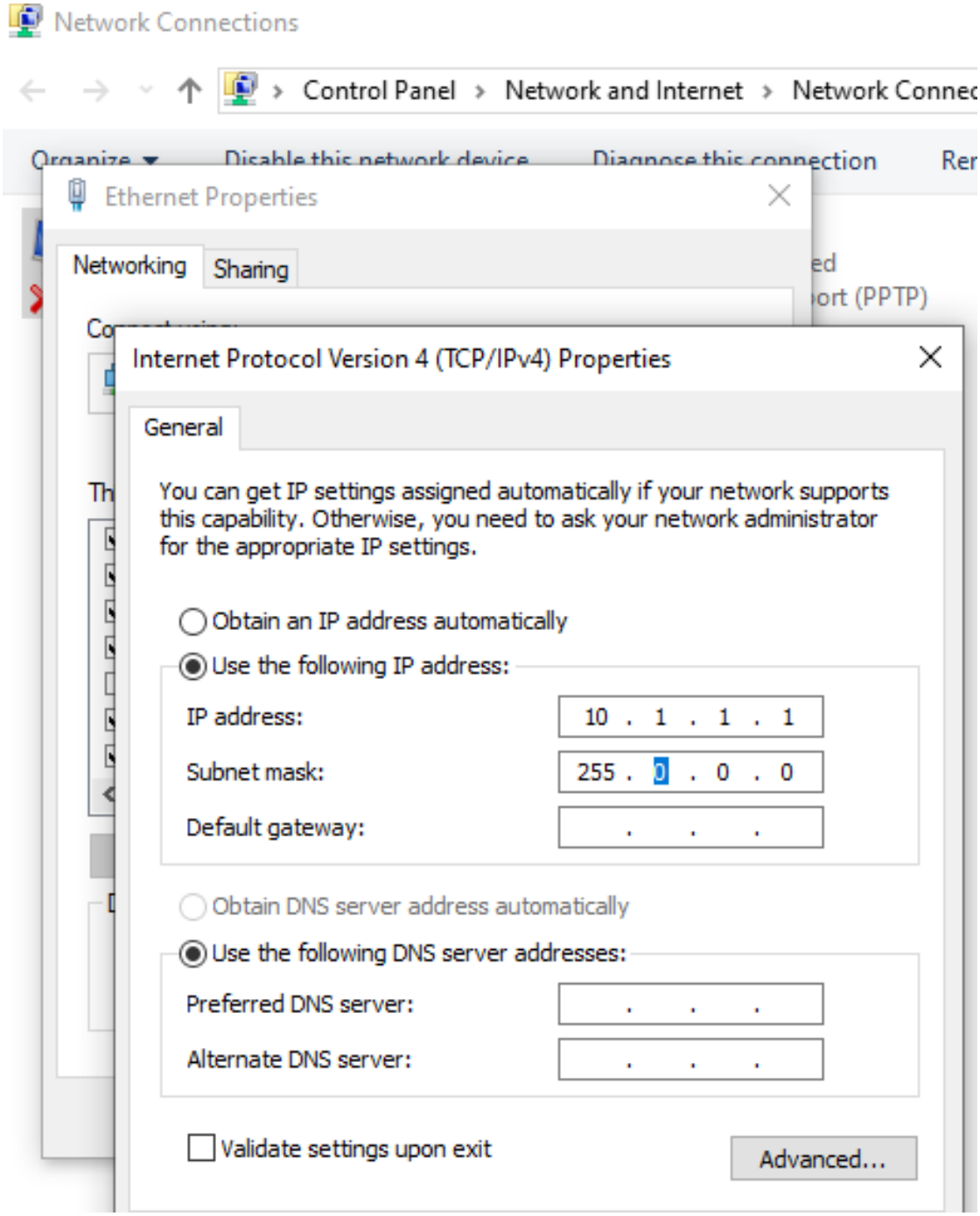


Figure 4.1: Windows Advanced Sharing and Network Discovery Configuration Interface

EXPERIMENT 5: PACKET CAPTURE & PROTOCOL STACK INSPECTION USING WIRESHARK

AIM: To master the operational features of Wireshark as an industry-standard packet analyzer, capture live network traffic, dissect protocol encapsulation across the TCP/IP stack (Ethernet, IP, TCP, UDP, ICMP, DNS, HTTP), and apply granular capture and display filters.

5.1 Packet Sniffing Principles & Wireshark Architecture

A packet sniffer (protocol analyzer) operates passively by capturing copies of raw data frames traversing a host's Network Interface Card (NIC). Normally, a NIC operates in non-promiscuous mode, discarding any incoming frame whose destination MAC address does not match its own burned-in address or a designated multicast/broadcast group. Wireshark leverages packet capture libraries (Npcap on Windows, libpcap on Linux) to set the interface into 'Promiscuous Mode', capturing all frames broadcast on the local link.

Figure 5.1: Packet Sniffer Interaction with Network Stack and Physical Medium

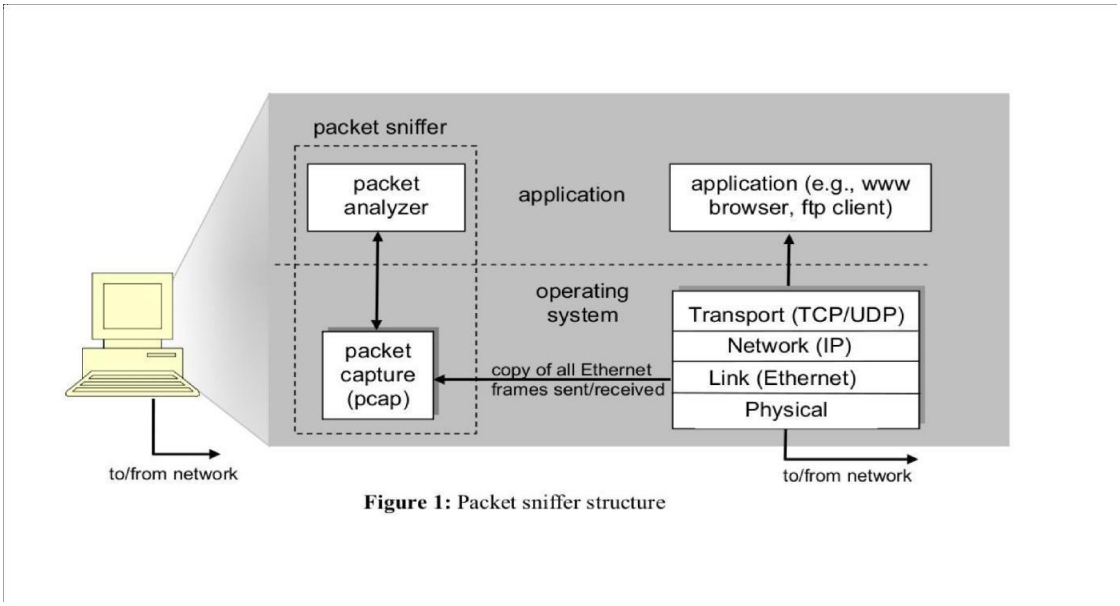


Figure 1: Packet sniffer structure

Figure 5.2: Wireshark Packet Capture Engine Architecture

5.2 Wireshark Three-Pane Graphical User Interface

Wireshark organizes captured packet telemetry across three coordinated panes:

1. Packet List Pane (Top): Displays a chronological list of all captured frames, summarizing Frame Number, Timestamp, Source IP, Destination IP, Protocol, Length, and Info.
2. Packet Details Pane (Middle): Displays an interactive hierarchical breakdown of protocol headers for the currently selected frame, mapped to the OSI/TCP-IP reference models (Frame -> Ethernet II -> IPv4 -> TCP/UDP -> Application Payload).
3. Packet Bytes Pane (Bottom): Displays the exact raw payload in traditional hex dump format alongside ASCII character representations.

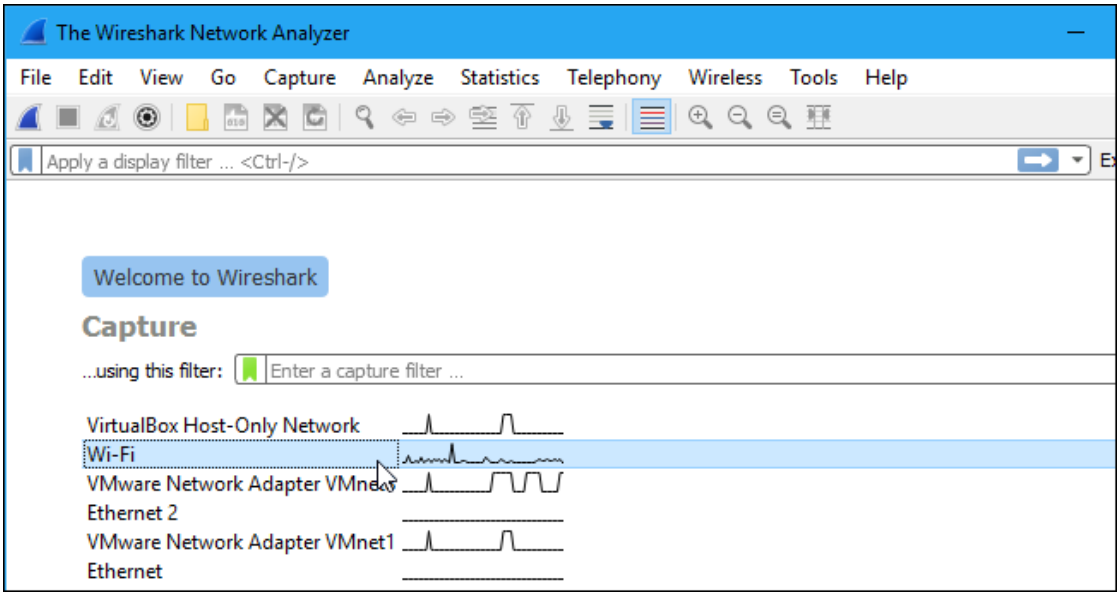


Figure 5.3: Wireshark Main 3-Pane Graphical Analysis Interface

Figure 5.4: Interface Selection for Packet Capture

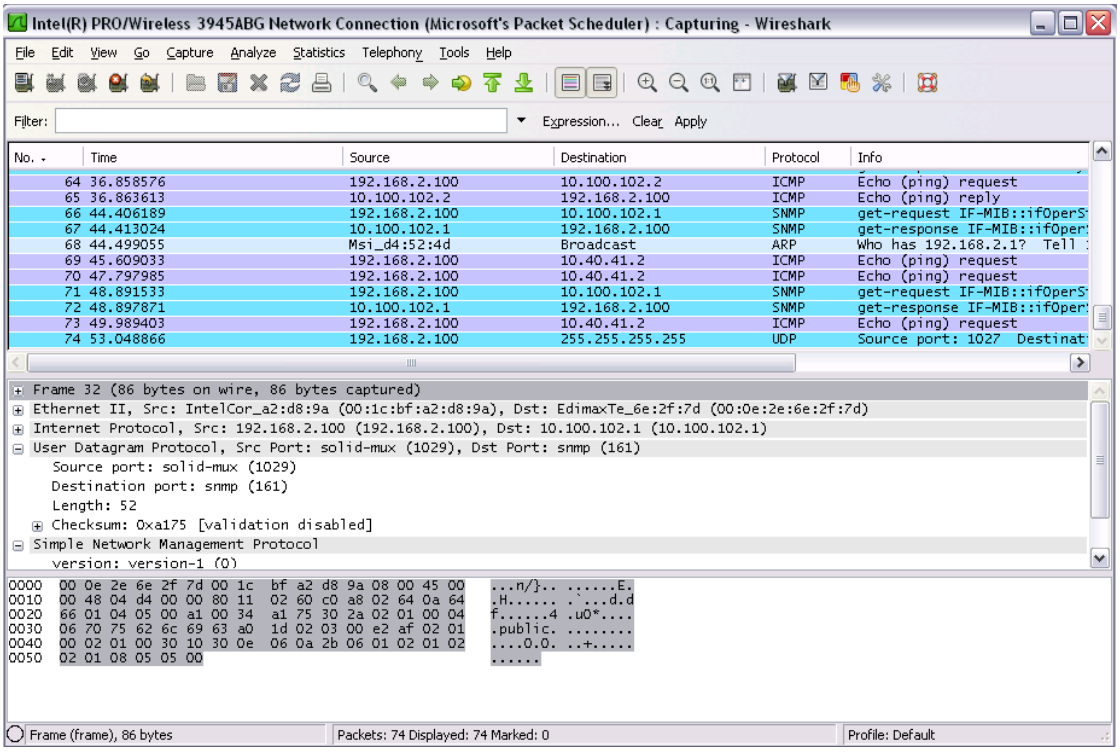


Figure 5.5: Starting Live Packet Capture on Active Interface

Figure 5.6: Stopping Live Capture

Figure 5.7: Restarting Active Capture

Figure 5.8: Capture Options and Buffer Configuration

Figure 5.9: Saving Capture Trace to PCAPNG Format

Figure 5.10: Opening Saved Packet Trace Files

5.3 Deep Dissection of Protocol Headers

Through Wireshark's Packet Details pane, each encapsulation header can be examined in detail:

1. Ethernet II Frame (Data Link Layer):
   - Destination MAC Address (6 bytes) & Source MAC Address (6 bytes)
   - EtherType (2 bytes): Identifies the encapsulated Network Layer protocol (e.g., 0x0800 for IPv4, 0x0806 for ARP, 0x86DD for IPv6).

2. Internet Protocol Version 4 (IPv4) Header (Network Layer):
   - Version (4 bits): Indicates IPv4.
   - Internet Header Length (IHL - 4 bits): Length of header in 32-bit words (typically 5, indicating 20 bytes).
   - Type of Service / DSCP (1 byte): Quality of service and packet priority markings.
   - Total Length (2 bytes): Entire packet size in bytes including header and data.
   - Identification, Flags (DF/MF), Fragment Offset: Support packet fragmentation.

- Time to Live (TTL - 1 byte): Hop count decremented by each router to prevent infinite loops.
- Protocol (1 byte): Identifies transport protocol (1 for ICMP, 6 for TCP, 17 for UDP).
- Header Checksum (2 bytes): Verifies IPv4 header integrity.
- Source and Destination IPv4 Addresses (4 bytes each).

3. Transmission Control Protocol (TCP) Header (Transport Layer):
- Source Port (2 bytes) & Destination Port (2 bytes): Identifies sending and receiving service processes.
- Sequence Number (4 bytes) & Acknowledgment Number (4 bytes): Provide reliable, ordered byte delivery.
- Data Offset (4 bits): TCP header length.
- Flags (9 bits): URG, ACK, PSH, RST, SYN, FIN control connection establishment, maintenance, and teardown.
- Window Size (2 bytes): Flow control credit indicating receiver buffer capacity.

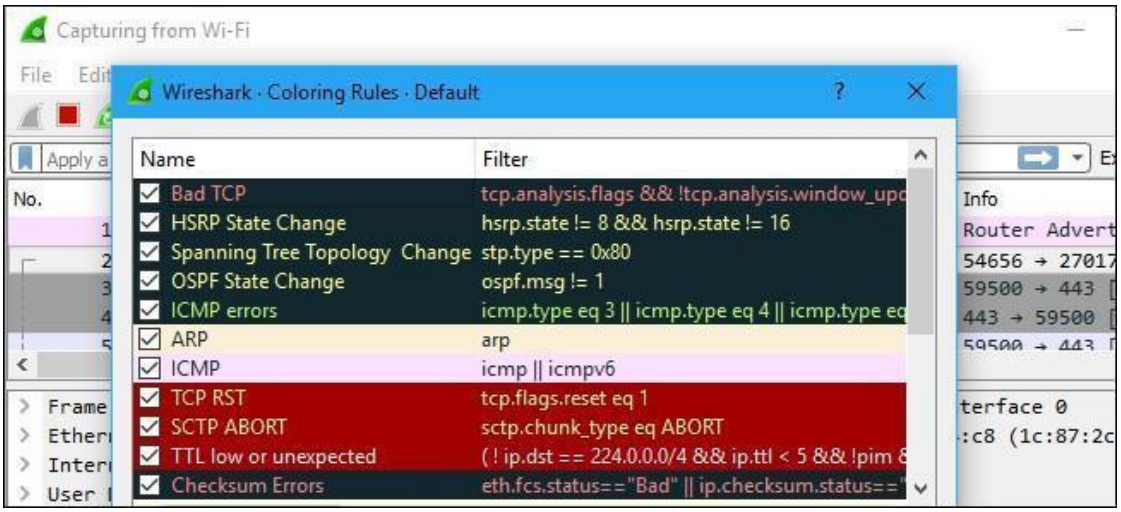


Figure 5.11: Ethernet II Header Inspection in Packet Details Pane

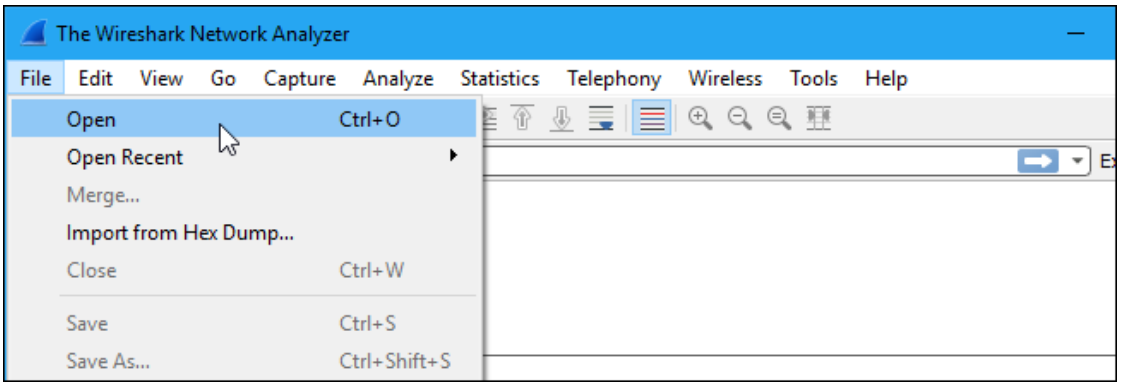


Figure 5.12: IPv4 Header Fields Dissection

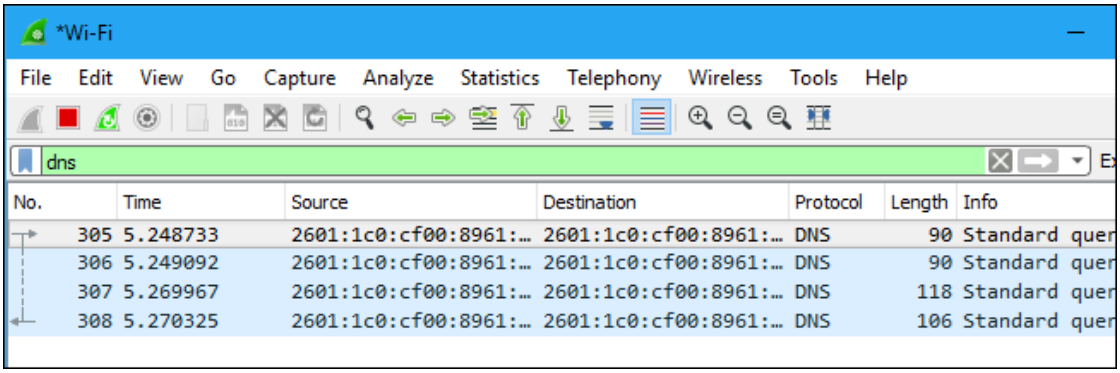


Figure 5.13: TCP Header Flags and Three-Way Handshake Breakdown

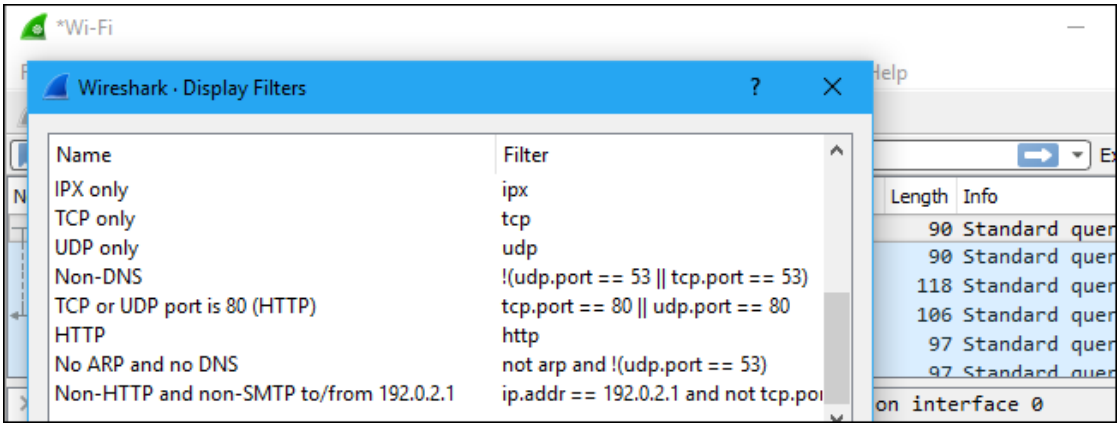


Figure 5.14: UDP Datagram Structure (Source/Dest Port, Length, Checksum)

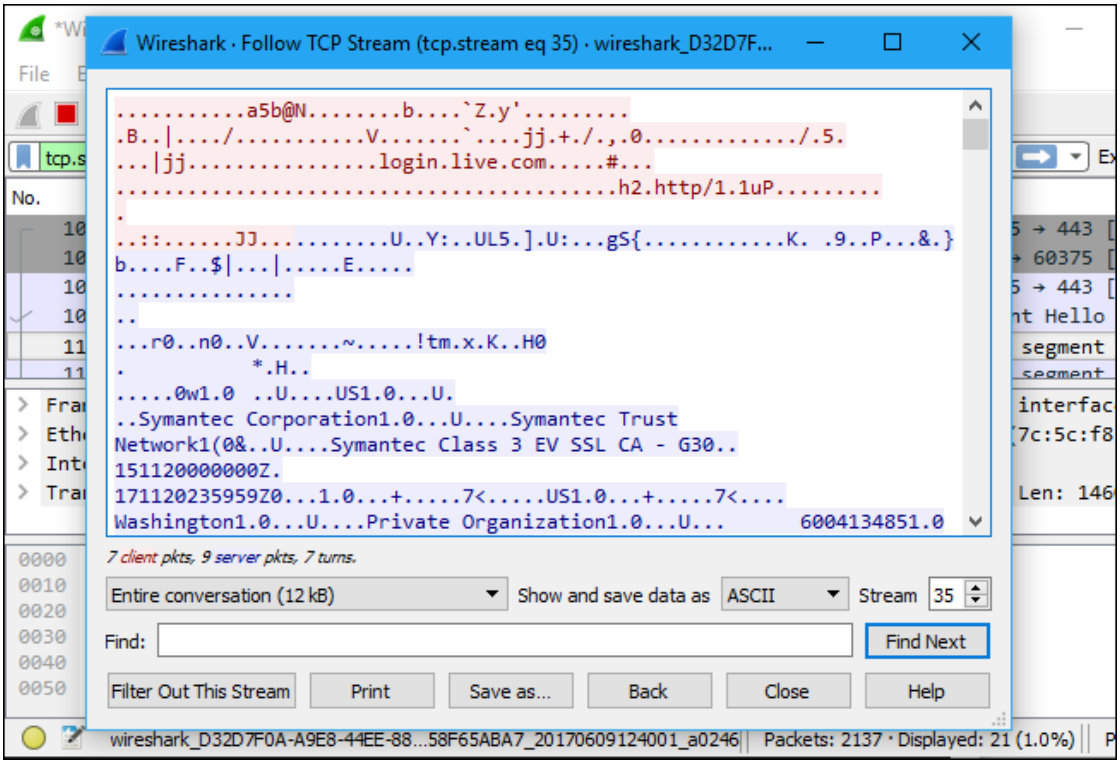


Figure 5.15: DNS Query/Response Transaction Encapsulated in UDP

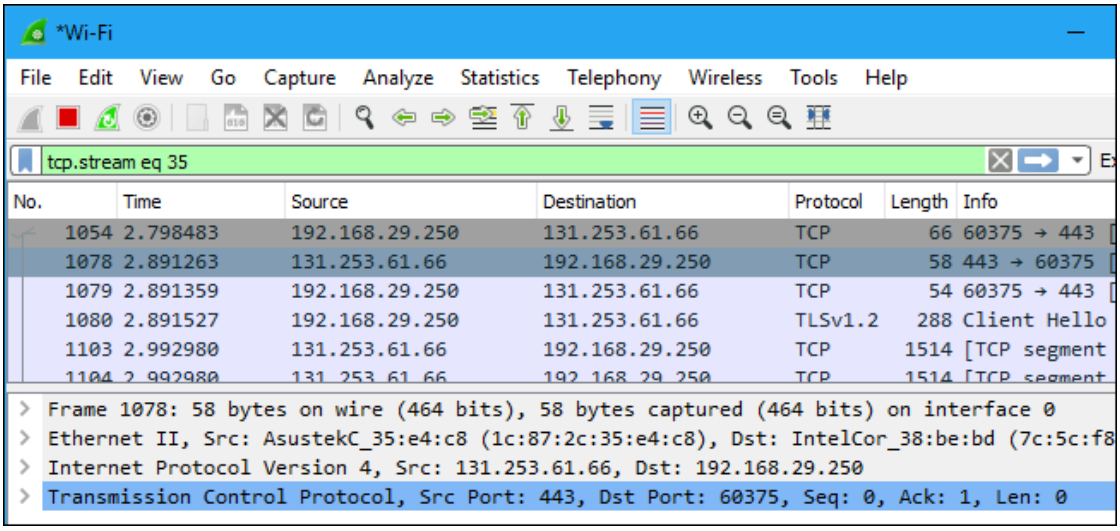


Figure 5.16: HTTP GET Request Method Header

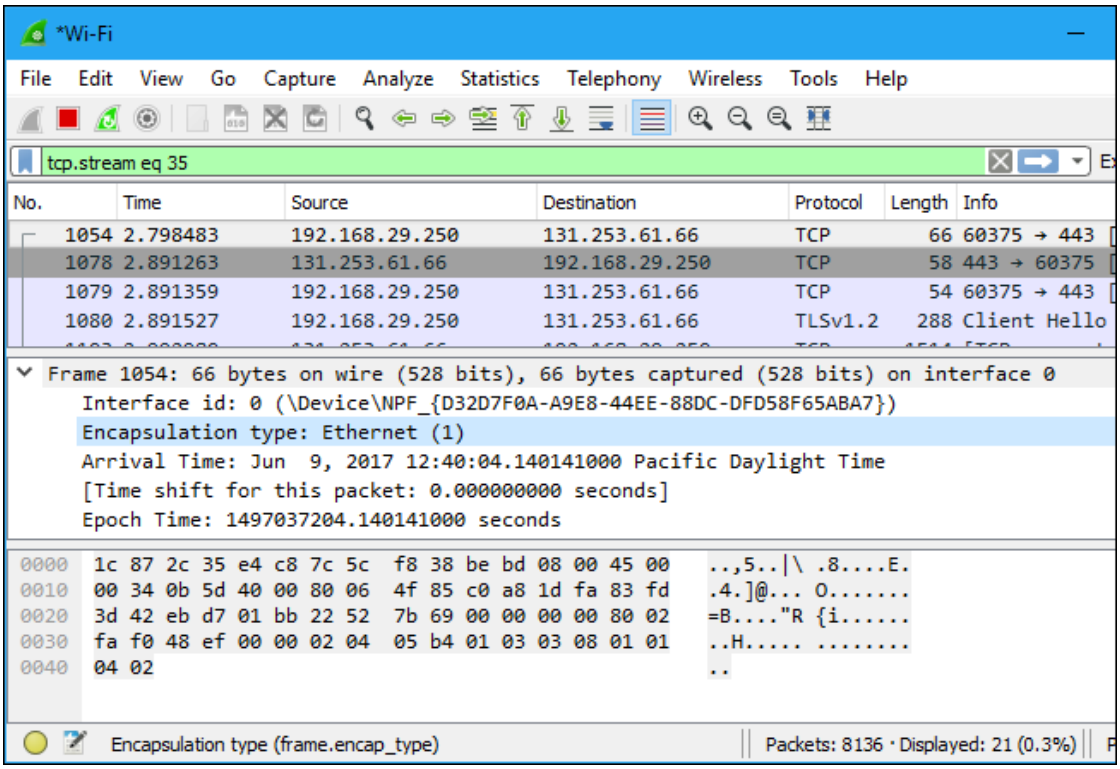


Figure 5.17: HTTP 200 OK Response Header with Web Payload

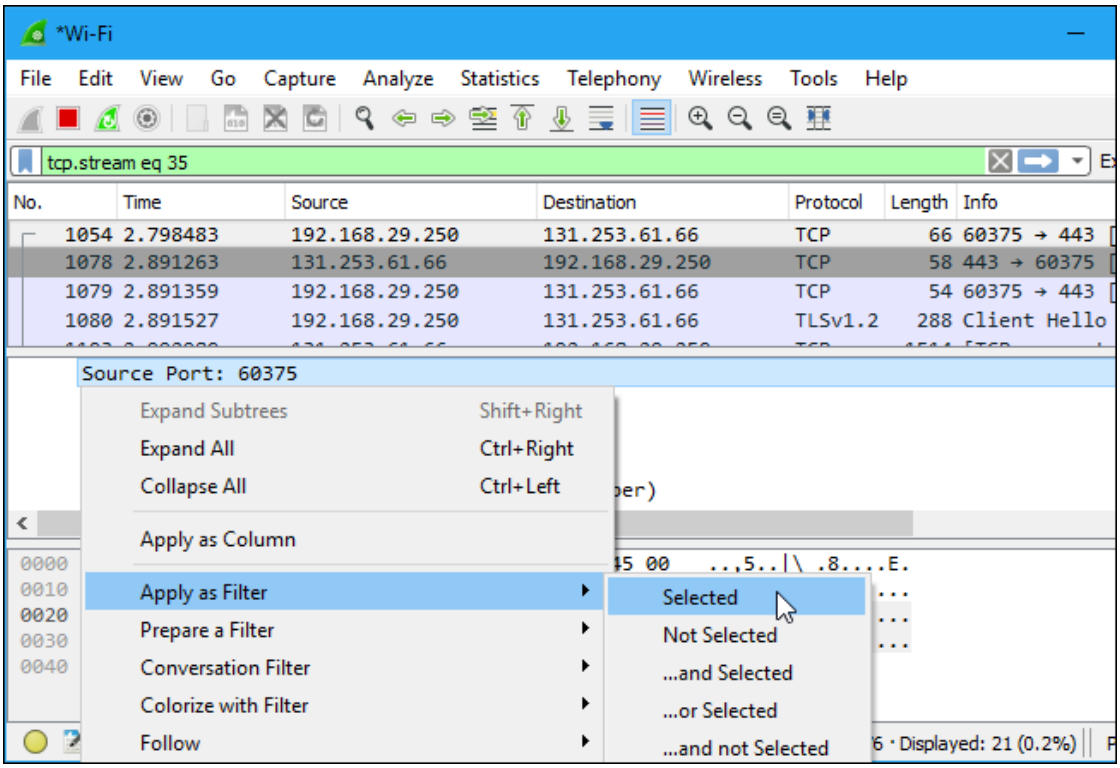


Figure 5.18: Packet Bytes Hex Dump with ASCII Decoding

Figure 5.19: Applying Display Filters in Wireshark Filter Bar

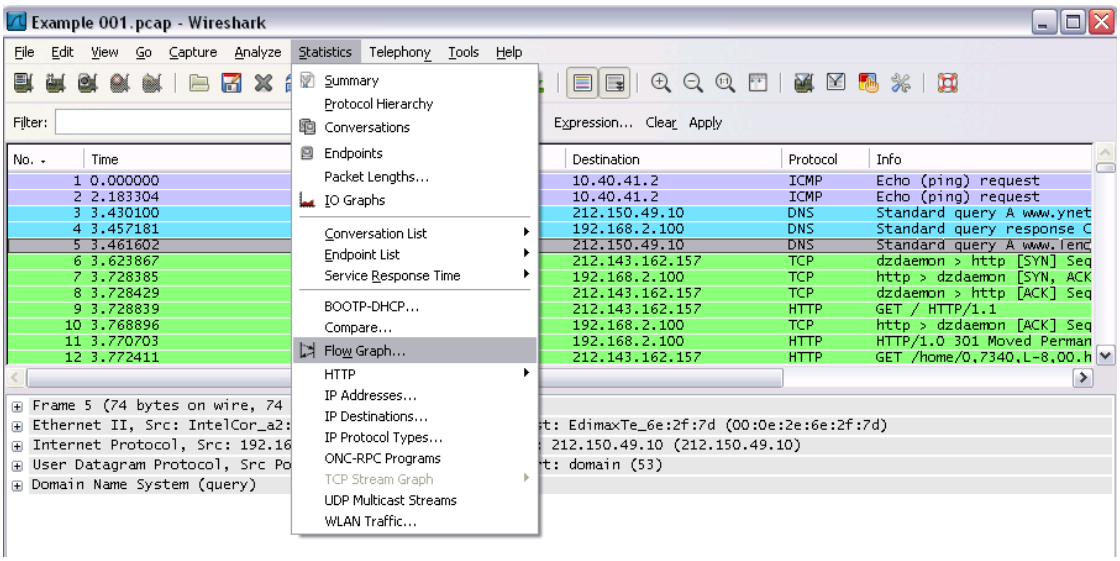


Figure 5.20: Protocol Hierarchy Statistics Window

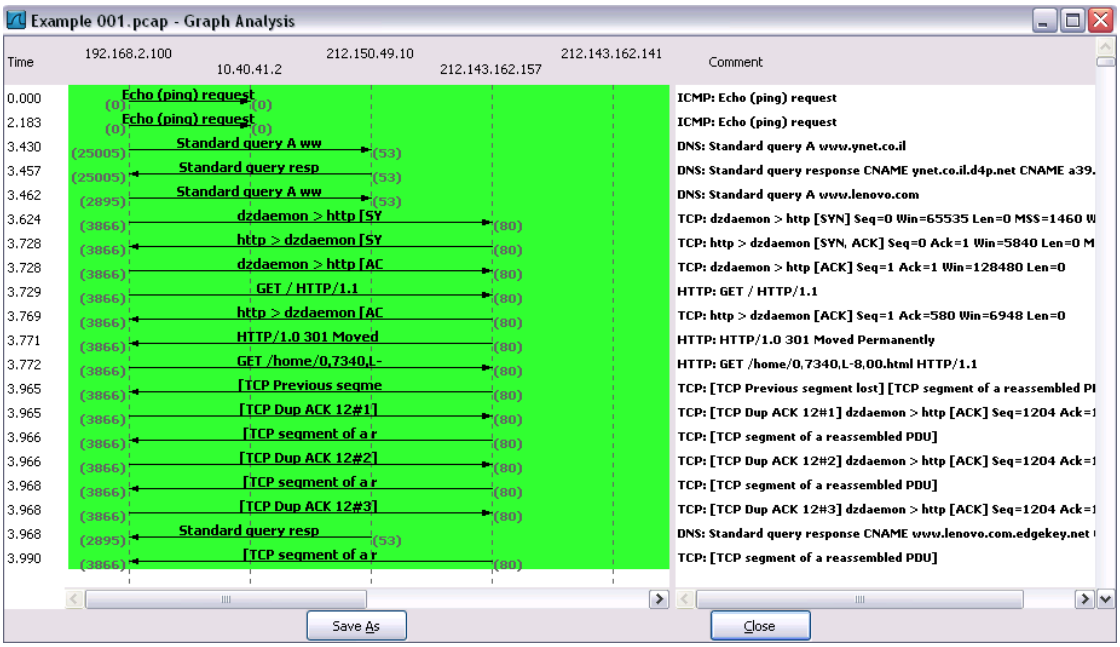


Figure 5.21: Conversations Statistics Overview (Ethernet/IP/TCP)

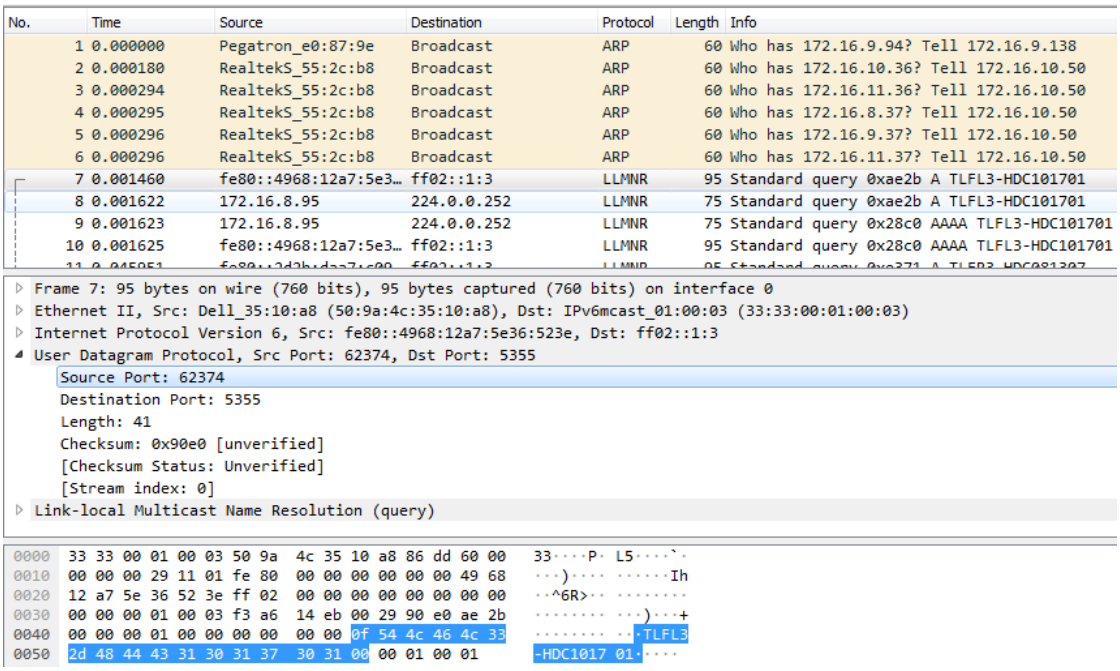


Figure 5.22: Following TCP Stream to Reassemble Application Data

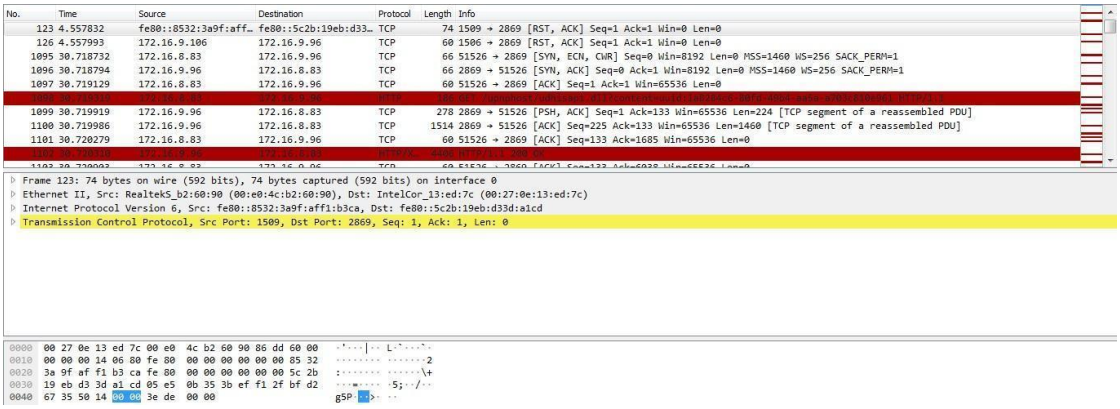


Figure 5.23: ICMP Echo Request Frame Breakdown

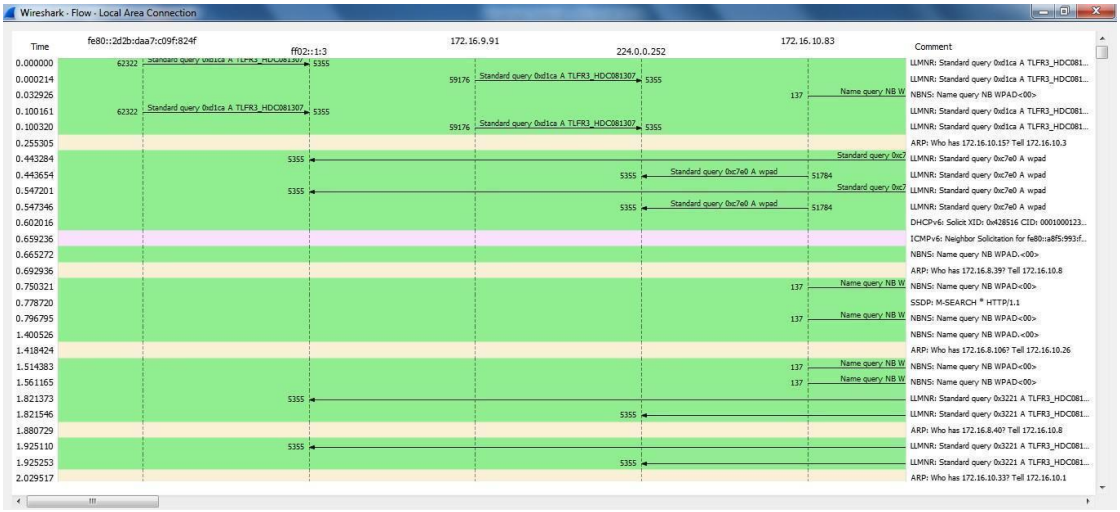


Figure 5.24: ICMP Echo Reply Verification

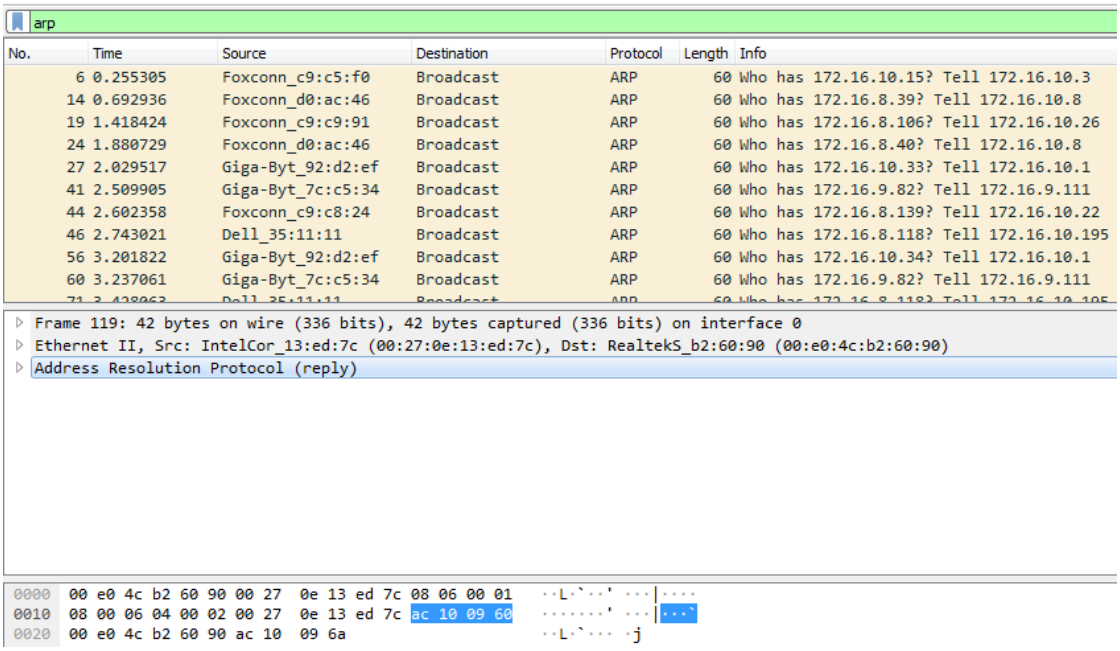


Figure 5.25: ARP Request Broadcast Frame Inspection

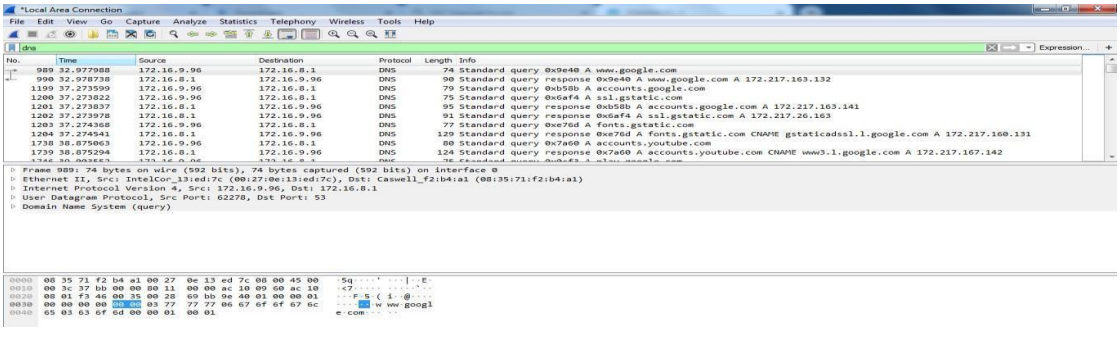


Figure 5.26: ARP Reply Unicast Resolution

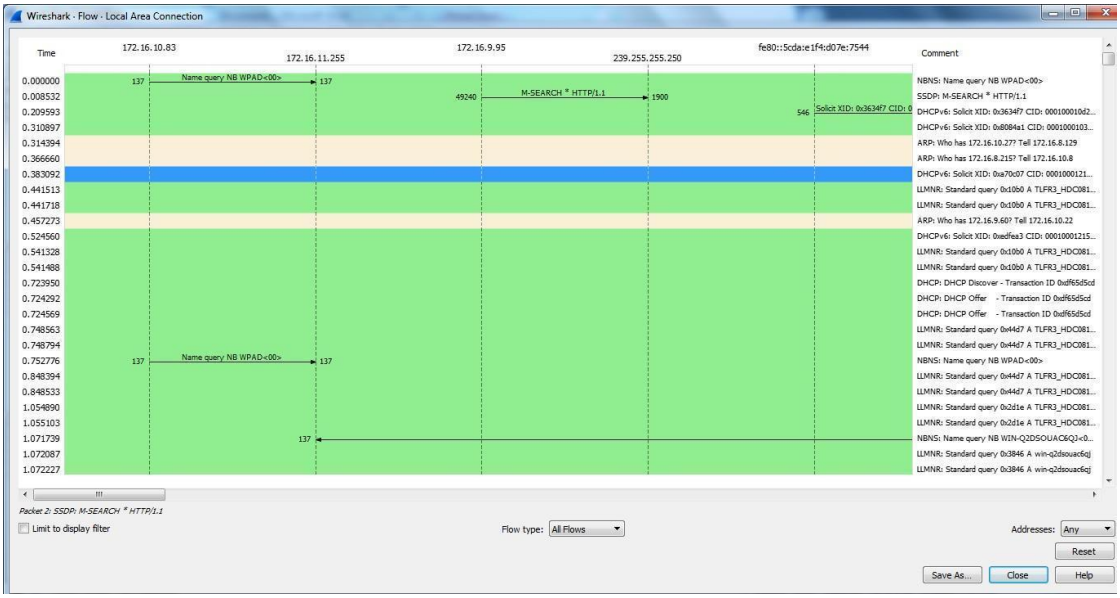


Figure 5.27: TCP SYN Packet Initiating Three-Way Handshake

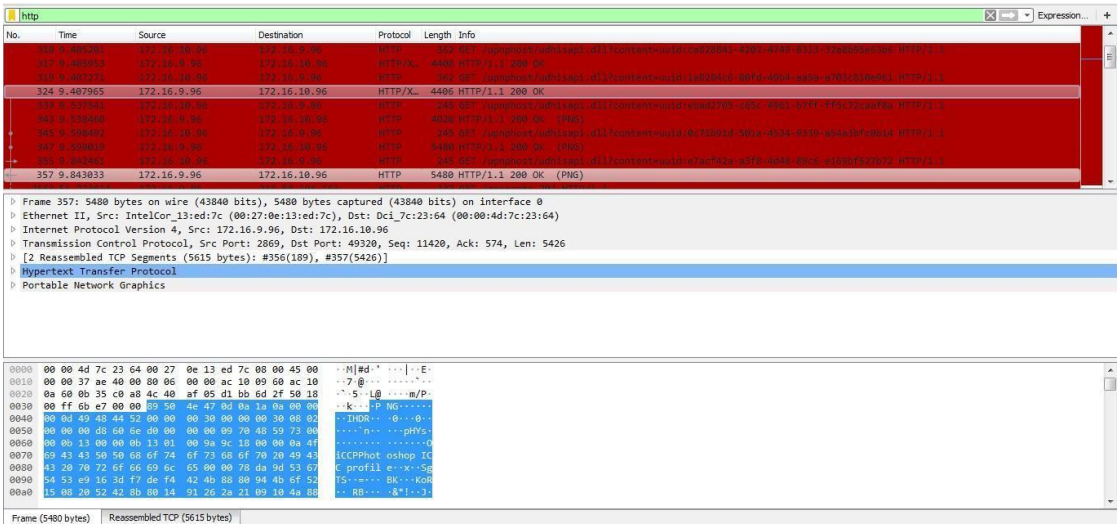


Figure 5.28: TCP SYN-ACK Response from Web Server

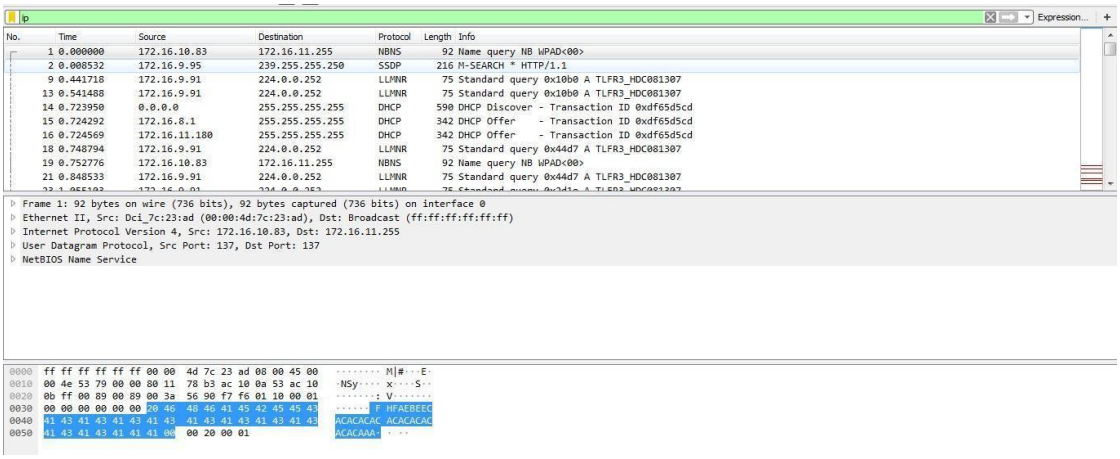


Figure 5.29: TCP ACK Completing Handshake Connection

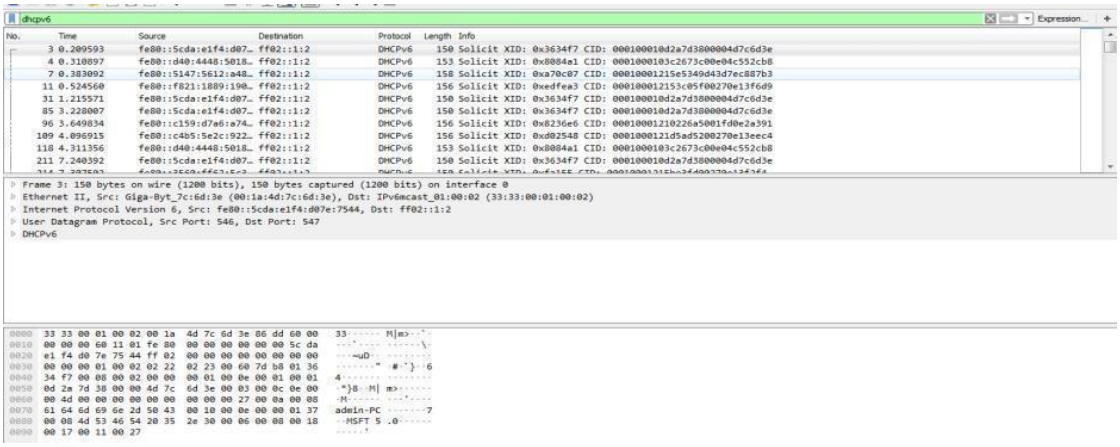


Figure 5.30: TCP FIN Handshake Gracefully Closing Connection

5.4 Practical Display Filter Cheat Sheet

Filter Syntax	Target Traffic Filtered
ip.addr == 192.168.1.1	Matches all packets where the IP is either the source or destination.
ip.src == 10.0.0.5 && ip.dst == 10.0.0.1	Matches traffic flowing specifically from host 10.0.0.5 to host 10.0.0.1.
tcp.port == 80 || tcp.port == 443	Filters for standard web traffic (HTTP or HTTPS).
http.request.method == "GET"	Isolates HTTP client GET request messages.
dns.flags.response == 0	Filters for DNS query requests transmitted by client machines.
icmp	Isolates all ping echo requests, replies, and unreachable notifications.

tcp.flags.syn == 1 && tcp.flags.ack == 0	Identifies initial TCP SYN connection attempt packets.

EXPERIMENT 6: ERROR CORRECTION AT DATA LINK LAYER (HAMMING CODE)

AIM: To implement single-bit error detection and automatic error correction at the Data Link Layer using the Hamming Code algorithm in Python, simulate bit-corruption across a communication channel file, and verify automated syndrome computation and data recovery.

6.1 Theoretical Concept & Mathematical Formulation

Transmission impairments across physical media frequently introduce bit errors into traversing data frames. While simple parity checks detect single-bit errors without identifying their location, Richard W. Hamming developed an ingenious linear block code that simultaneously detects and pinpoints the precise bit in error, enabling automatic correction.

1. Redundant Parity Bits Allocation:
For a dataword of length 'm' bits, 'r' parity bits are introduced such that:
2^r >= m + r + 1
Parity bits are positioned at bit indices that are integer powers of 2 (i.e., bit positions 1, 2, 4, 8, 16, ...). All remaining positions represent the actual data bits.

2. Parity Calculation (Even Parity):
Each parity bit P_i covers a specific set of bit positions whose binary representation has a 1 in the i-th bit position:
- P1 (position 1 = 0001_2): checks positions 1, 3, 5, 7, 9, 11, 13, 15, ...
- P2 (position 2 = 0010_2): checks positions 2, 3, 6, 7, 10, 11, 14, 15, ...
- P4 (position 4 = 0100_2): checks positions 4, 5, 6, 7, 12, 13, 14, 15, ...
- P8 (position 8 = 1000_2): checks positions 8, 9, 10, 11, 12, 13, 14, 15, ...

3. Syndrome Evaluation at Receiver:
Upon receiving a codeword, the receiver re-evaluates parity equations to yield syndrome bits (S_r ... S_2 S_1). If S = 0, the codeword is error-free. If S != 0, the binary integer value of S directly indicates the 1-based index of the corrupted bit! Inverting that bit completely corrects the error.

6.2 Step-by-Step Algorithm

Sender Algorithm:
1. Accept text message string from user.
2. Convert each ASCII character into an 8-bit binary string.
3. Divide binary stream into 4-bit nibbles (m = 4). With m=4, r=3 parity bits are required (Total codeword n = 7).
4. Construct 7-bit codeword: Positions P1=1, P2=2, D1=3, P4=4, D2=5, D3=6, D4=7.
5. Calculate even parity for P1, P2, P4:

P1 = D1 ^ D2 ^ D4
P2 = D1 ^ D3 ^ D4
P4 = D2 ^ D3 ^ D4
6. Write all generated 7-bit codewords into the channel file ('channel.txt').

Channel Noise Simulation:
1. Prompt the user or simulate random noise by flipping a chosen bit at index 'k' in 'channel.txt'.

Receiver Algorithm:
1. Read the transmitted codewords from 'channel.txt'.
2. For each 7-bit codeword (b1, b2, b3, b4, b5, b6, b7), evaluate syndrome bits:
S1 = b1 ^ b3 ^ b5 ^ b7
S2 = b2 ^ b3 ^ b6 ^ b7
S4 = b4 ^ b5 ^ b6 ^ b7
3. Calculate Error Syndrome Position: S = (S4 * 4) + (S2 * 2) + (S1 * 1).
4. If S == 0: Report block is error-free.
5. If S > 0: Report single-bit error detected at position S. Invert bit at index S to correct the corrupted codeword.
6. Strip parity bits P1, P2, P4 to recover original 4-bit data (D1, D2, D3, D4).
7. Recombine nibbles into 8-bit bytes, decode back into ASCII characters, and display the verified message.

Source Code: Hamming Code (7, 4) Sender & Receiver Simulation

```
# ====================================================================
# CS23532 Computer Networks Lab - Experiment 6
# Implementation of Hamming Code (7, 4) Error Detection and Correction
# Student Name: THANIKAIVEL S | Reg No: 2116240701562
# ====================================================================
import os

CHANNEL_FILE = "channel.txt"

def encode_hamming_7_4(data_nibble):
    """Encodes a 4-bit string into a 7-bit Hamming codeword."""
    d1 = int(data_nibble[0])
    d2 = int(data_nibble[1])
    d3 = int(data_nibble[2])
    d4 = int(data_nibble[3])

    # Calculate even parity bits
    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p4 = d2 ^ d3 ^ d4

    # Structure: [p1, p2, d1, p4, d2, d3, d4] at positions 1..7
    codeword = [p1, p2, d1, p4, d2, d3, d4]
    return "".join(map(str, codeword))
```

```
def decode_hamming_7_4(codeword_str):
    """Detects and corrects single-bit error, returns 4-bit data."""
    bits = [int(b) for b in codeword_str]
    p1, p2, d1, p4, d2, d3, d4 = bits

    # Calculate syndrome bits
    s1 = p1 ^ d1 ^ d2 ^ d4
    s2 = p2 ^ d1 ^ d3 ^ d4
    s4 = p4 ^ d2 ^ d3 ^ d4

    syndrome = (s4 << 2) | (s2 << 1) | s1

    if syndrome != 0:
        print(f"  [!] Single-bit error detected at position {syndrome}!")
        # Correct the error by flipping bit at (syndrome - 1)
        bits[syndrome - 1] ^= 1
        print(f"  [+] Error automatically corrected: {bits}")
    else:
        print("  [+] Codeword verified: No transmission errors.")

    # Extract data bits: positions 3, 5, 6, 7
    recovered_nibble = f"{bits[2]}{bits[4]}{bits[5]}{bits[6]}"
    return recovered_nibble

def sender(message):
    print(f"\n--- SENDER: Transmitting Message '{message}' ---")
    binary_stream = "".join(f"{ord(c):08b}" for c in message)
    print(f"Binary Stream: {binary_stream} ({len(binary_stream)} bits)")

    codewords = []
    for i in range(0, len(binary_stream), 4):
        nibble = binary_stream[i:i+4]
        cw = encode_hamming_7_4(nibble)
        codewords.append(cw)
        print(f"  Nibble: {nibble} -> Hamming(7,4) Codeword: {cw}")

    with open(CHANNEL_FILE, "w") as f:
        f.write(" ".join(codewords))
    print(f"[Sender] Encoded codewords successfully saved to '{CHANNEL_FILE}'.")

def inject_noise(block_index=0, bit_pos=3):
    """Simulates channel noise by flipping a bit at bit_pos (1-based)."""
    with open(CHANNEL_FILE, "r") as f:
        codewords = f.read().split()

    cw = list(codewords[block_index])
    cw[bit_pos - 1] = '0' if cw[bit_pos - 1] == '1' else '1'
    codewords[block_index] = "".join(cw)
    print(f"\n[Channel Noise Injected] Flipped bit {bit_pos} in block {block_index}!")

    with open(CHANNEL_FILE, "w") as f:
        f.write(" ".join(codewords))

def receiver():
    print(f"\n--- RECEIVER: Reading from '{CHANNEL_FILE}' ---")
    with open(CHANNEL_FILE, "r") as f:
        codewords = f.read().split()
```

```
    recovered_binary = ""
    for idx, cw in enumerate(codewords):
        print(f"Processing Block {idx+1} (Received: {cw}):")
        data_nibble = decode_hamming_7_4(cw)
        recovered_binary += data_nibble

    # Convert binary back to ASCII characters
    chars = []
    for i in range(0, len(recovered_binary), 8):
        byte = recovered_binary[i:i+8]
        chars.append(chr(int(byte, 2)))

    decoded_message = "".join(chars)
    print(f"\n[Receiver] Decoded ASCII Message: '{decoded_message}'")
    return decoded_message

if __name__ == "__main__":
    msg = "NETWORKS"
    sender(msg)
    inject_noise(block_index=0, bit_pos=3) # corrupt bit 3 in first block
    recovered = receiver()
    assert recovered == msg, "Decoded message mismatch!"
    print("[SUCCESS] Automated error detection and correction verified!")
```

Sample Output / Verification: Hamming Code Error Detection & Correction Verification

```
--- SENDER: Transmitting Message 'NETWORKS' ---
Binary Stream: 0100111001000101010101000101011101001111010100100100101101010011 (64 bits)
  Nibble: 0100 -> Hamming(7,4) Codeword: 1101100
  Nibble: 1110 -> Hamming(7,4) Codeword: 0011110
  Nibble: 0100 -> Hamming(7,4) Codeword: 1101100
  Nibble: 0101 -> Hamming(7,4) Codeword: 0101101
  Nibble: 0101 -> Hamming(7,4) Codeword: 0101101
  Nibble: 0100 -> Hamming(7,4) Codeword: 1101100
  Nibble: 0101 -> Hamming(7,4) Codeword: 0101101
  Nibble: 0111 -> Hamming(7,4) Codeword: 1000111
  Nibble: 0100 -> Hamming(7,4) Codeword: 1101100
  Nibble: 1111 -> Hamming(7,4) Codeword: 1111111
  Nibble: 0101 -> Hamming(7,4) Codeword: 0101101
  Nibble: 0010 -> Hamming(7,4) Codeword: 1000010
  Nibble: 0100 -> Hamming(7,4) Codeword: 1101100
  Nibble: 1011 -> Hamming(7,4) Codeword: 0110011
  Nibble: 0101 -> Hamming(7,4) Codeword: 0101101
  Nibble: 0011 -> Hamming(7,4) Codeword: 0100011
[Sender] Encoded codewords successfully saved to 'channel.txt'.

[Channel Noise Injected] Flipped bit 3 in block 0!

--- RECEIVER: Reading from 'channel.txt' ---
Processing Block 1 (Received: 1111100):
  [!] Single-bit error detected at position 3!
  [+] Error automatically corrected: [1, 1, 0, 1, 1, 0, 0]
Processing Block 2 (Received: 0011110):
  [+] Codeword verified: No transmission errors.
Processing Block 3 (Received: 1101100):
  [+] Codeword verified: No transmission errors.
```

```
...
[Receiver] Decoded ASCII Message: 'NETWORKS'
[SUCCESS] Automated error detection and correction verified!
```

EXPERIMENT 7: FLOW CONTROL AT DATA LINK LAYER USING SLIDING WINDOW PROTOCOL

AIM: To implement flow control at the Data Link Layer using a Sliding Window Protocol (Go-Back-N / Selective Repeat simulation) in Python, manage buffer files ('Sender_Buffer.txt' and 'Receiver_Buffer.txt'), handle acknowledgments and timeouts, and simulate error retransmission.

7.1 Theoretical Principles of Sliding Window Flow Control

Flow control ensures that a fast sending station does not overwhelm a slower receiving node with frames that cannot be processed in time. Unlike Stop-and-Wait (which restricts throughput by requiring an acknowledgment before each subsequent frame), the Sliding Window Protocol allows the sender to transmit up to 'W' frames (the window size) before pausing for an acknowledgment.

1. Sequence Numbering & Modulo Arithmetic:
Frames are indexed by integer sequence numbers ranging from 0 to 2^m - 1. For a window of size W, the maximum window size in Go-Back-N is 2^m - 1.

2. Sender Buffer Window:
Maintains boundaries [S_first, S_last], where S_first is the oldest unacknowledged frame and S_last is the newest transmitted frame. As cumulative acknowledgments arrive, S_first advances, allowing new frames to enter the transmission window.

3. Piggybacking & Feedback:
In bidirectional communication, acknowledgments are piggybacked onto reverse data frames, conserving bandwidth. If a frame is dropped or corrupted, the receiver issues a Negative Acknowledgment (NACK) or the sender experiences a timeout, triggering retransmission.

7.2 Step-by-Step Algorithm

Sender Flow:
1. Prompt user for window size W (e.g., W = 4) and message text.
2. Package characters into discrete frames: Frame = [Sequence_Number, Character_Data].
3. Transmit frames within the active window [S_base, S_base + W - 1] and record them into 'Sender_Buffer.txt'.
4. Induce a propagation delay, then poll 'Receiver_Buffer.txt' for acknowledgment.
5. If positive ACK(k) is received: Slide window so S_base = k. Overwrite 'Sender_Buffer.txt' with newly available frames.
6. If NACK(k) or timeout occurs: Reset transmission pointer to k and retransmit unacknowledged frames.

Receiver Flow:
1. Read frame from 'Sender_Buffer.txt'.

2. Verify whether incoming Frame_No matches Expected_Sequence_No.
3. If match: Accept frame, append character to received stream, increment Expected_Sequence_No, and write ACK(Expected_Sequence_No) into 'Receiver_Buffer.txt'.
4. If corrupted or out-of-order: Discard frame and write NACK(Expected_Sequence_No) to signal retransmission.

Source Code: Sliding Window Protocol Simulator

```
# ====================================================================
# CS23532 Computer Networks Lab - Experiment 7
# Sliding Window Protocol Simulation with File Buffers & Retransmission
# Student Name: THANIKAIVEL S | Reg No: 2116240701562
# ====================================================================
import time
import os

SENDER_BUFFER = "Sender_Buffer.txt"
RECEIVER_BUFFER = "Receiver_Buffer.txt"

class SlidingWindowSimulator:
    def __init__(self, message, window_size=4):
        self.message = message
        self.window_size = window_size
        self.total_frames = len(message)
        self.frames = [(i, message[i]) for i in range(self.total_frames)]

    def run_simulation(self, induce_error_at_frame=3):
        print(f"--- SLIDING WINDOW SIMULATION ---")
        print(f"Message: '{self.message}' | Total Frames: {self.total_frames} | Window
Size: {self.window_size}")

        s_base = 0
        error_injected = False

        while s_base < self.total_frames:
            s_end = min(s_base + self.window_size, self.total_frames)
            current_window = self.frames[s_base:s_end]

            print(f"\n[Current Window] Frames: {[f[0] for f in current_window]} | Base
Index: {s_base}")

            # Write frames to Sender_Buffer
            with open(SENDER_BUFFER, "w") as f:
                for frame_no, char in current_window:
                    f.write(f"{frame_no}:{char}\n")
            print(f"  [Sender -> Buffer] Transmitted frames: {[f'F{f[0]}(\'{f[1]}\')' for f
in current_window]}")

            time.sleep(0.3) # induced channel propagation delay

            # Receiver processes buffer
            with open(SENDER_BUFFER, "r") as f:
                incoming_lines = [line.strip().split(":") for line in f if line.strip()]

            # Simulate error condition
            if not error_injected and induce_error_at_frame is not None and s_base <=
```

```
induce_error_at_frame < s_end:
                print(f"  [!] NOISE INDUCED: Simulating packet corruption / drop on Frame
{induce_error_at_frame}!")
                with open(RECEIVER_BUFFER, "w") as f:
                    f.write(f"NACK:{induce_error_at_frame}\n")
                error_injected = True
            else:
                # Normal successful reception of window
                last_ack = s_end
                with open(RECEIVER_BUFFER, "w") as f:
                    f.write(f"ACK:{last_ack}\n")

            time.sleep(0.2)

            # Sender reads Receiver_Buffer
            with open(RECEIVER_BUFFER, "r") as f:
                ack_type, ack_val = f.readline().strip().split(":")
                ack_val = int(ack_val)

            if ack_type == "ACK":
                print(f"  [Receiver -> Sender] Positive ACK received for frames up to
{ack_val}!")
                s_base = ack_val # Advance window
            elif ack_type == "NACK":
                print(f"  [Receiver -> Sender] NACK received for Frame {ack_val}!
Retransmitting from Frame {ack_val}...")
                s_base = ack_val # Slide back / Go-Back-N

        print("\n[SUCCESS] Entire message successfully transmitted and acknowledged!")
        # Clean up temporary buffers
        for b in [SENDER_BUFFER, RECEIVER_BUFFER]:
            if os.path.exists(b): os.remove(b)

if __name__ == "__main__":
    sim = SlidingWindowSimulator(message="COMPUTER_NETWORKS", window_size=4)
    sim.run_simulation(induce_error_at_frame=4)
```

Sample Output / Verification: Sliding Window Execution with Simulated Retransmission

```
--- SLIDING WINDOW SIMULATION ---
Message: 'COMPUTER_NETWORKS' | Total Frames: 17 | Window Size: 4

[Current Window] Frames: [0, 1, 2, 3] | Base Index: 0
  [Sender -> Buffer] Transmitted frames: ["F0('C')", "F1('O')", "F2('M')", "F3('P')"]
  [Receiver -> Sender] Positive ACK received for frames up to 4!

[Current Window] Frames: [4, 5, 6, 7] | Base Index: 4
  [Sender -> Buffer] Transmitted frames: ["F4('U')", "F5('T')", "F6('E')", "F7('R')"]
  [!] NOISE INDUCED: Simulating packet corruption / drop on Frame 4!
  [Receiver -> Sender] NACK received for Frame 4! Retransmitting from Frame 4...

[Current Window] Frames: [4, 5, 6, 7] | Base Index: 4
  [Sender -> Buffer] Transmitted frames: ["F4('U')", "F5('T')", "F6('E')", "F7('R')"]
  [Receiver -> Sender] Positive ACK received for frames up to 8!

[Current Window] Frames: [8, 9, 10, 11] | Base Index: 8
```

```
  [Sender -> Buffer] Transmitted frames: ["F8('_')", "F9('N')", "F10('E')", "F11('T')"]
  [Receiver -> Sender] Positive ACK received for frames up to 12!

[Current Window] Frames: [12, 13, 14, 15] | Base Index: 12
  [Sender -> Buffer] Transmitted frames: ["F12('W')", "F13('O')", "F14('R')", "F15('K')"]
  [Receiver -> Sender] Positive ACK received for frames up to 16!

[Current Window] Frames: [16] | Base Index: 16
  [Sender -> Buffer] Transmitted frames: ["F16('S')"]
  [Receiver -> Sender] Positive ACK received for frames up to 17!

[SUCCESS] Entire message successfully transmitted and acknowledged!
```

EXPERIMENT 8: LIVE HOST DISCOVERY USING NMAP SCANS ON TRYHACKME PLATFORM

AIM: To perform active network reconnaissance and host discovery across subnets using Nmap (Network Mapper) scans (ARP, ICMP, TCP SYN/ACK, UDP) within the TryHackMe cybersecurity training environment, and interpret open port states and service fingerprinting.

8.1 Host Discovery Concepts & Nmap Scanning Taxonomy

Network Mapper (Nmap) is the premier open-source utility for network exploration and vulnerability auditing. Before performing port auditing, Nmap establishes which target IP addresses represent live, responsive hosts. Depending on network topology (local Ethernet segment vs routed Internet subnets) and firewall filtering policies, different probing protocols must be utilized:

Scan Technique	Command Syntax	Protocol Mechanism & Operational Scope
ARP Ping Scan	nmap -PR -sn <target_subnet>	Default on local Ethernet subnets. Emits ARP requests to discover live hosts regardless of host firewalls blocking ICMP.
ICMP Echo Ping	nmap -PE -sn <target>	Sends classic ICMP Echo Requests (Type 8). Simple, but frequently blocked by corporate border firewalls.
ICMP Timestamp Ping	nmap -PP -sn <target>	Queries host clock timestamp (Type 13). Useful when ICMP Echo is blocked but timestamp queries remain unmonitored.
ICMP Address Mask	nmap -PM -sn <target>	Requests subnet mask info (Type 17) to identify live systems.
TCP SYN Ping	nmap -PS22,80,443 -sn <target>	Sends TCP SYN packets to designated ports. SYN-ACK (open) or RST (closed) confirms the host is alive.
TCP ACK Ping	nmap -PA80,443 -sn <target>	Transmits TCP ACK packets. Unsolicited ACKs elicit an RST from responsive hosts, bypassing stateless SYN packet filters.
UDP Ping	nmap -PU53,161 -sn <target>	Sends UDP datagrams to ports. Receiving ICMP Port Unreachable confirms host presence.
Port Scanning (-sS)	nmap -sS -p- <target>	Stealth TCP SYN Half-Open scan. Never completes three-way handshake, avoiding application-layer logging.

Service Versioning	nmap -sV -sC <target>	Queries open ports and matches banners against Nmap's signature database to detect server OS and daemon versions.

8.2 TryHackMe Laboratory Walkthrough & Telemetry

The following screenshots depict live terminal execution across the TryHackMe 'Network Scanning' room targets:

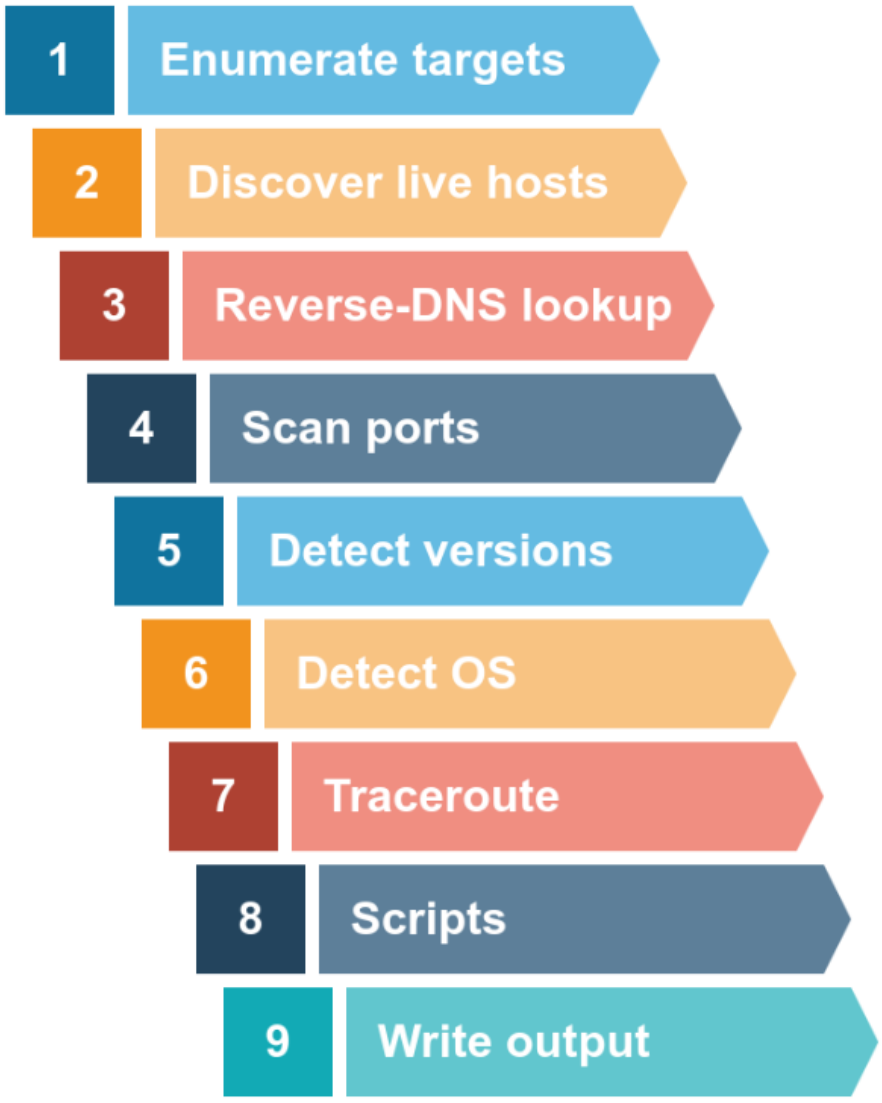


Figure 8.1: TryHackMe AttackBox Nmap Target Scope Initialization

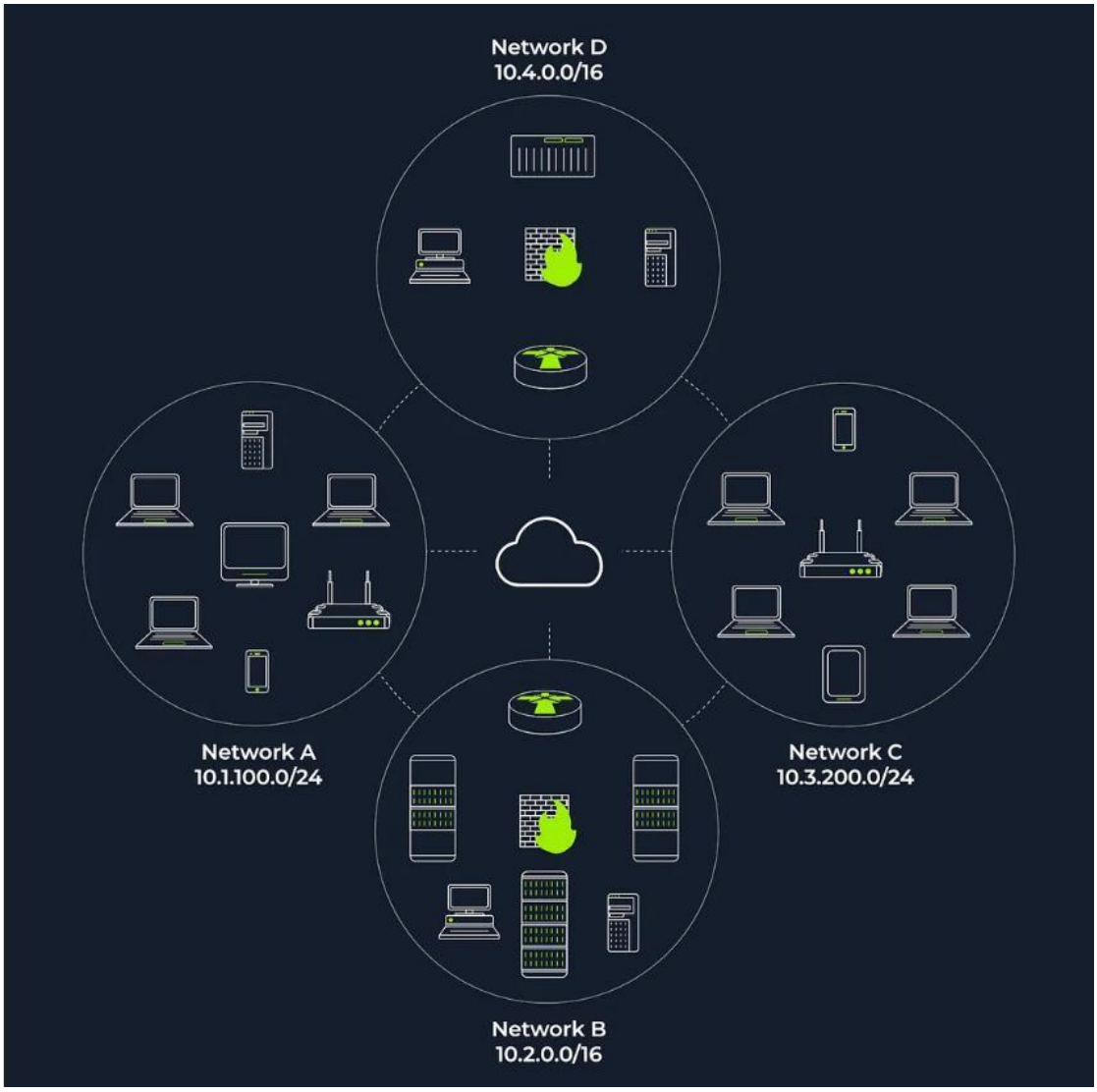


Figure 8.2: ARP Scan Discovery (-PR) on Local Subnet

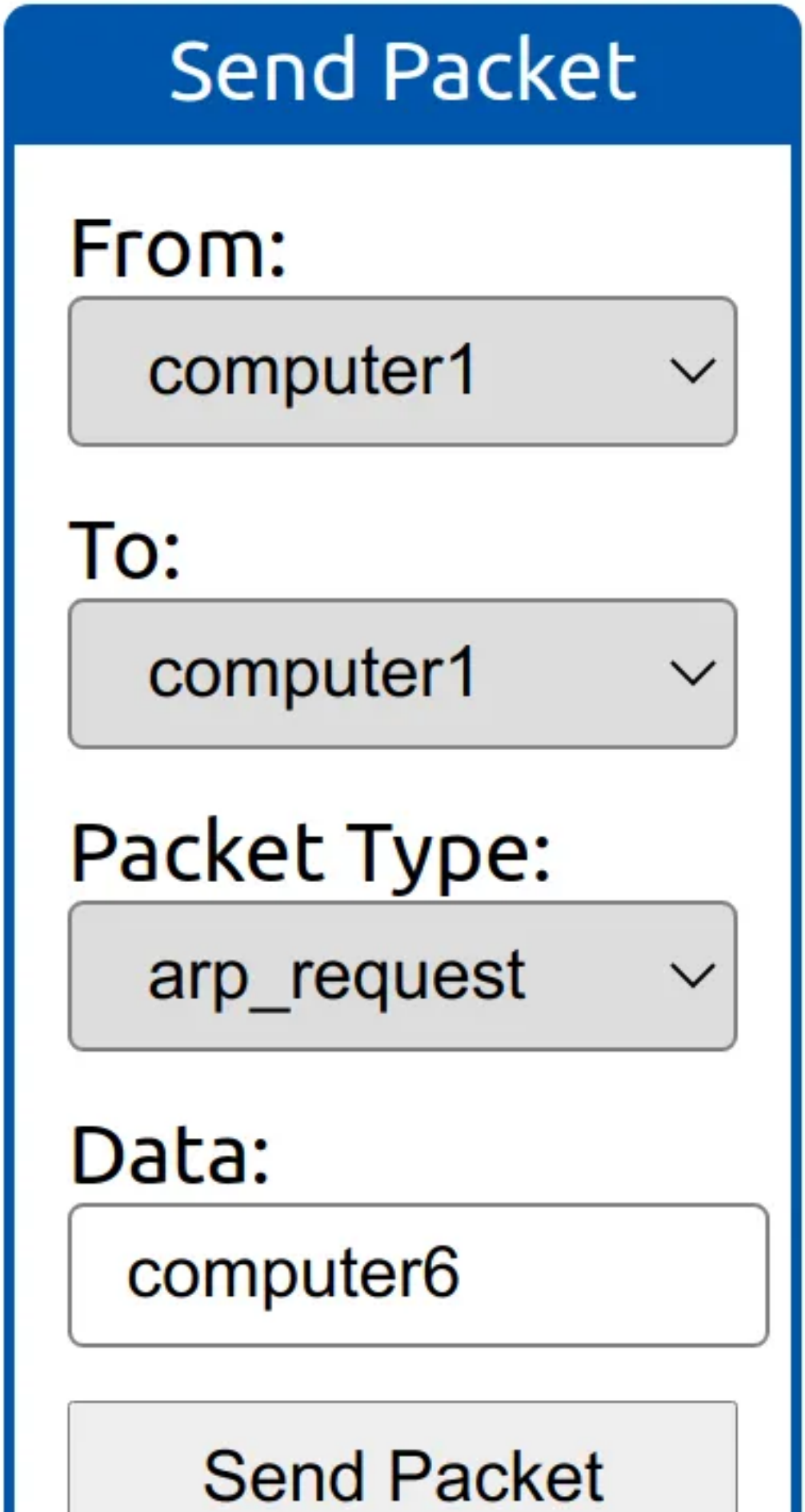

Send Packet
From:
computer1
To:
computer1
Packet Type:
arp_request
Data:
computer6
Send Packet

Figure 8.3: ICMP Echo Ping Scan (-PE) Analysis

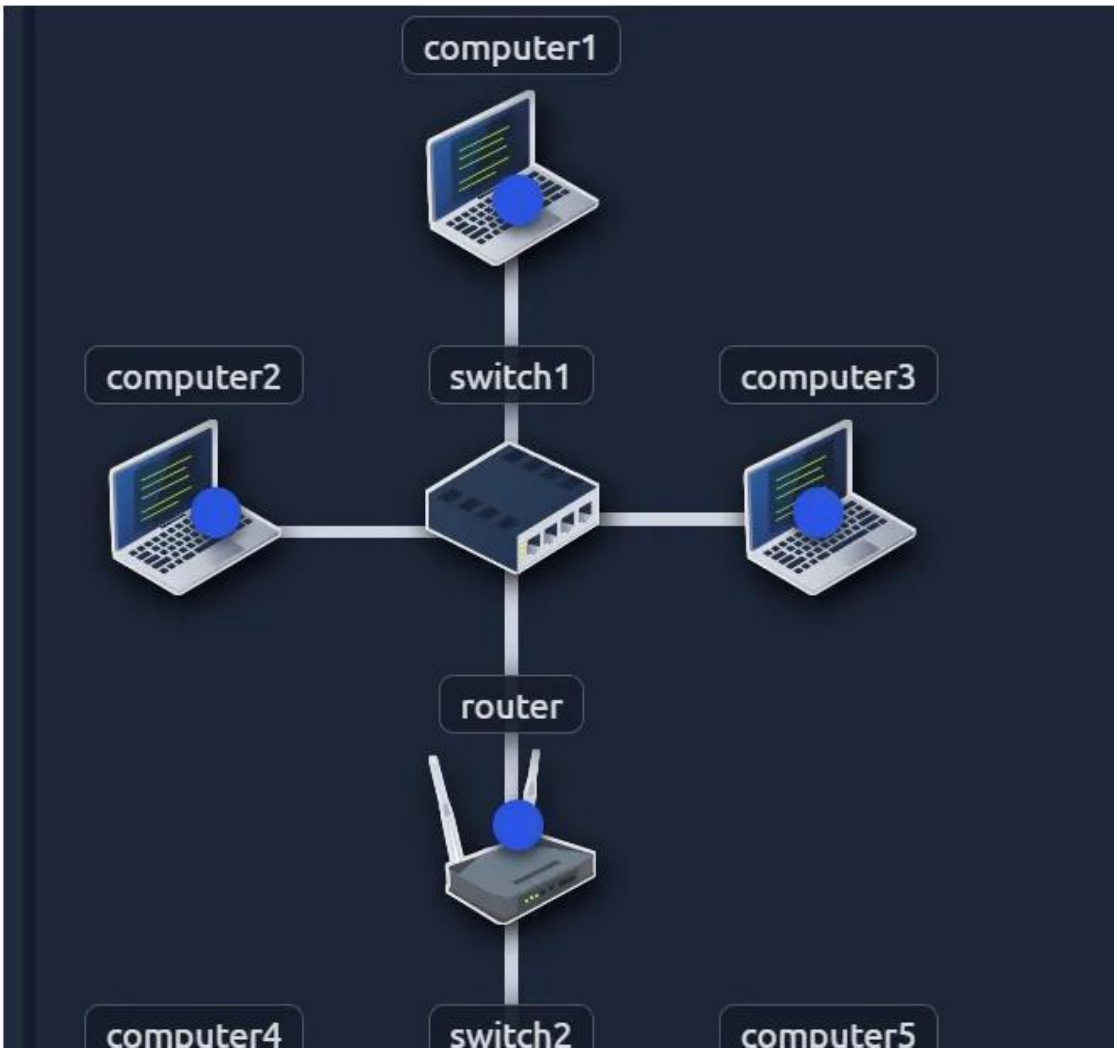
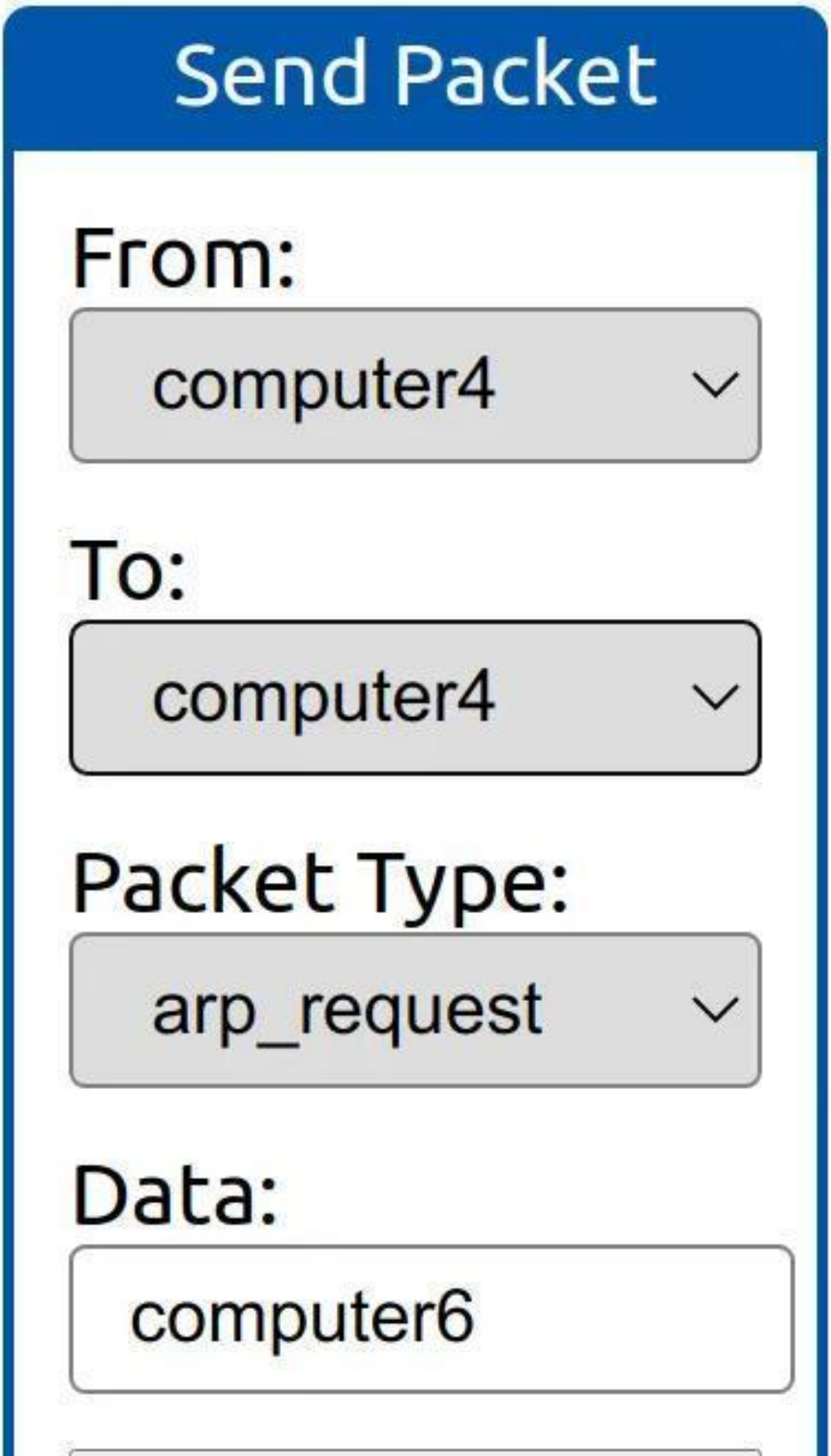


Figure 8.4: ICMP Timestamp Probe (-PP) Bypassing Standard Filters


Send Packet
From:
computer4
To:
computer4
Packet Type:
arp_request
Data:
computer6

01562)

Figure 8.5: ICMP Address Mask Query (-PM)

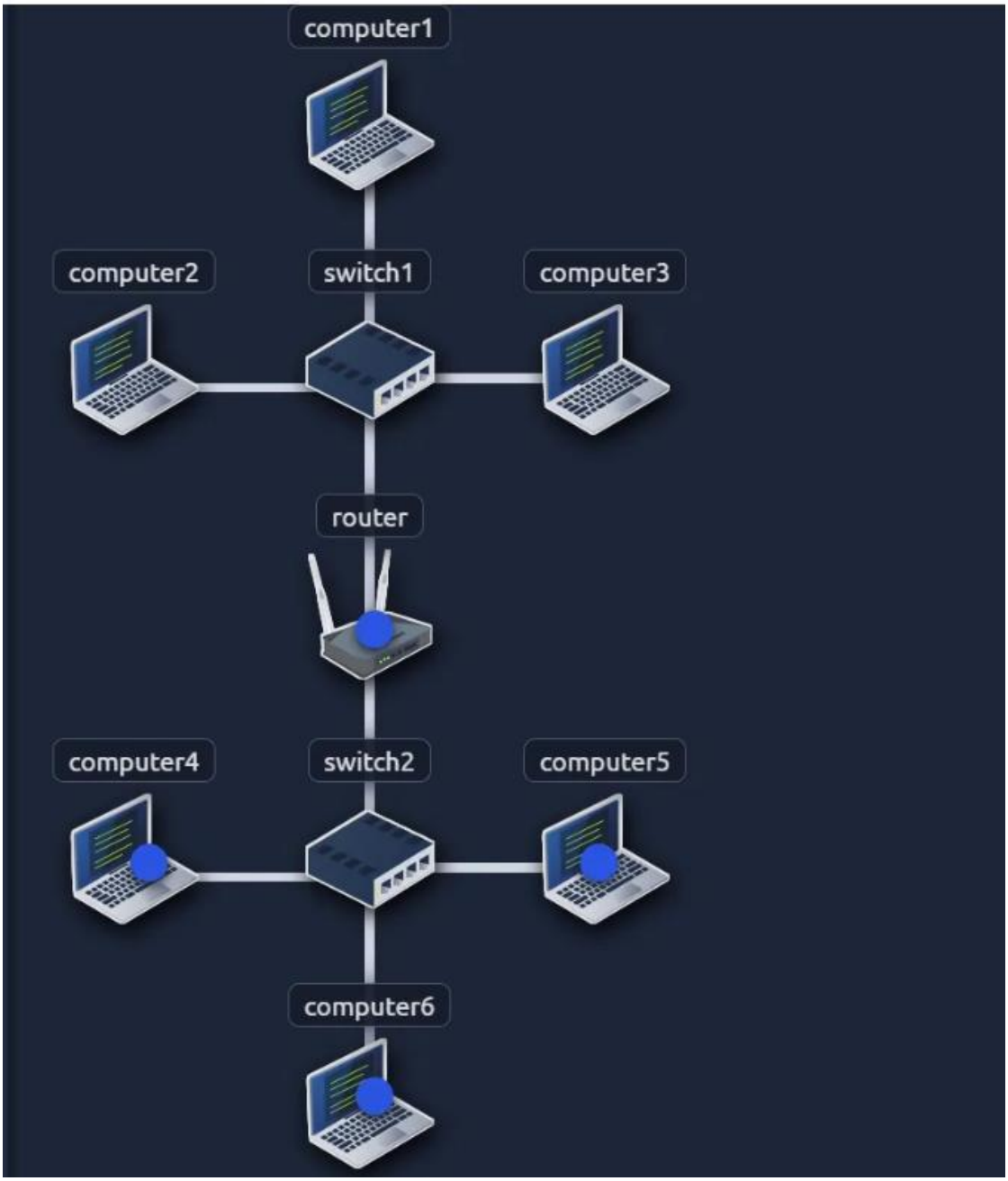


Figure 8.6: TCP SYN Ping (-PS) Probing Common Ports (22, 80)

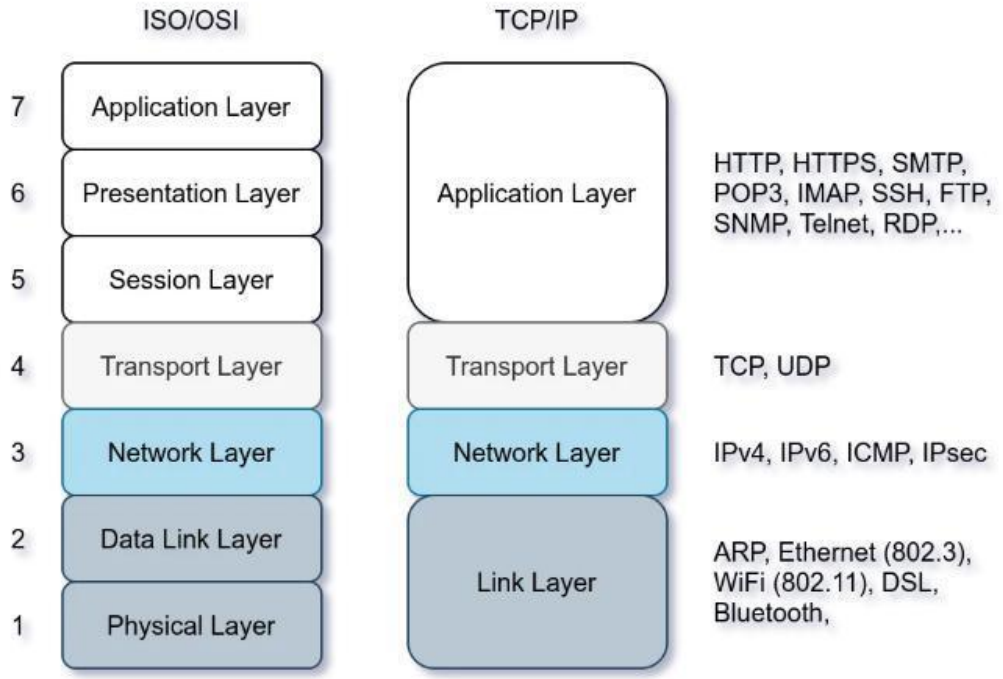


Figure 8.7: TCP ACK Ping (-PA) Filtering Verification

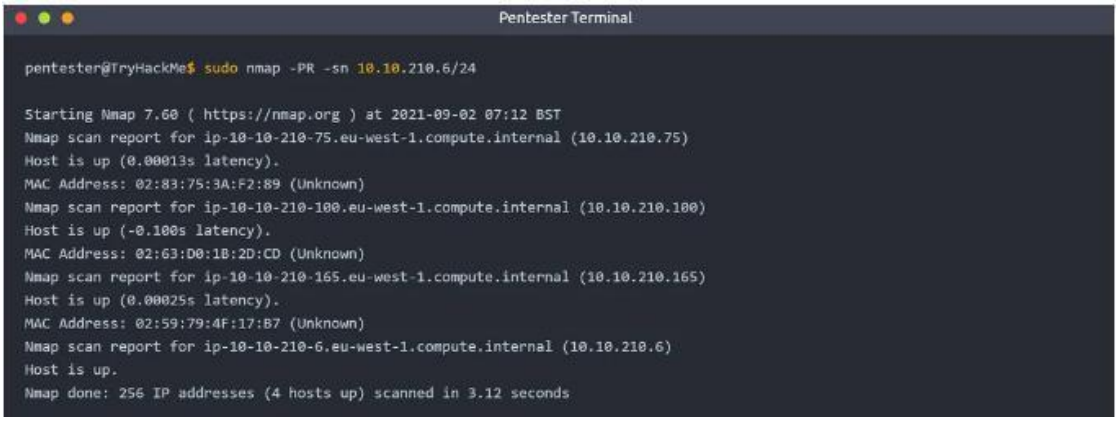


Figure 8.8: UDP Port Probing (-PU) Response Extraction

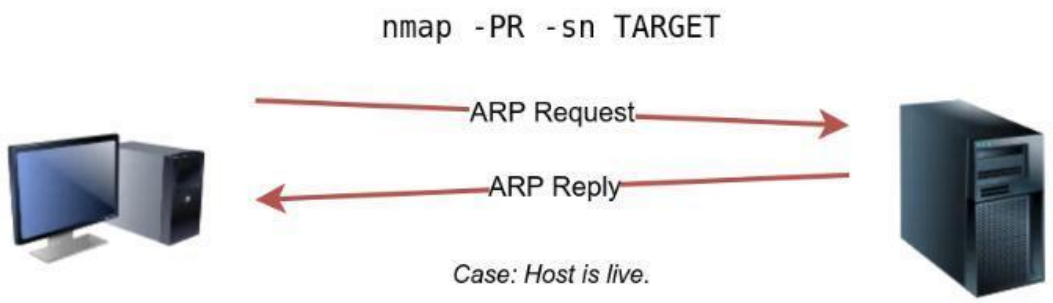


Figure 8.9: Combined Multi-Vector Ping Sweeps

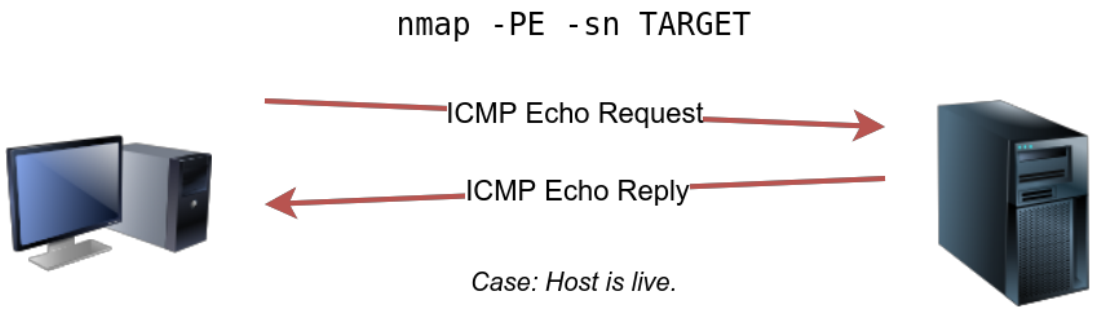


Figure 8.10: SYN Stealth Port Scanning (-sS) Open Port Detection

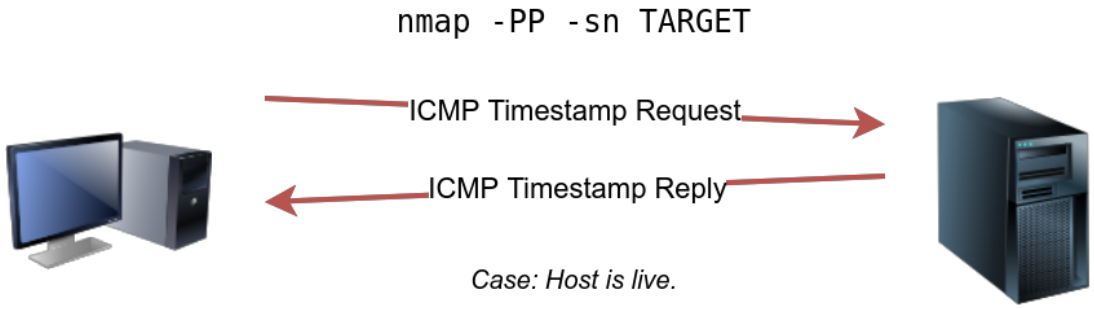


Figure 8.11: Identifying Filtered vs Closed Port States

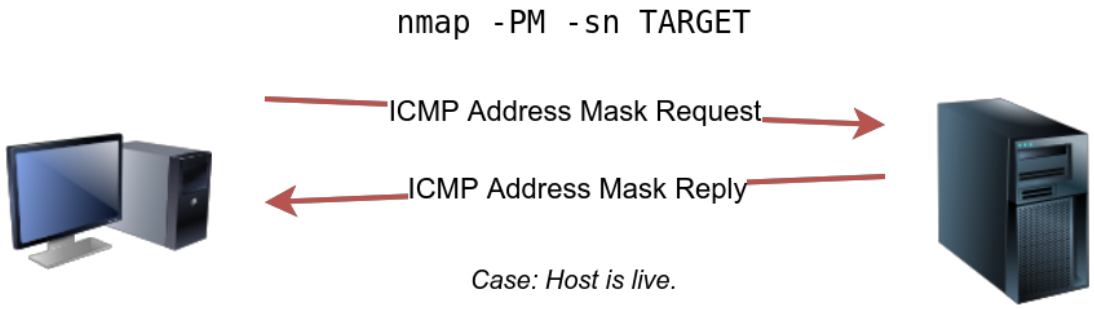


Figure 8.12: Full TCP Connect Scan (-sT) Verification

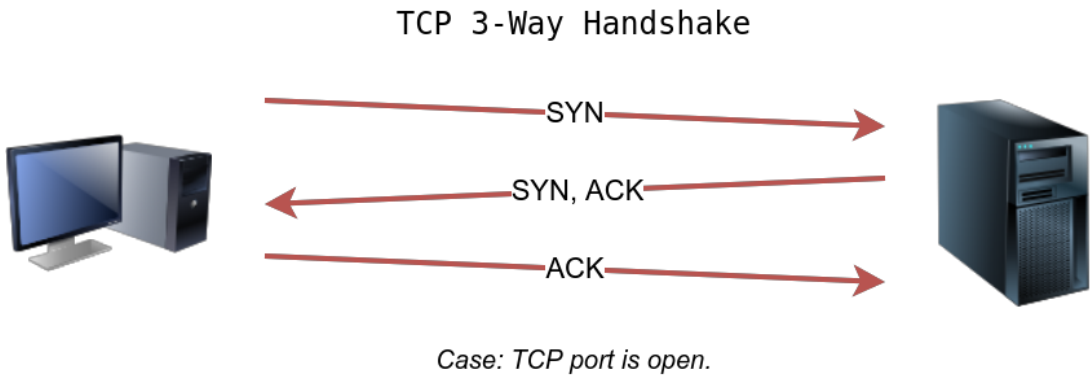


Figure 8.13: UDP Port Scanning (-sU) against DNS Service

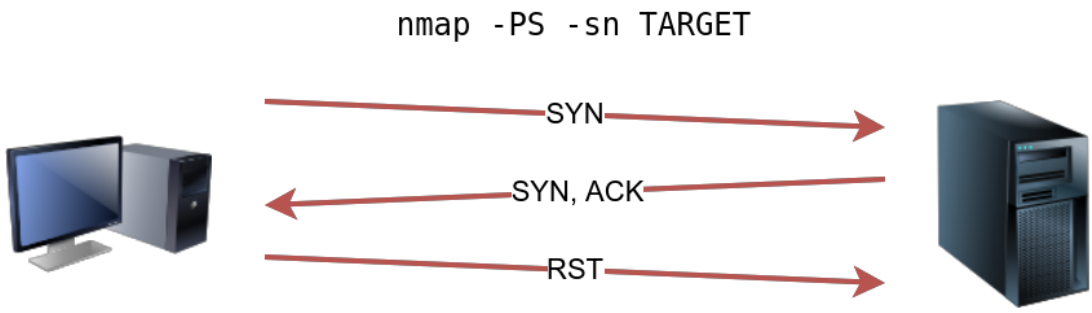


Figure 8.14: Service and Daemon Version Fingerprinting (-sV)

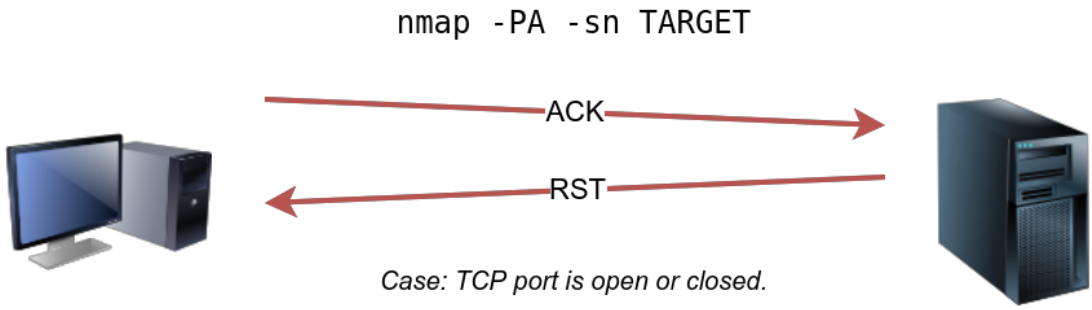


Figure 8.15: Operating System Detection via TCP/IP Stack Fingerprinting (-O)

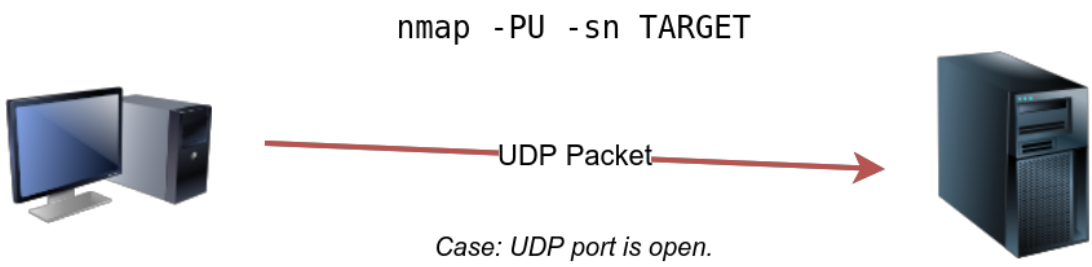


Figure 8.16: NSE Default Vulnerability Script Execution (-sC)

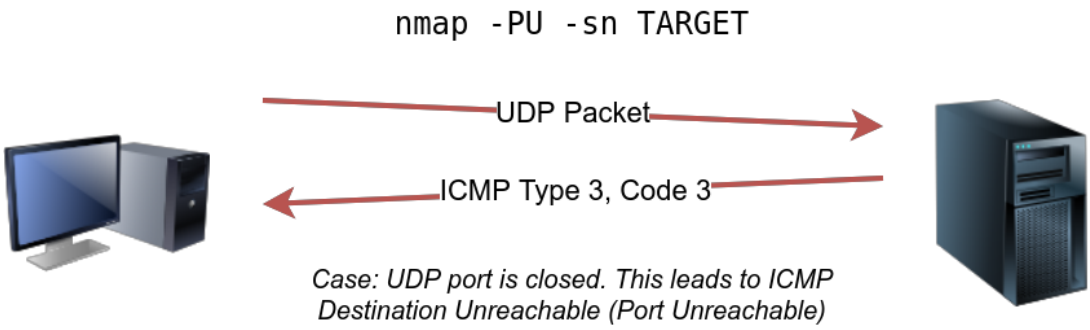


Figure 8.17: Comprehensive Aggressive Scan (-A) Telemetry Output

EXPERIMENT 8(b): CONFIGURATION OF WIRELESS LAN USING CISCO PACKET TRACER

AIM: To design, configure, and secure an IEEE 802.11 Wireless Local Area Network (WLAN) in Cisco Packet Tracer utilizing a Linksys WRT300N Wireless Router, configure WPA2-Personal security, and associate multiple wireless laptop/desktop hosts with dynamic DHCP addressing.

8.1 Wireless Architecture & Component Specifications

Wireless LANs replace physical Ethernet patch cables with radio frequency (RF) waves in the 2.4 GHz and 5 GHz Industrial, Scientific, and Medical (ISM) bands. The Linksys WRT300N serves as a multi-function device integrating an 802.11b/g/n Wireless Access Point, an internal 4-port Ethernet switch, a NAT router, and a DHCP server.

Parameter	Configured Setting	Description
Network SSID	MotherNet	Service Set Identifier broadcast across the 2.4 GHz RF channel.
Radio Channel	Channel 6 - 2.437 GHz	Standard non-overlapping ISM radio channel.
Security Protocol	WPA2-Personal (AES)	Robust encryption utilizing the Advanced Encryption Standard.
Pre-Shared Key (PSK)	Password@123	Shared authentication passphrase.
LAN IP Configuration	192.168.0.1 / 255.255.255.0	Wireless router internal gateway IP address.
DHCP Pool Range	192.168.0.100 - 192.168.0.149	Dynamic address block leased automatically to associated clients.

8.2 Configuration Procedure

1. Host Hardware Modification: In Cisco Packet Tracer, end devices default to wired Ethernet NICs. Power off each PC, remove the PT-HOST-NM-1CFE module, and install the WMP300N 2.4GHz Wireless Module. Power the PC back on.
2. Wireless Router Setup: Open the Linksys WRT300N GUI tab. Under Wireless -> Basic Wireless Settings, configure Network Mode to 'Mixed', Network Name (SSID) to 'MotherNet', and Channel to '6-2.437GHz'.
3. Security Setup: Navigate to Wireless -> Wireless Security. Select WPA2-Personal, Encryption 'AES', and enter the Passphrase 'Password@123'. Save settings.
4. Client Association: On each PC, open Desktop -> PC Wireless. Navigate to the 'Connect' tab, click 'Refresh', select 'MotherNet', click 'Connect', and enter the WPA2 passphrase.

5. Verification: Confirm the wireless link waves appear in Packet Tracer workspace. Check IP Configuration to verify dynamic acquisition of IP 192.168.0.100+ via DHCP, and ping between PCs.

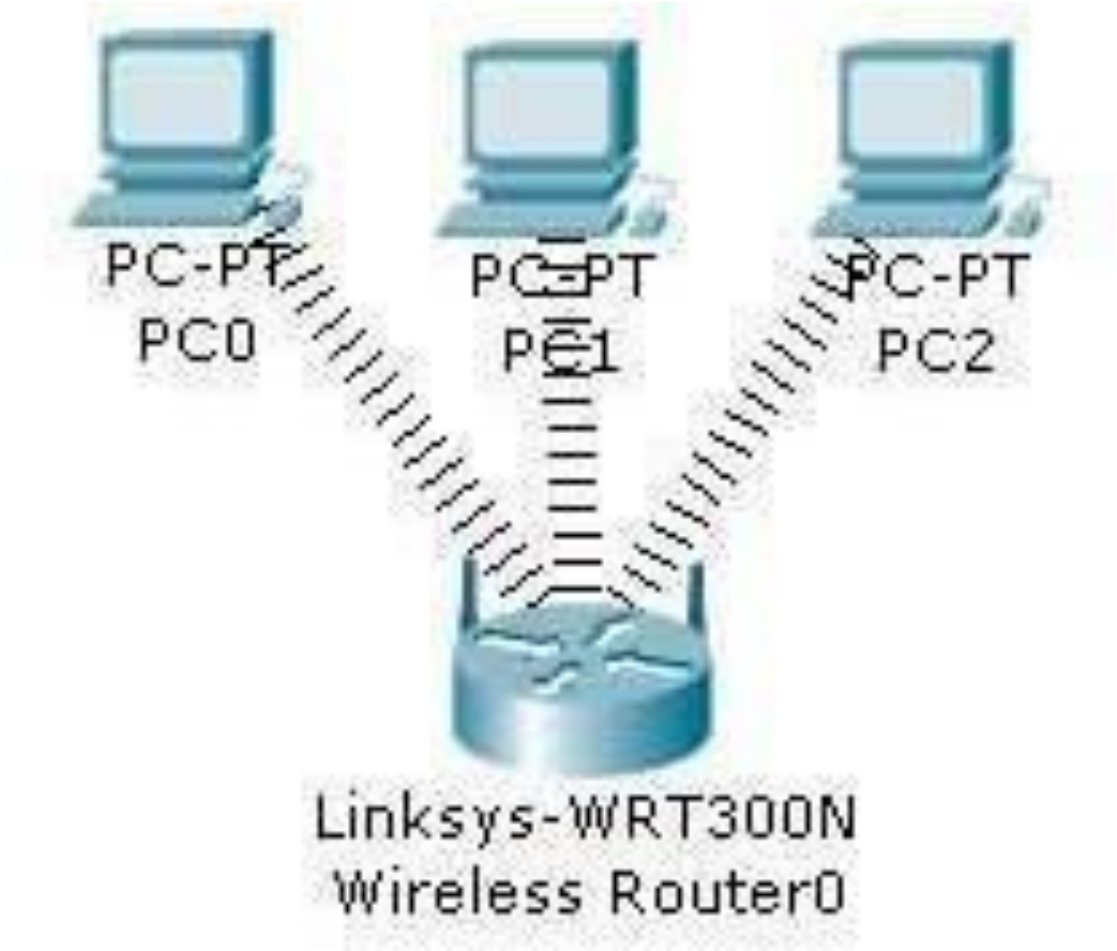


Figure 8.18: Linksys WRT300N Basic Wireless Settings (SSID Configuration)



Figure 8.19: Installing WMP300N Wireless NIC onto Workstation



Figure 8.20: WPA2-Personal Wireless Security Configuration



Figure 8.21: PC Wireless Utility Scanning Available Networks

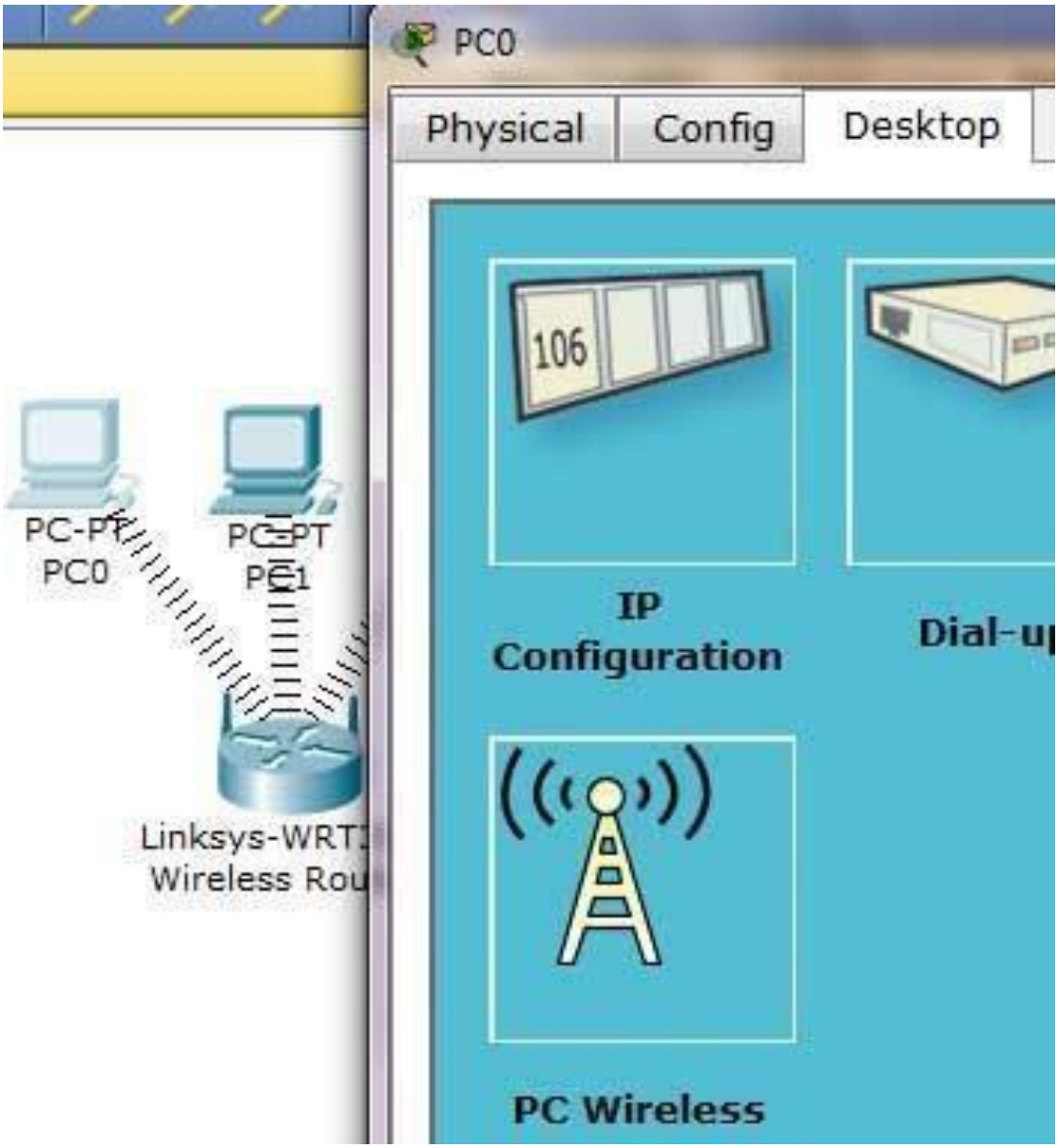


Figure 8.22: Authenticating with WPA2 Passphrase

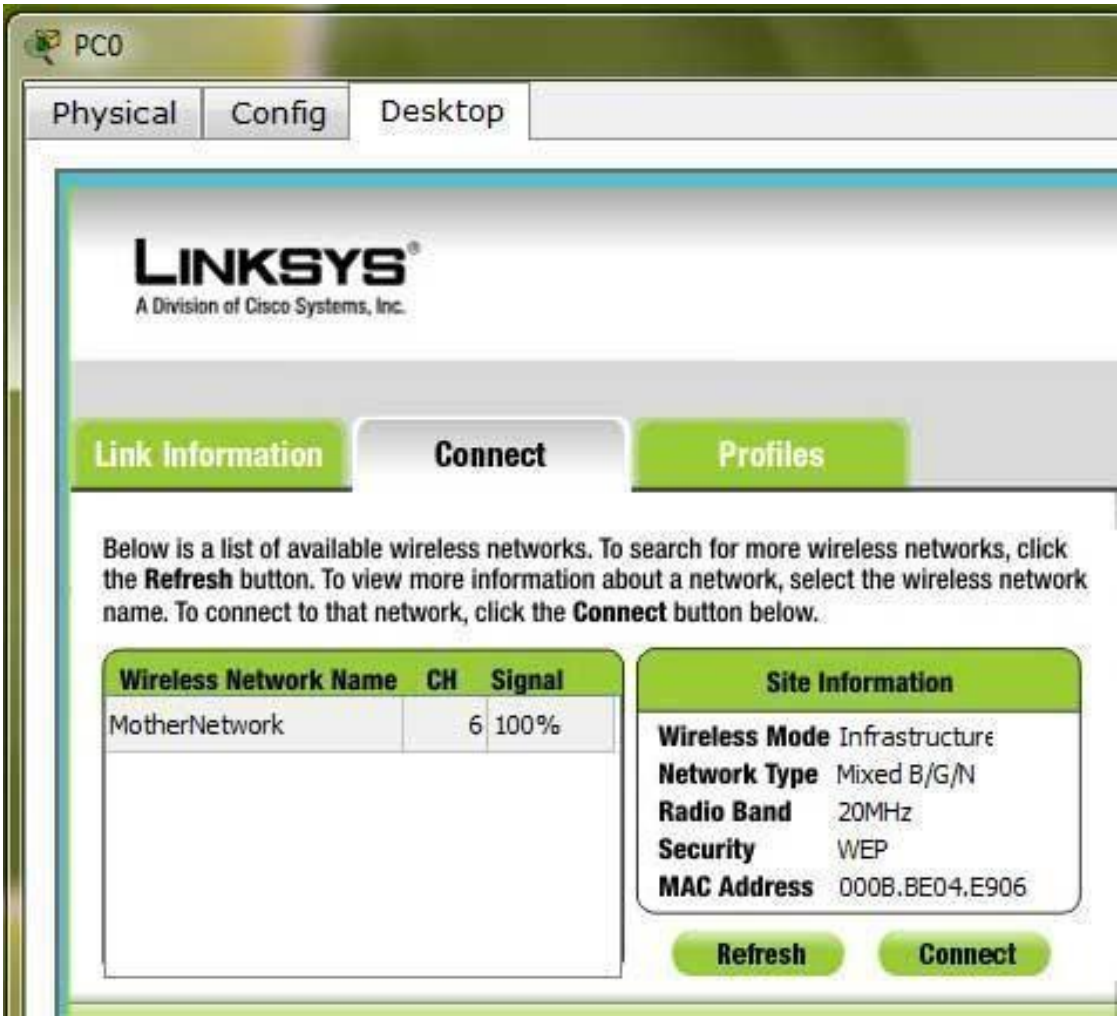


Figure 8.23: Successful Association and Link Telemetry

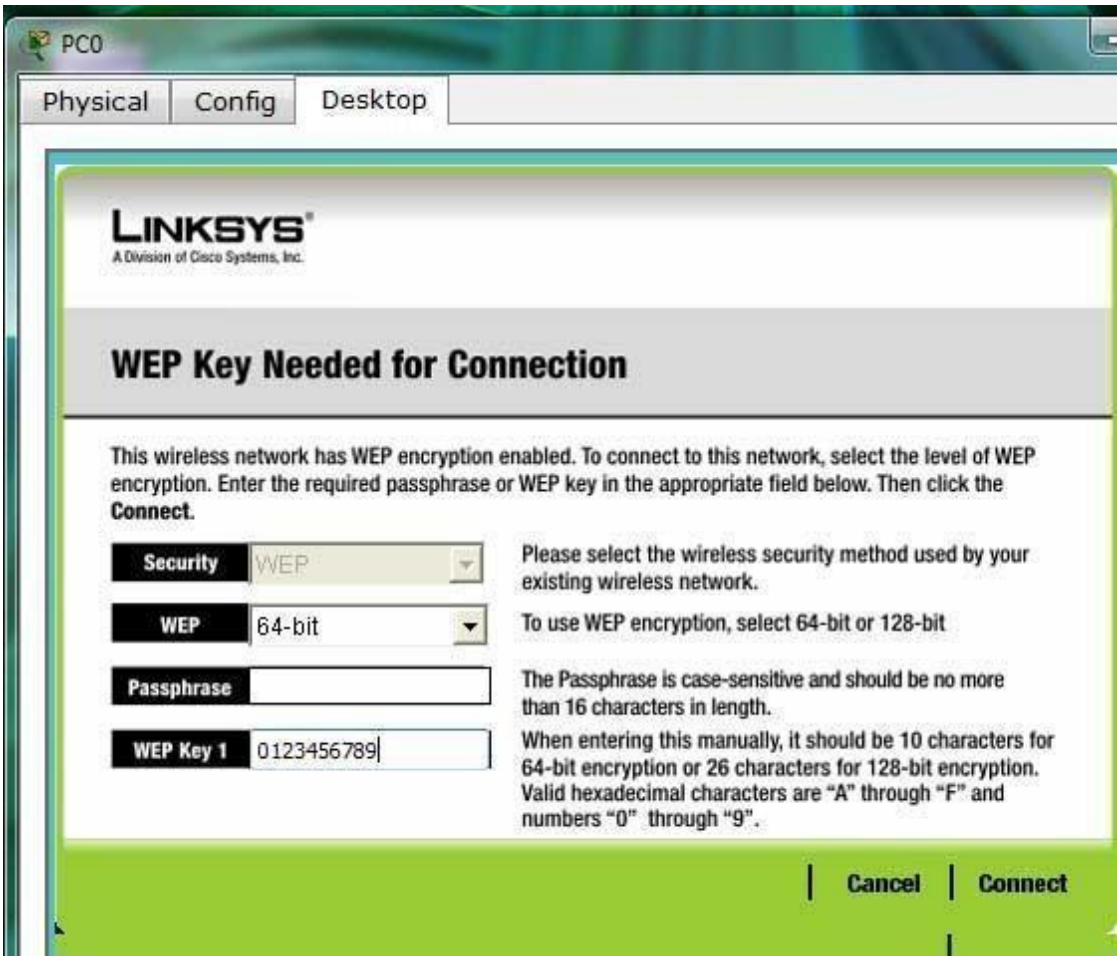


Figure 8.24: Dynamic DHCP Address Acquisition Verification



Figure 8.25: End-to-End Wireless Ping Reachability Test

EXPERIMENT 9: CLASSLESS SUBNETTING IMPLEMENTATION IN CISCO PACKET TRACER

AIM: To design and implement Classless Inter-Domain Routing (CIDR) subnetting in Cisco Packet Tracer, partition a Class C network (192.168.1.0/24) into multiple logical subnets with appropriate host allocations, configure router interfaces, and demonstrate inter-subnet routing.

9.1 Subnetting Theory & CIDR Calculation

Classless IP subnetting partitions a single large network address space into smaller, manageable broadcast domains, improving security, containing broadcast storms, and conserving limited IPv4 address blocks. By borrowing host bits for the network prefix, custom subnet masks are constructed.

Given Network: 192.168.1.0/24 (Default mask: 255.255.255.0):
To support multiple departmental networks, we borrow 2 host bits, creating 2^2 = 4 subnets with a /26 prefix (Mask: 255.255.255.192, Block Size = 64):

Subnet Name	Network Address	Subnet Mask	Usable Host IP Range	Broadcast IP	Capacity
Subnet 1 (Finance)	192.168.1.0	255.255.255.192 (/26)	192.168.1.1 - 192.168.1.62	192.168.1.63	62 Hosts
Subnet 2 (R&D)	192.168.1.64	255.255.255.192 (/26)	192.168.1.65 - 192.168.1.126	192.168.1.127	62 Hosts
Subnet 3 (Operations)	192.168.1.128	255.255.255.192 (/26)	192.168.1.129 - 192.168.1.190	192.168.1.191	62 Hosts
Subnet 4 (Management)	192.168.1.192	255.255.255.192 (/26)	192.168.1.193 - 192.168.1.254	192.168.1.255	62 Hosts

9.2 Packet Tracer Topology & Routing Configuration

1. Deploy a Cisco 1941/2811 Router and connect two distinct switches (Switch0 for Subnet 1, Switch1 for Subnet 2).
2. Connect client PCs to each respective switch.
3. Router Interface Configuration (CLI):

```
Router> enable
Router# configure terminal
Router(config)# interface GigabitEthernet0/0
Router(config-if)# ip address 192.168.1.1 255.255.255.192
Router(config-if)# no shutdown
```

```
Router(config-if)# exit
Router(config)# interface GigabitEthernet0/1
Router(config-if)# ip address 192.168.1.65 255.255.255.192
Router(config-if)# no shutdown
Router(config-if)# exit
```

4. Client Workstation IP Configuration:

- PC0 (Subnet 1): IP = 192.168.1.10 | Mask = 255.255.255.192 | Gateway = 192.168.1.1
- PC1 (Subnet 2): IP = 192.168.1.70 | Mask = 255.255.255.192 | Gateway = 192.168.1.65

5. Testing and Observation:

- Case A (With Router): Execute 'ping 192.168.1.70' from PC0. The packet is routed through the default gateway (192.168.1.1) to GigabitEthernet0/1, reaching PC1 successfully.
- Case B (Without Router): When connected directly to the same unmanaged switch without router intervention, PCs in different subnets cannot communicate due to differing network prefixes.

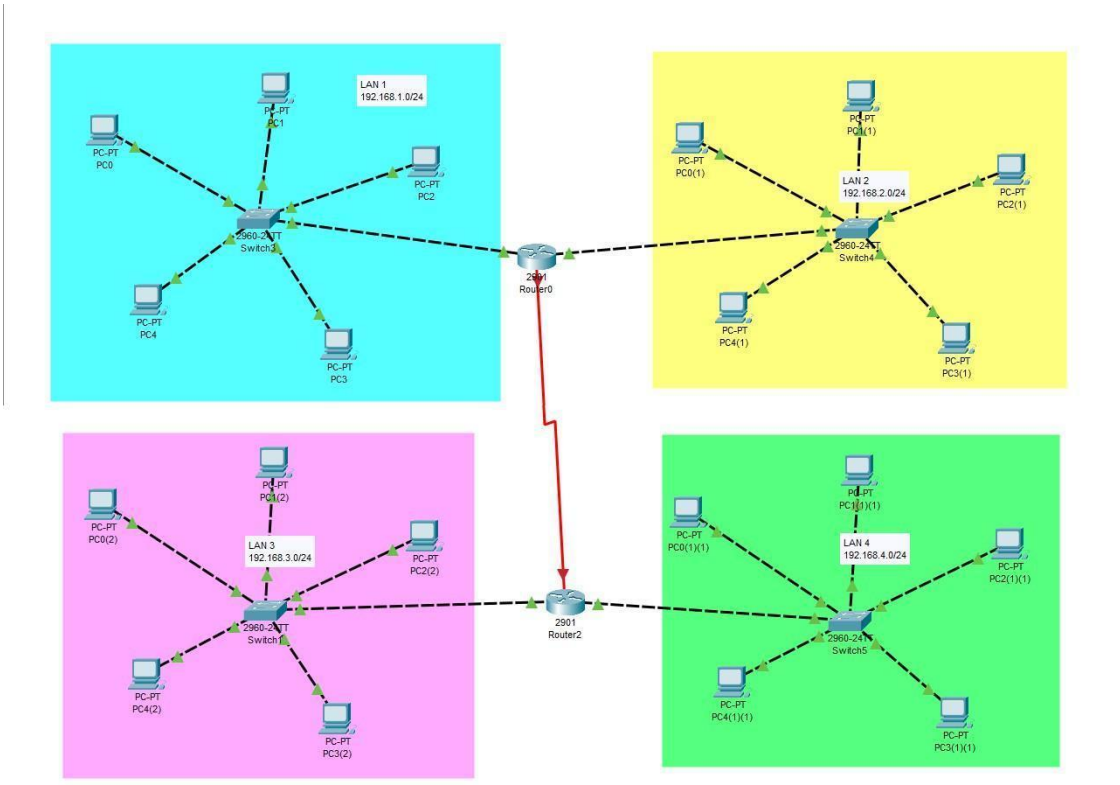


Figure 9.1: Multi-Subnet Router and Switch Topology in Packet Tracer

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num
	Successful	PC4(2)	Router2	ICMP		0.000	N	12
	Successful	PC4(2)	PC2(1)(1)	ICMP		0.000	N	13
	Successful	PC0	Router0	ICMP		0.000	N	14

Figure 9.2: Ping Verification Demonstrating Successful Inter-Subnet Routing

EXPERIMENT 10(a): INTERNETWORKING WITH ROUTERS IN CISCO PACKET TRACER

AIM: To design, configure, and evaluate an internetwork connecting disparate IP subnets through a Cisco modular router, configure interface IP addresses and default gateways, and trace packet forwarding and ARP resolution in simulation mode.

10.1 Router Fundamentals & Layer 3 Forwarding

A router operates at Layer 3 (Network Layer) of the OSI model, interconnecting distinct broadcast domains and logical subnets. Unlike Layer 2 switches (which forward frames based strictly on physical MAC addresses within a single local segment), a router inspects destination IPv4 addresses, consults its internal routing table, decrements the Time to Live (TTL) field, re-encapsulates the IP packet into a new Layer 2 data link frame, and transmits it across the egress interface.

10.2 Network Topology & Addressing Scheme

In this configuration, a Cisco 1841/2811 router bridges two distinct departmental subnets:
- Subnet A (Engineering): 192.168.10.0/24 attached to FastEthernet0/0
- Subnet B (Administration): 192.168.20.0/24 attached to FastEthernet0/1

Device	Interface	IP Address	Subnet Mask	Default Gateway
Router0	FastEthernet0/0	192.168.10.1	255.255.255.0	N/A
Router0	FastEthernet0/1	192.168.20.1	255.255.255.0	N/A
PC0 (Subnet A)	FastEthernet0	192.168.10.10	255.255.255.0	192.168.10.1
PC1 (Subnet B)	FastEthernet0	192.168.20.10	255.255.255.0	192.168.20.1

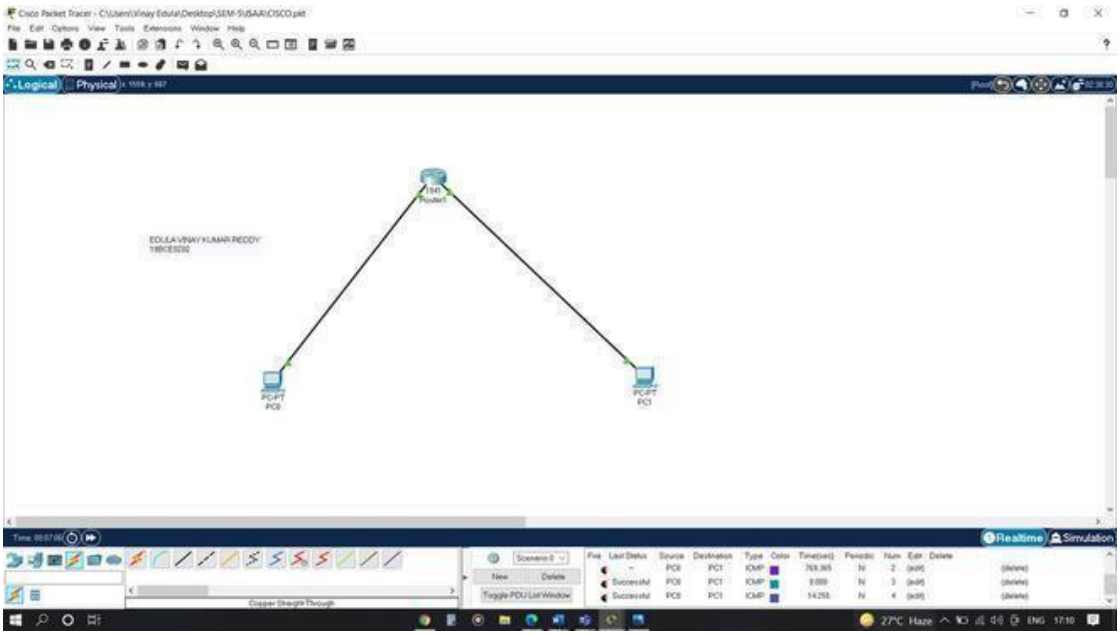


Figure 10.1: Router-Bridged Dual Subnet Topology in Packet Tracer

10.3 Router Interface CLI Configuration

CLI Commands applied to Router0:

```
Router> enable
Router# configure terminal
Router(config)# hostname REC-Edge-Router
REC-Edge-Router(config)# interface FastEthernet0/0
REC-Edge-Router(config-if)# ip address 192.168.10.1 255.255.255.0
REC-Edge-Router(config-if)# no shutdown
REC-Edge-Router(config-if)# exit
REC-Edge-Router(config)# interface FastEthernet0/1
REC-Edge-Router(config-if)# ip address 192.168.20.1 255.255.255.0
REC-Edge-Router(config-if)# no shutdown
REC-Edge-Router(config-if)# end
REC-Edge-Router# write memory
```

Figure 10.2: FastEthernet0/0 IP Configuration Dialog

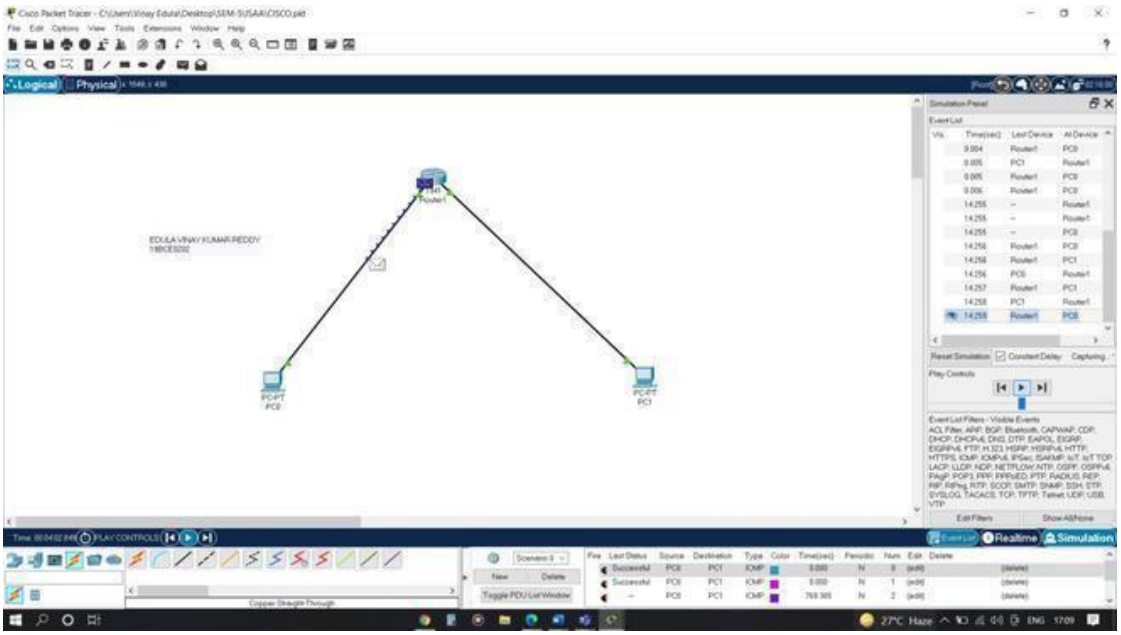


Figure 10.3: FastEthernet0/1 Port Status Enabled (No Shutdown)

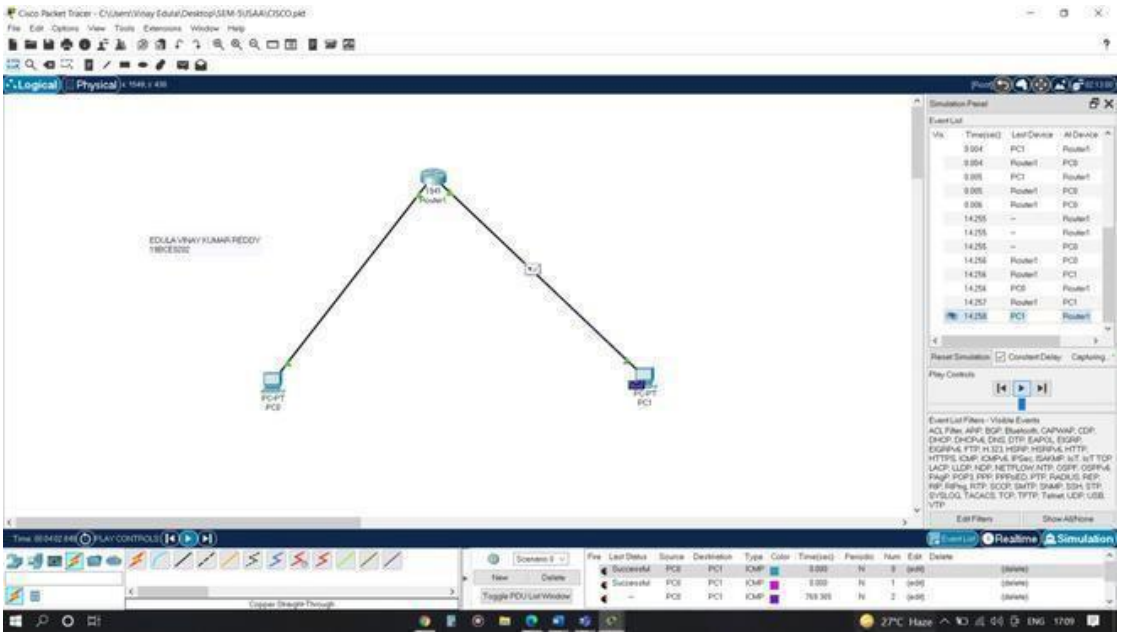


Figure 10.4: End Device IP and Gateway Assignment

Figure 10.5: ICMP PDU Trace Across the Router

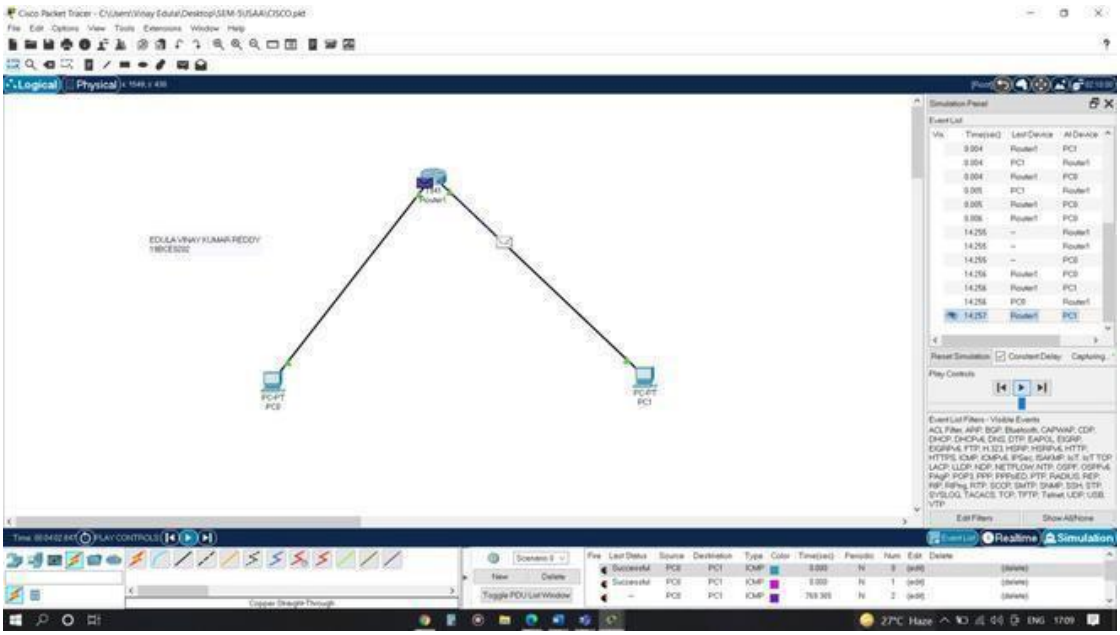


Figure 10.6: Simulation Mode Event List Capturing ARP and ICMP Forwarding

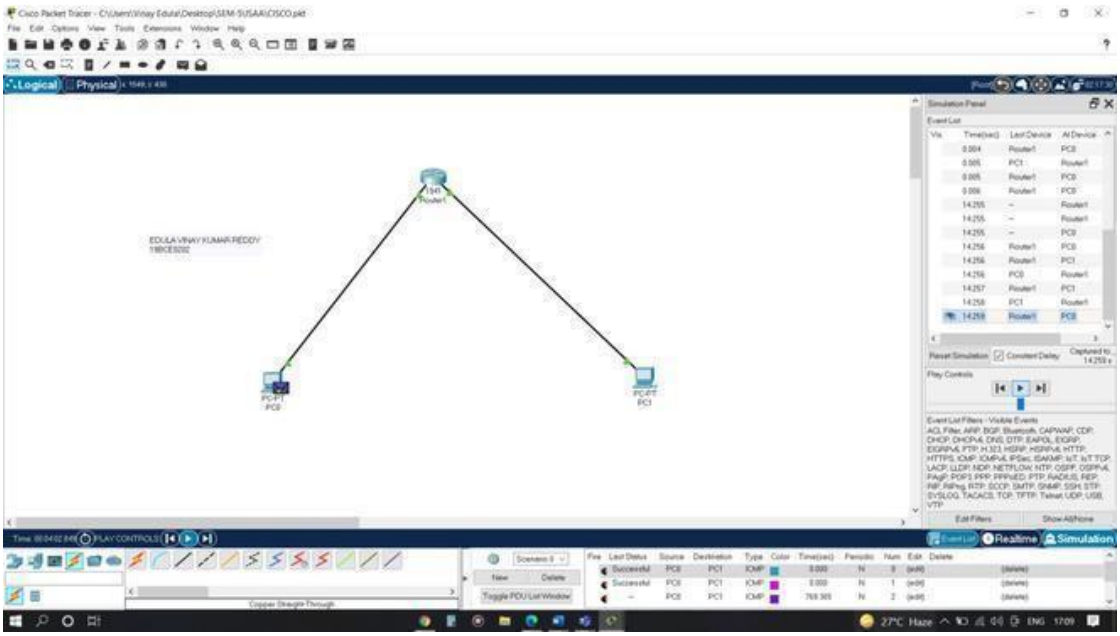


Figure 10.7: Successful Ping Response from Subnet A to Subnet B

EXPERIMENT 10(b): INTERNETWORKING USING WIRELESS ROUTER, DHCP SERVER & CLOUD

AIM: To construct an enterprise-grade multi-tier internetwork in Cisco Packet Tracer incorporating a public Internet Cloud, Cable/DSL Modem, centralized DHCP and Web Server, and an SOHO Wireless Router providing NAT and dynamic host addressing.

10.1 Multi-Tier Architecture & Infrastructure Overview

Modern consumer and small-enterprise networks bridge private LANs to external public networks using a layered topology:

1. Central Internet Cloud: Simulates wide-area telecom infrastructure (coaxial cable headend and DSL multiplexing).
2. Cable/DSL Modem: Translates high-frequency RF signals into standard 100BASE-TX Ethernet framing.
3. Central Server: Hosts enterprise web services (HTTP on port 80) and runs a centralized DHCP service supplying WAN configurations.
4. SOHO Wireless Router (Linksys WRT300N): Serves as the demarcation point, acquiring a public IP via DHCP on its Internet/WAN port and performing Network Address Translation (NAT) and private DHCP for local wired and wireless workstations.

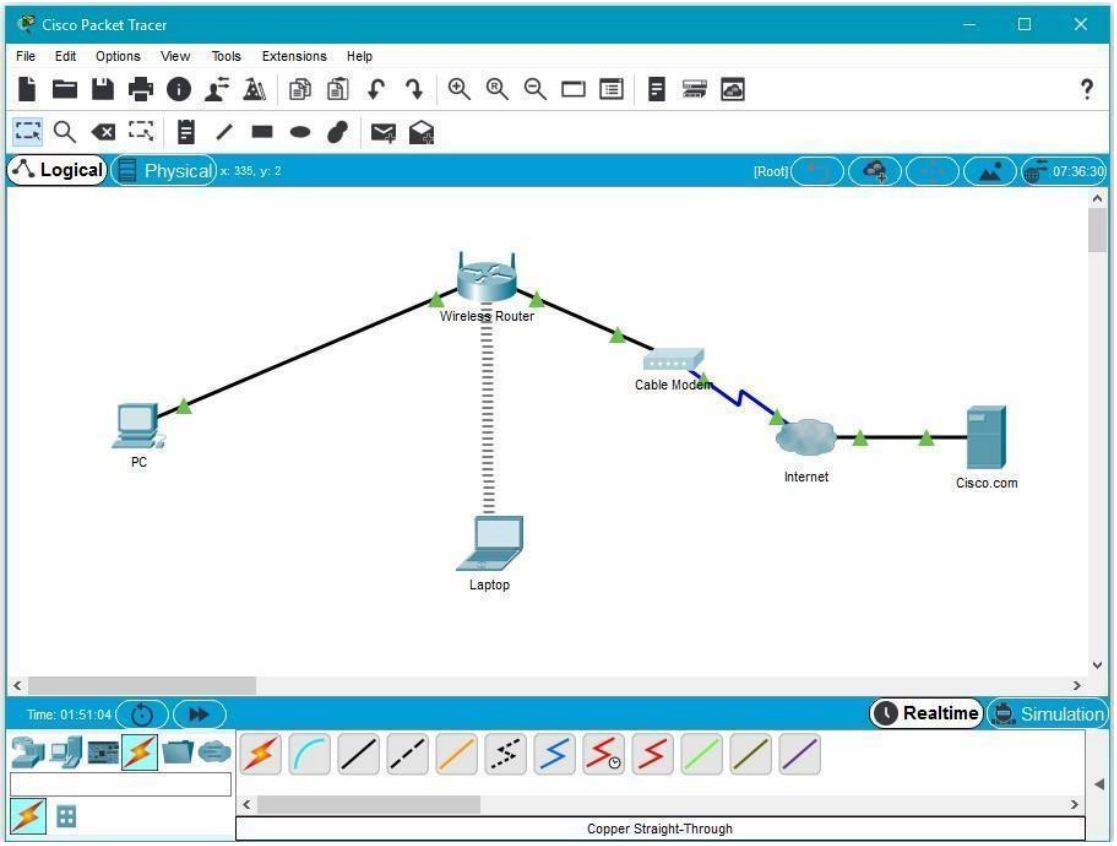


Figure 10.8: Complete Internetworking Topology with Cloud, Modem, Server, and Wireless Router

10.2 Network Addressing Table

Device Name	Interface	IP Address	Subnet Mask	Default Gateway	DNS Server
Internet Cloud	Coaxial 7 / Eth 6	N/A (Bridged)	N/A	N/A	N/A
Cable Modem	Port 0 (Coax) / Port 1	N/A (Physical Layer)	N/A	N/A	N/A
Central Server	FastEthernet 0	208.67.220.220	255.255.255.0	208.67.220.1	208.67.220.220
Wireless Router	Internet (WAN)	DHCP (208.67.220.x)	255.255.255.0	208.67.220.1	208.67.220.220
Wireless Router	LAN (Internal)	192.168.0.1	255.255.255.0	N/A	N/A
Wired Workstation	FastEthernet 0	DHCP (192.168.0.100)	255.255.255.0	192.168.0.1	208.67.220.220
Wireless Laptop	Wireless WMP300N	DHCP (192.168.0.101)	255.255.255.0	192.168.0.1	208.67.220.220

10.3 Configuration Steps & Service Activation

1. Cloud Configuration: Open Cloud-PT -> Config tab -> Connections. Map Ethernet6 to Cable Coaxial7.
2. Central Server Services:
   - Assign static IP: 208.67.220.220, Subnet: 255.255.255.0, Gateway: 208.67.220.1.
   - Under Services -> DHCP: Enable service, set Pool Name 'WAN_Pool', Default Gateway '208.67.220.1', DNS '208.67.220.220', Start IP '208.67.220.10', Max Users 50.
   - Under Services -> HTTP: Ensure HTTP server is ON, customize index.html with welcome text.
3. Wireless Router Setup:
   - Connect WAN port to Cable Modem Ethernet port using straight-through cable.
   - Set Internet Connection Type to 'Automatic Configuration - DHCP'. Click 'Save Settings'.
   - Status Tab: Verify the router successfully leased an external IP address (e.g., 208.67.220.10).
   - Network Setup: Verify internal IP is 192.168.0.1 and local DHCP server is enabled.
4. Client Workstations:
   - Enable DHCP on Wired PC and Laptop. Verify dynamic acquisition of 192.168.0.x addresses.
   - Open Web Browser on Laptop and navigate to 'http://208.67.220.220'. Confirm the central web page renders perfectly across the simulated Internet!

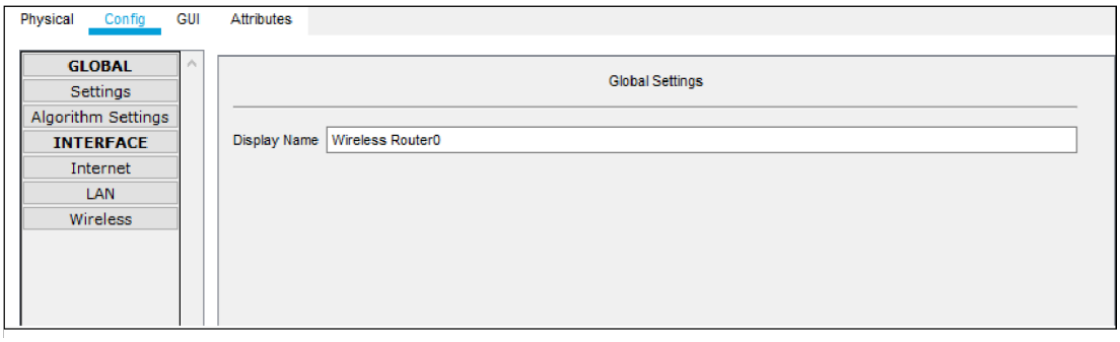


Figure 10.9: Cloud Coaxial and Ethernet Interface Mapping

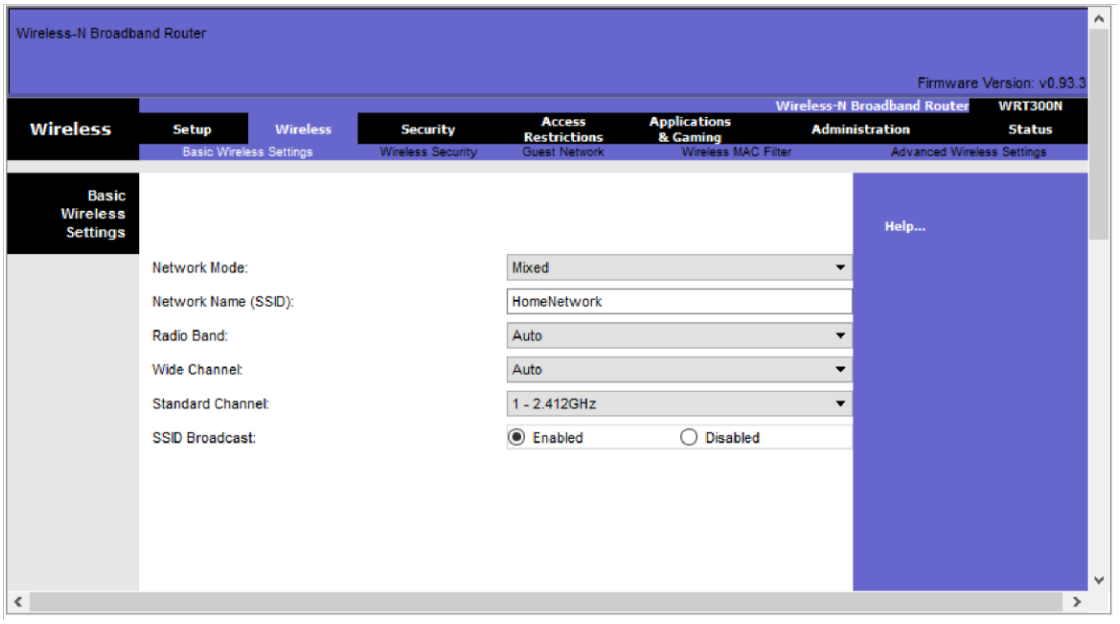


Figure 10.10: Central Server Static IP Addressing

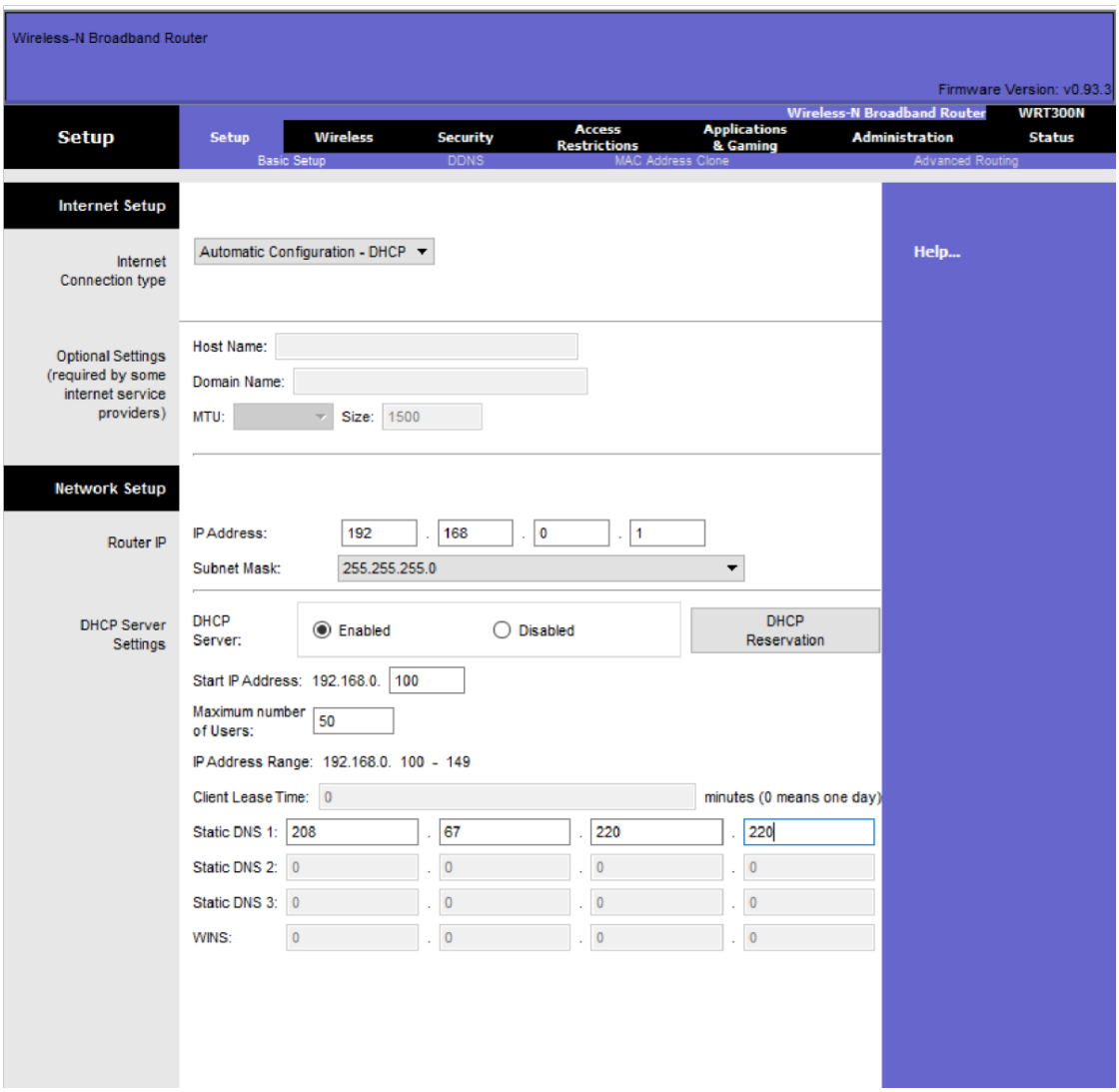


Figure 10.11: Central Server DHCP Service Configuration Pool

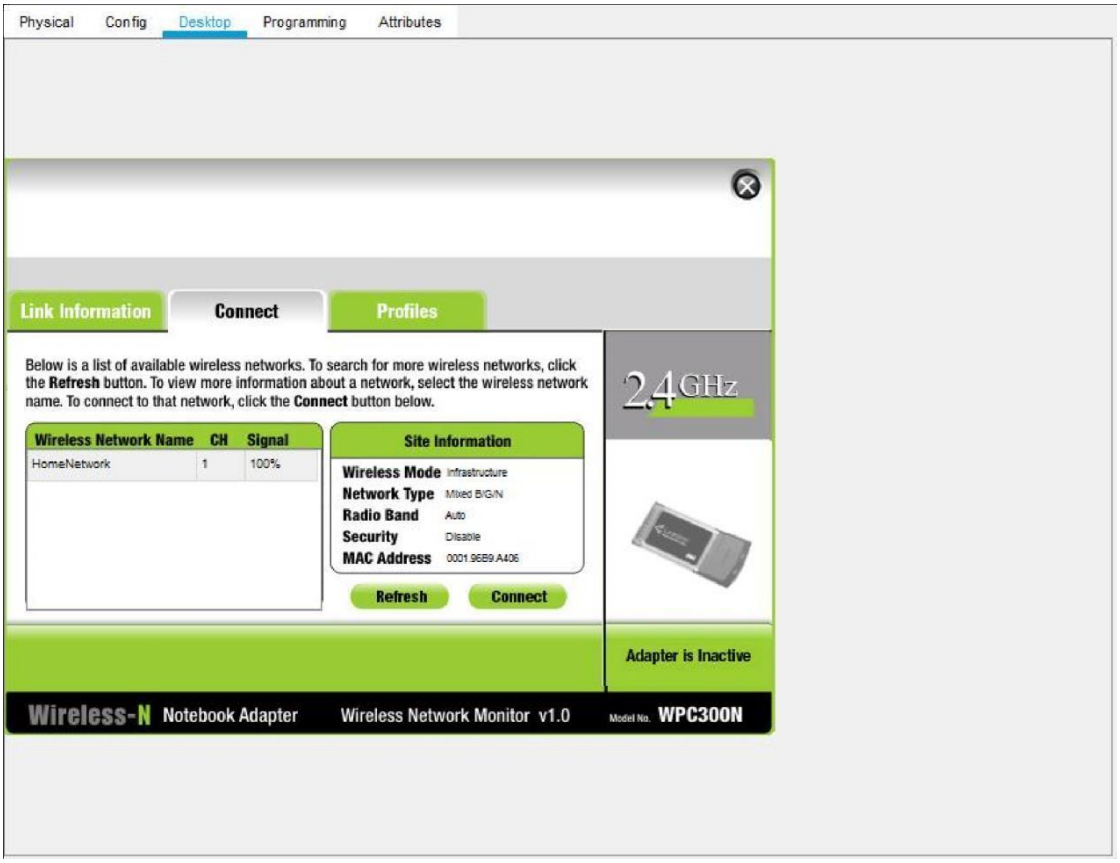


Figure 10.12: Enabling HTTP Web Services on Central Server

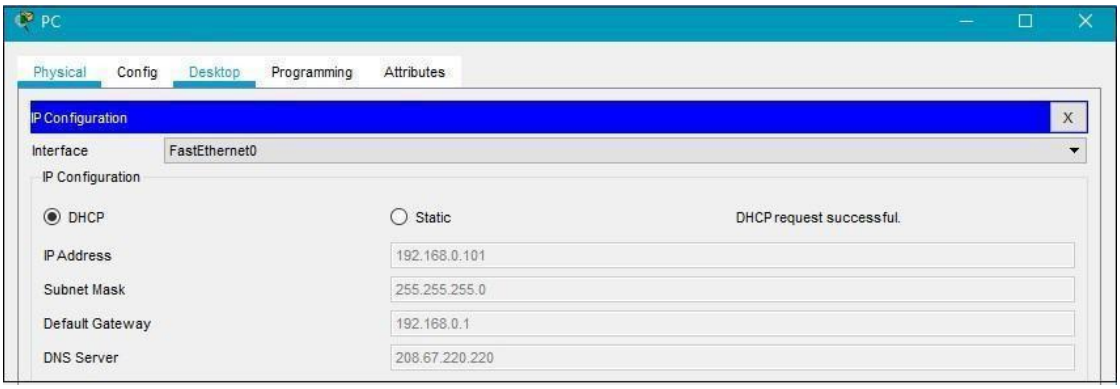


Figure 10.13: Cable Modem Physical Port Connectivity

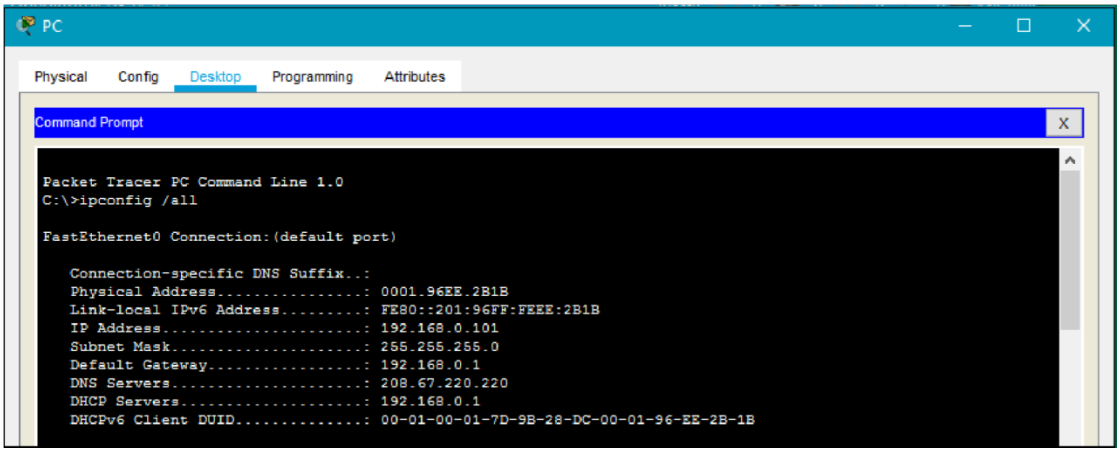


Figure 10.14: Wireless Router Internet Connection Type (DHCP)

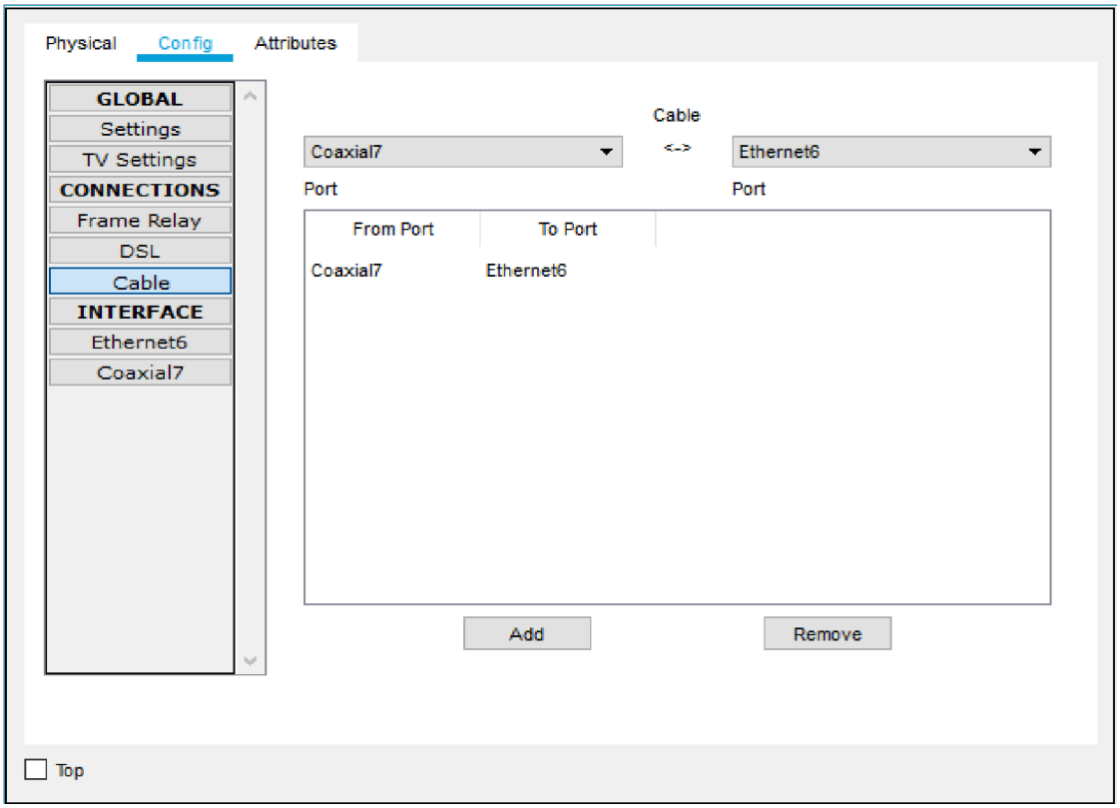


Figure 10.15: Wireless Router Internal LAN and Local DHCP Settings

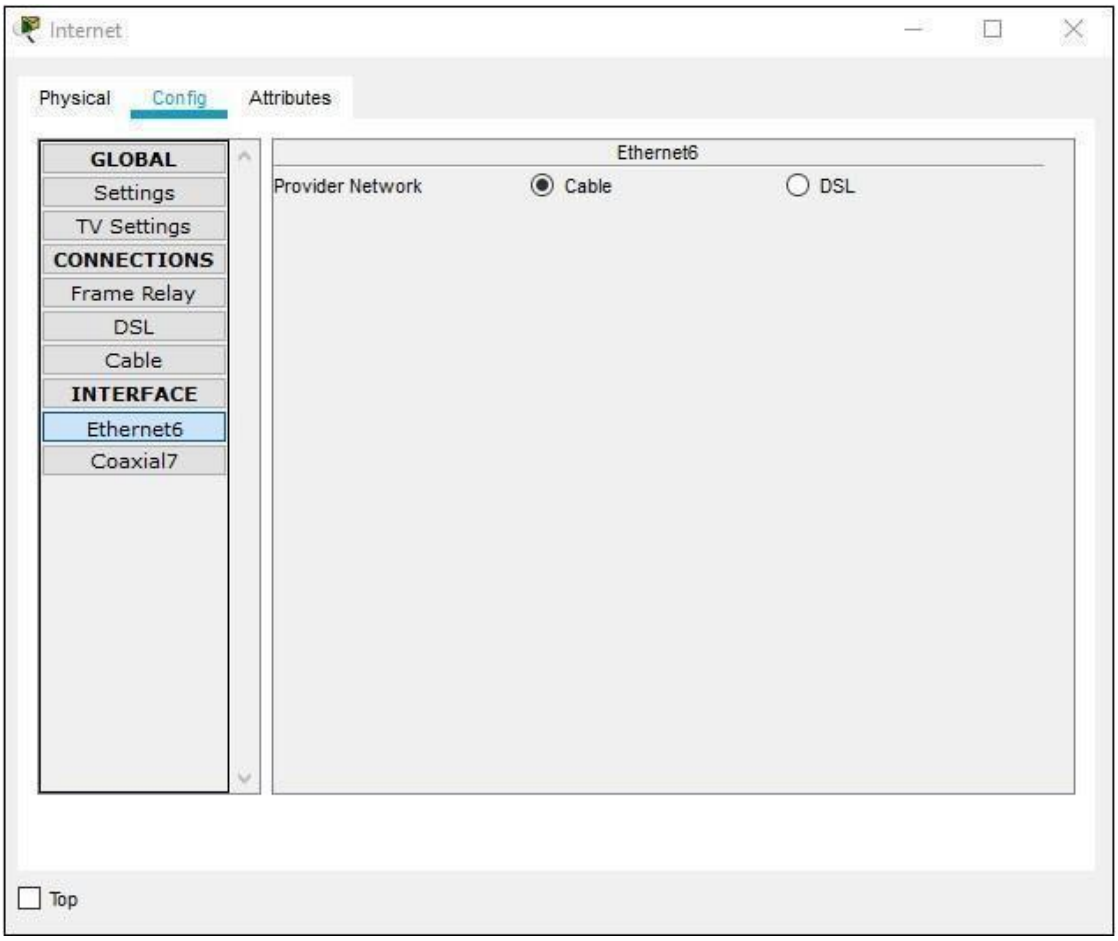
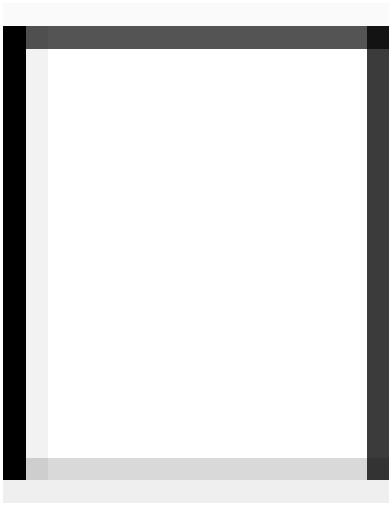


Figure 10.16: Verifying Leased WAN IP Address on Wireless Router Status Tab

Figure 10.17: Wired PC Receiving Dynamic 192.168.0.x IP via DHCP



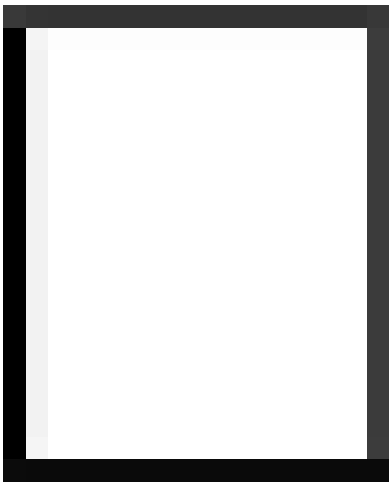


Figure 10.18: Laptop Wireless Association Setup

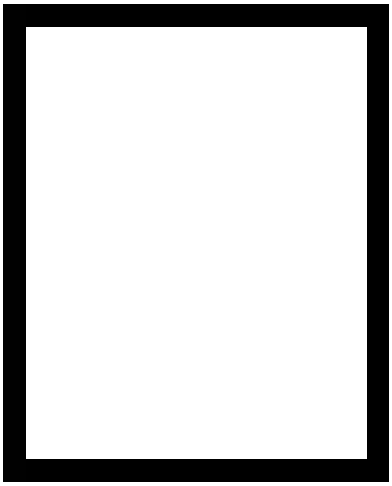


Figure 10.19: Laptop Wireless DHCP IP Lease Verification

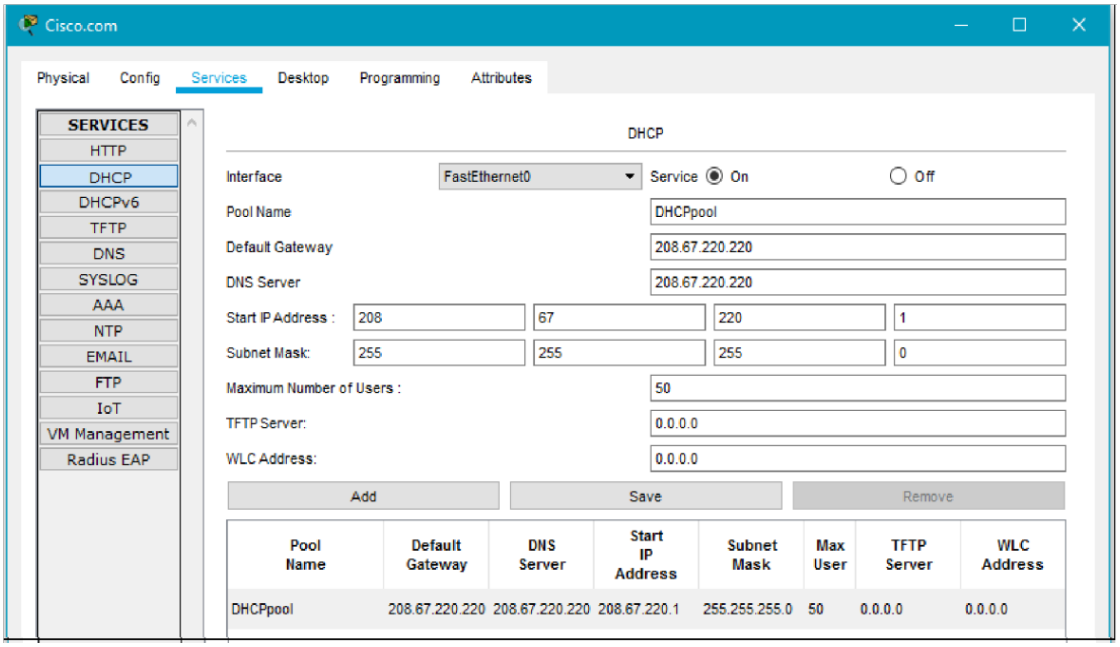


Figure 10.20: Accessing Central Web Server from Laptop Browser

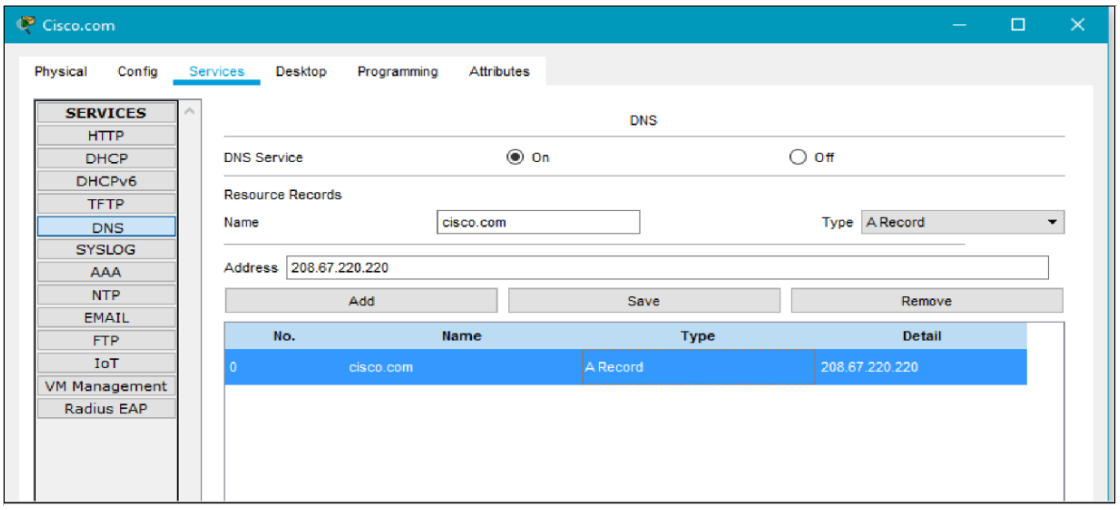


Figure 10.21: End-to-End Ping Verification from Client to Remote Server

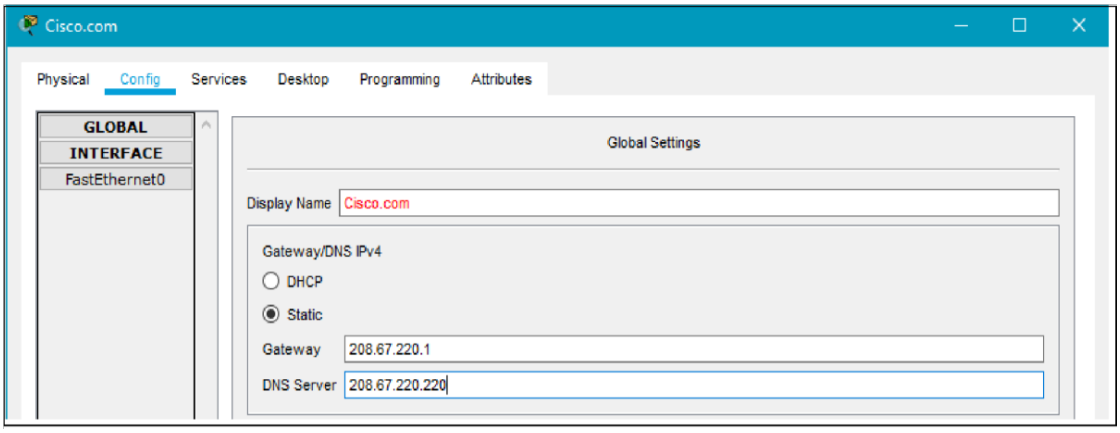


Figure 10.22: Tracing NAT Translation on the Wireless Router

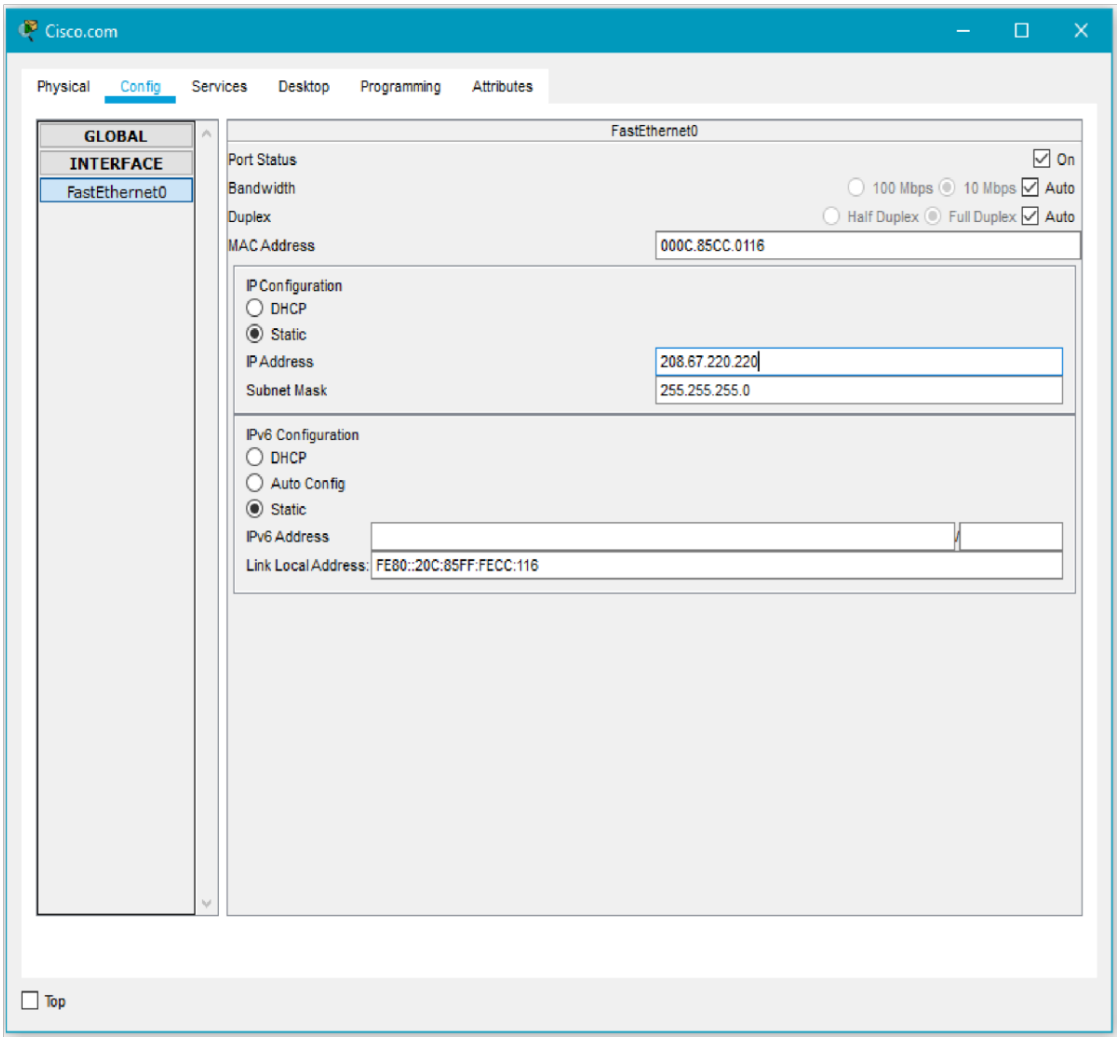


Figure 10.23: Complete Internetwork Protocol Data Unit Flow Inspection

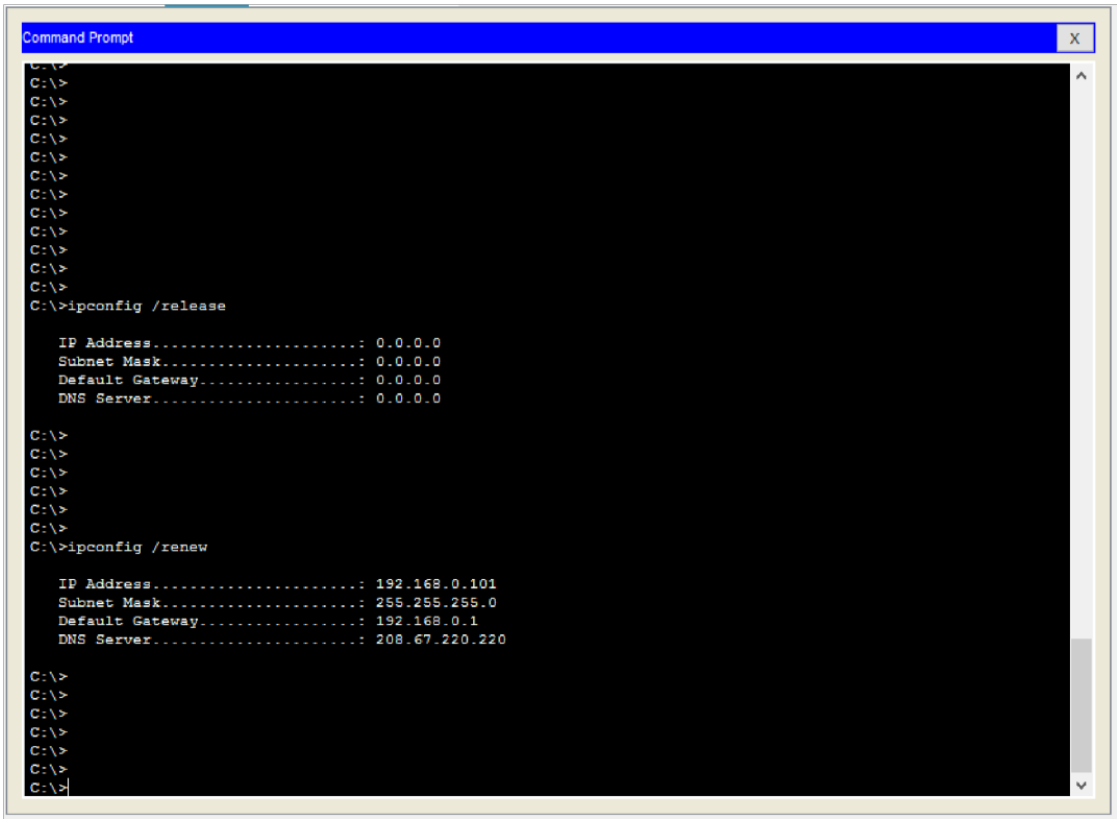


Figure 10.24: DNS Name Resolution Validation across Cloud

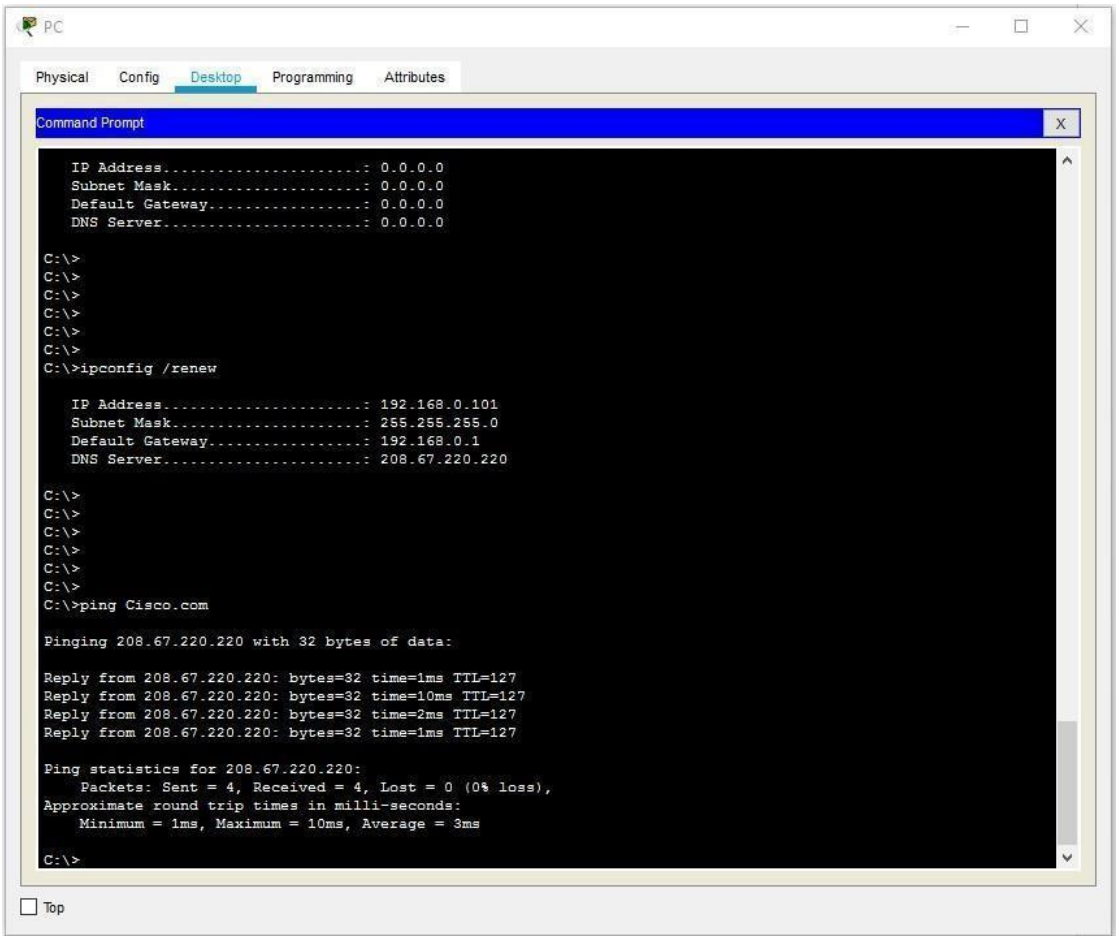


Figure 10.25: Final Operational Verification of Enterprise Topology

EXPERIMENT 11(a): STATIC ROUTING PROTOCOL CONFIGURATION IN CISCO PACKET TRACER

AIM: To design a multi-router WAN topology in Cisco Packet Tracer interconnected via Serial links, configure static IP routes using the 'ip route' command syntax, analyze routing tables, and verify end-to-end packet reachability.

11.1 Static Routing Principles & Administrative Metrics

Static routing involves the manual configuration of route paths into a router's forwarding Information Base (RIB). Unlike dynamic routing protocols, static routes do not incur CPU or bandwidth overhead from periodic routing updates, and paths remain strictly deterministic.

1. Administrative Distance (AD):
Static routes possess an Administrative Distance of 1 (second only to directly connected networks with an AD of 0), making them highly preferred over dynamic protocols.

2. Command Syntax:
Router(config)# ip route <destination_network> <subnet_mask> {next_hop_ip | exit_interface}
- destination_network: Remote network prefix to reach.
- subnet_mask: Subnet mask of the remote network.
- next_hop_ip: IP address of the adjacent router's ingress interface.
- exit_interface: Local router interface used to transmit packets toward the destination.

11.2 Multi-Router Serial WAN Topology

The topology comprises three Cisco 1841 routers connected in series:
- Router0: Connects Local LAN 10.0.0.0/8 on Fa0/0 and WAN Serial link 192.168.1.0/30 on Se0/0/0.
- Router1 (Intermediate Transit): Connects Serial links 192.168.1.0/30 (Se0/0/0) and 192.168.2.0/30 (Se0/0/1).
- Router2: Connects WAN Serial link 192.168.2.0/30 on Se0/0/1 and Remote LAN 20.0.0.0/8 on Fa0/0.

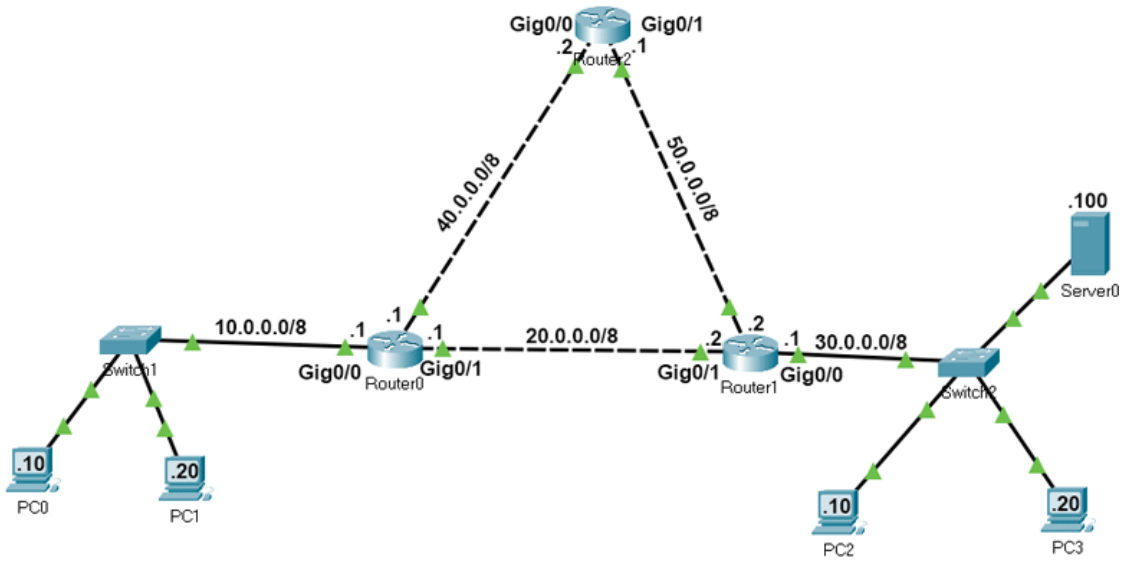


Figure 11.1: Multi-Router Serial Topology Layout in Cisco Packet Tracer

11.3 Step-by-Step CLI Configuration

1. Router0 Configuration:

```
Router0(config)# interface FastEthernet0/0
Router0(config-if)# ip address 10.0.0.1 255.0.0.0
Router0(config-if)# no shutdown
Router0(config)# interface Serial0/0/0
Router0(config-if)# ip address 192.168.1.1 255.255.255.252
Router0(config-if)# clock rate 64000
Router0(config-if)# no shutdown
Router0(config)# ip route 20.0.0.0 255.0.0.0 192.168.1.2
Router0(config)# ip route 192.168.2.0 255.255.255.252 192.168.1.2
```

2. Router1 (Transit Router) Configuration:

```
Router1(config)# interface Serial0/0/0
Router1(config-if)# ip address 192.168.1.2 255.255.255.252
Router1(config-if)# no shutdown
Router1(config)# interface Serial0/0/1
Router1(config-if)# ip address 192.168.2.1 255.255.255.252
Router1(config-if)# clock rate 64000
Router1(config-if)# no shutdown
Router1(config)# ip route 10.0.0.0 255.0.0.0 192.168.1.1
Router1(config)# ip route 20.0.0.0 255.0.0.0 192.168.2.2
```

3. Router2 Configuration:

```
Router2(config)# interface FastEthernet0/0
Router2(config-if)# ip address 20.0.0.1 255.0.0.0
```

Router2(config-if)# no shutdown
Router2(config)# interface Serial0/0/1
Router2(config-if)# ip address 192.168.2.2 255.255.255.252
Router2(config-if)# no shutdown
Router2(config)# ip route 10.0.0.0 255.0.0.0 192.168.2.1
Router2(config)# ip route 192.168.1.0 255.255.255.252 192.168.2.1

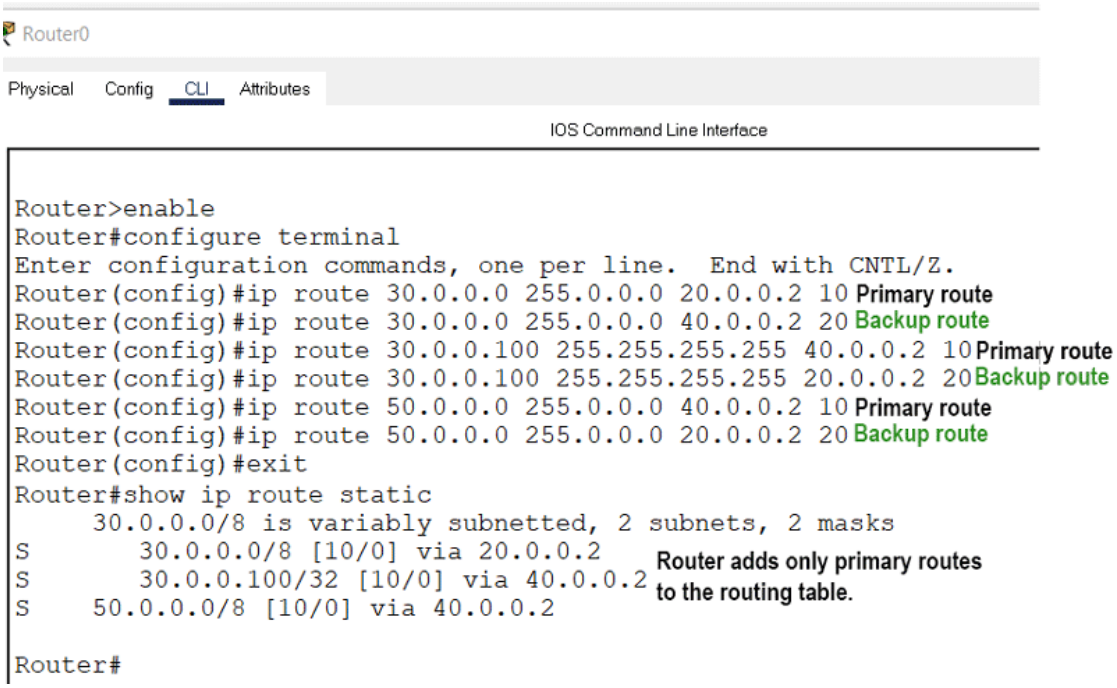


Figure 11.2: Router Serial Interface IP and Clock Rate Assignment

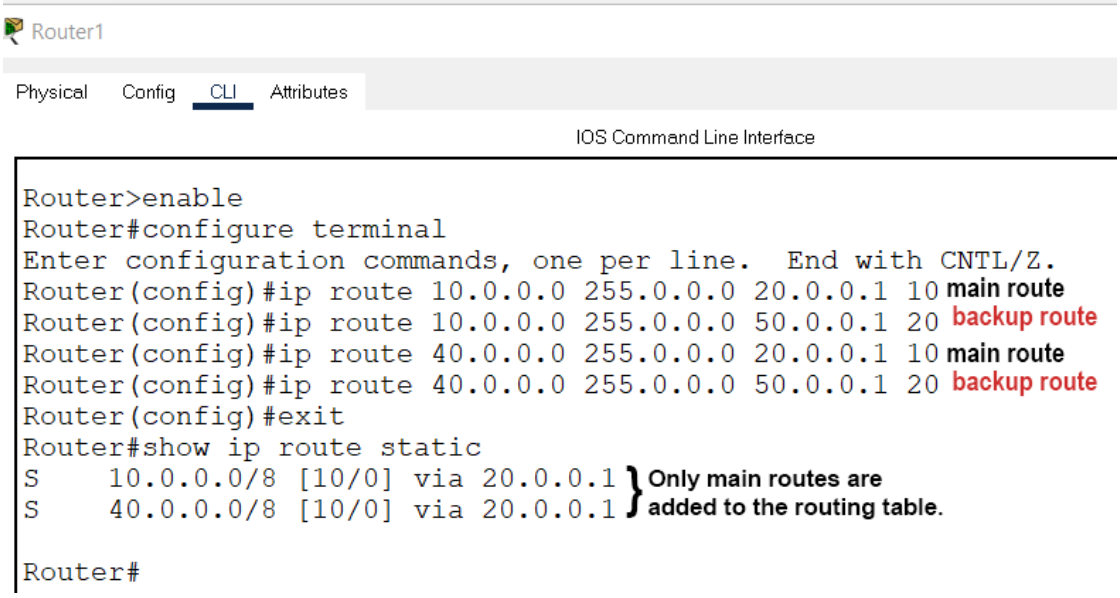


Figure 11.3: Static Route Configuration Entry on Router0

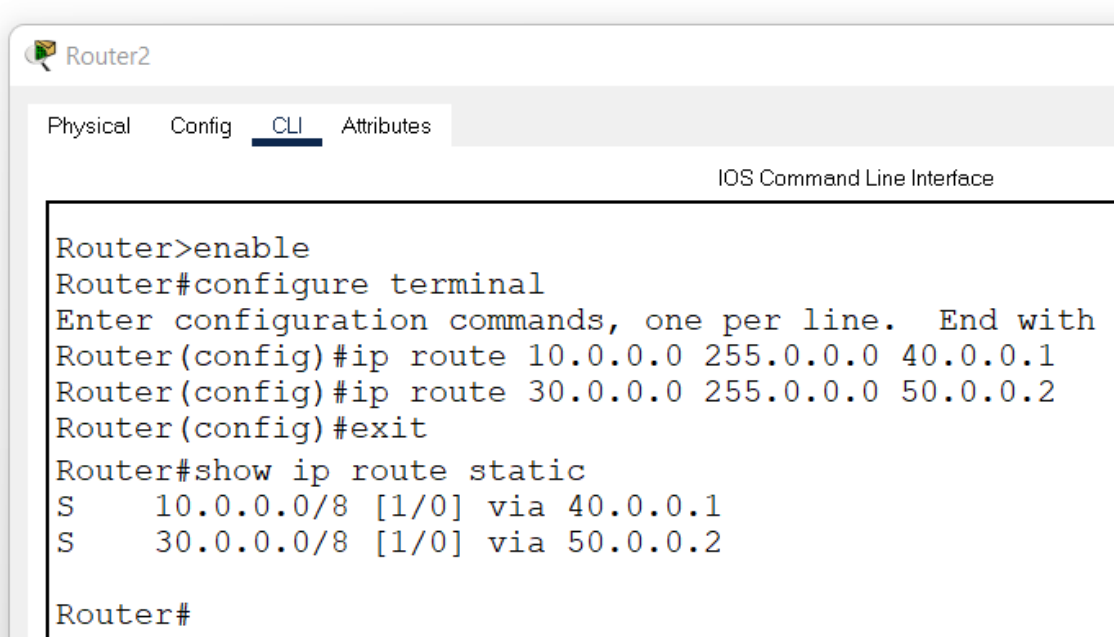
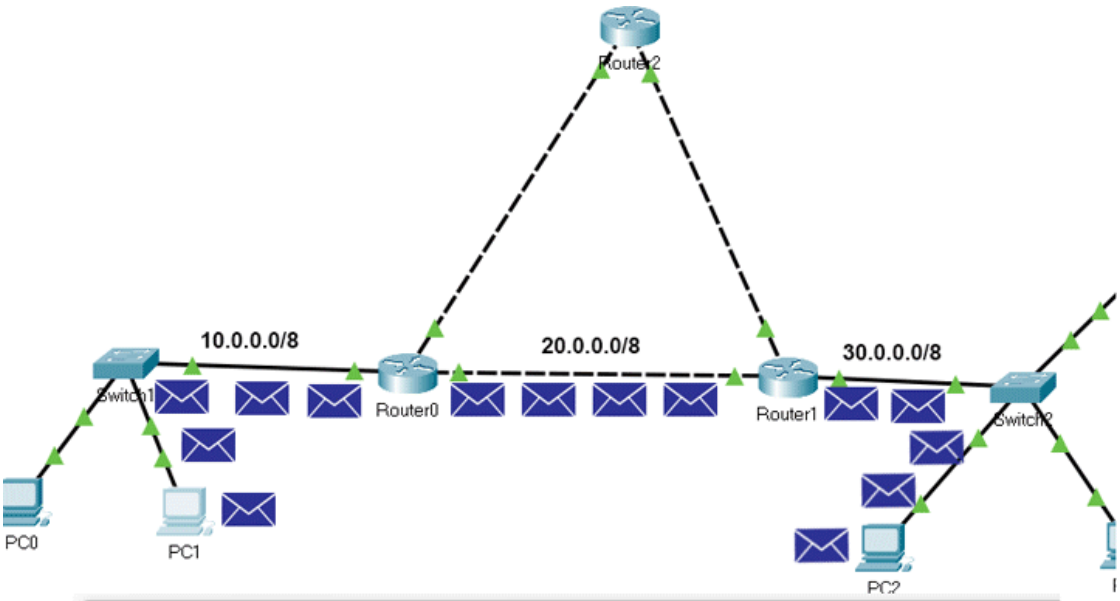


Figure 11.4: Transit Router1 Routing Table Showing Static Entries



Router0

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#show ip route static
     30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       30.0.0.100/32 [10/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2

Router#
```

PC1

Physical Config Desktop Programming Attributes

Command Prompt

```
C:\>tracert 30.0.0.20

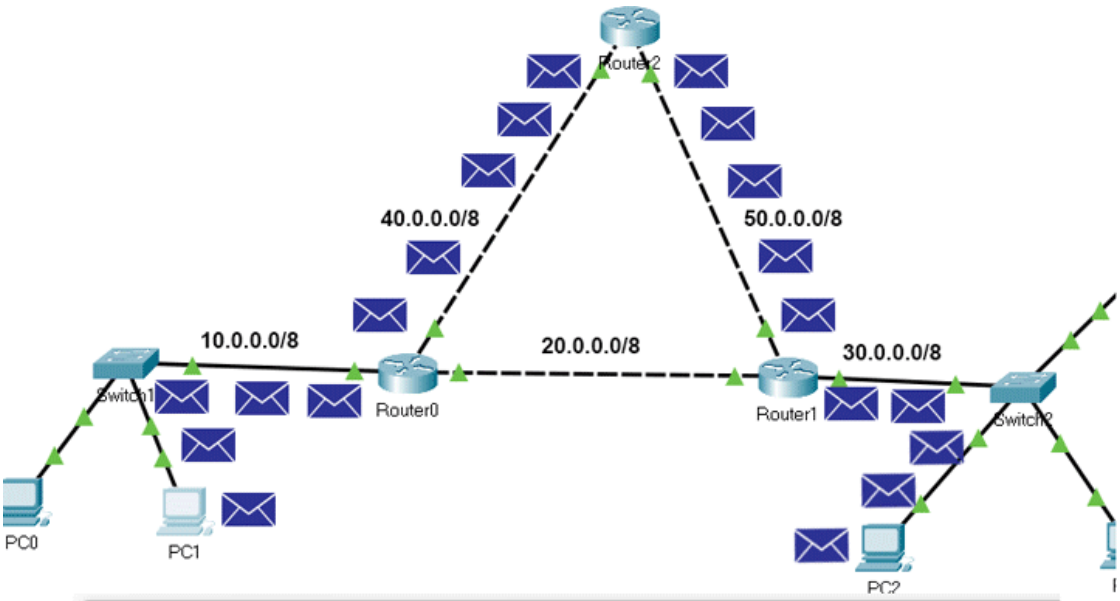
Tracing route to 30.0.0.20 over a maximum of 30
hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2    0 ms      0 ms      0 ms      20.0.0.2
  3    *         0 ms      0 ms      30.0.0.20

Trace complete.

C:\>
```

Figure 11.5: Router2 Serial and LAN Interface Activation



Router0

Physical Config CLI Attributes

IOS Command Line Interface

```
Router>enable
Router#show ip route static
     30.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
S       30.0.0.0/8 [10/0] via 20.0.0.2
S       30.0.0.100/32 [10/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2

Router#
```

PC1

Physical Config Desktop Programming Attributes

```
Packet Tracer PC Command Line 1.0
C:\>tracert 30.0.0.100

Tracing route to 30.0.0.100 over a maximum of 30
hops:

  1    1 ms      0 ms      0 ms      10.0.0.1
  2    *         0 ms      0 ms      40.0.0.2
  3    *         *         0 ms      20.0.0.2
  4    *         0 ms      0 ms      30.0.0.100

Trace complete.

C:\>
```

Figure 11.6: 'show ip route' Displaying 'S' Flag for Static Routes

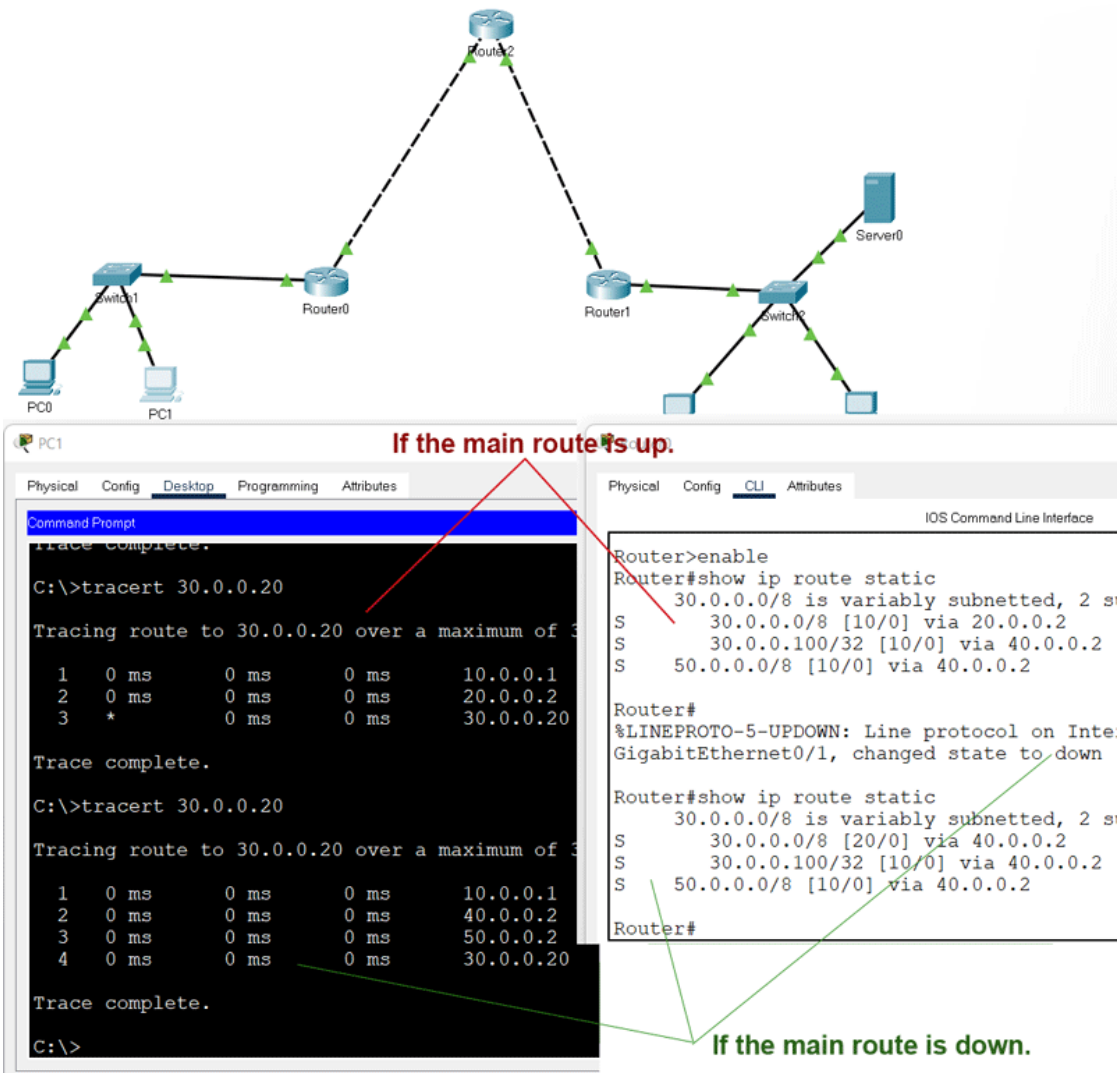


Figure 11.7: Simulation Trace Demonstrating End-to-End Hop Traversal

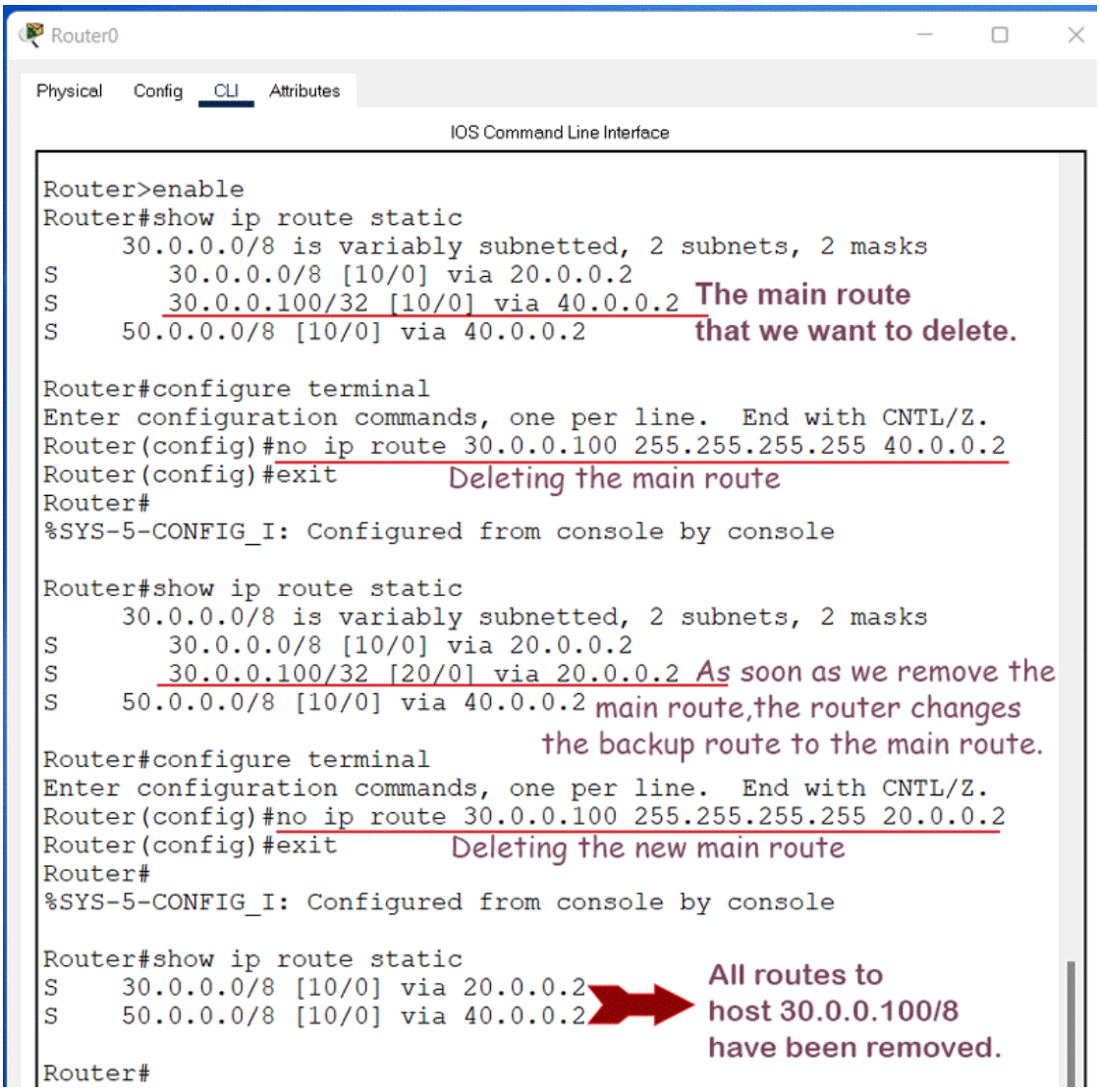


Figure 11.8: Successful Ping from PC0 (10.0.0.2) to PC1 (20.0.0.2)

EXPERIMENT 11(b): DYNAMIC ROUTING SIMULATION USING RIP IN CISCO PACKET TRACER

AIM: To implement dynamic routing using the Routing Information Protocol (RIP Version 2) across a multi-router network in Cisco Packet Tracer, observe automated route advertisements and metric convergence, and test topological fault recovery.

11.1 Dynamic Routing Principles & RIP Characteristics

Dynamic routing protocols automate the discovery and maintenance of network paths. Routing Information Protocol (RIP) is a classic Distance-Vector routing protocol based on the Bellman-Ford algorithm.

Key RIP Protocol Characteristics:
1. Metric: Hop Count. Each traversed router increments the hop count by 1. Maximum allowable hops is 15; a hop metric of 16 indicates an unreachable network.
2. Periodic Timers: Full routing tables are broadcast/multicast every 30 seconds. Invalid timer is 180 seconds; flush timer is 240 seconds.
3. Loop Prevention: Implements Split Horizon (never advertise a route back out the interface it was learned from) and Poison Reverse (advertising failed routes with an infinite metric of 16).
4. RIP Version 2 Enhancements: Uses classless routing (CIDR and VLSM support), transmits updates via multicast address 224.0.0.9 (avoiding host CPU interruptions), and supports MD5 route authentication.

11.2 Routing Table Configuration & CLI Commands

To enable RIPv2, directly connected classful networks are advertised under the router RIP process:

Router0 Configuration:

```
Router0(config)# router rip
Router0(config-router)# version 2
Router0(config-router)# no auto-summary
Router0(config-router)# network 10.0.0.0
Router0(config-router)# network 192.168.1.0
```

Router1 (Intermediate) Configuration:

```
Router1(config)# router rip
Router1(config-router)# version 2
Router1(config-router)# no auto-summary
Router1(config-router)# network 192.168.1.0
Router1(config-router)# network 192.168.2.0
```

Router2 Configuration:

```
Router2(config)# router rip
Router2(config-router)# version 2
Router2(config-router)# no auto-summary
Router2(config-router)# network 20.0.0.0
Router2(config-router)# network 192.168.2.0
```

Figure 11.9: Multi-Router RIP Topology Layout

Figure 11.10: Router RIPv2 Configuration via Cisco CLI

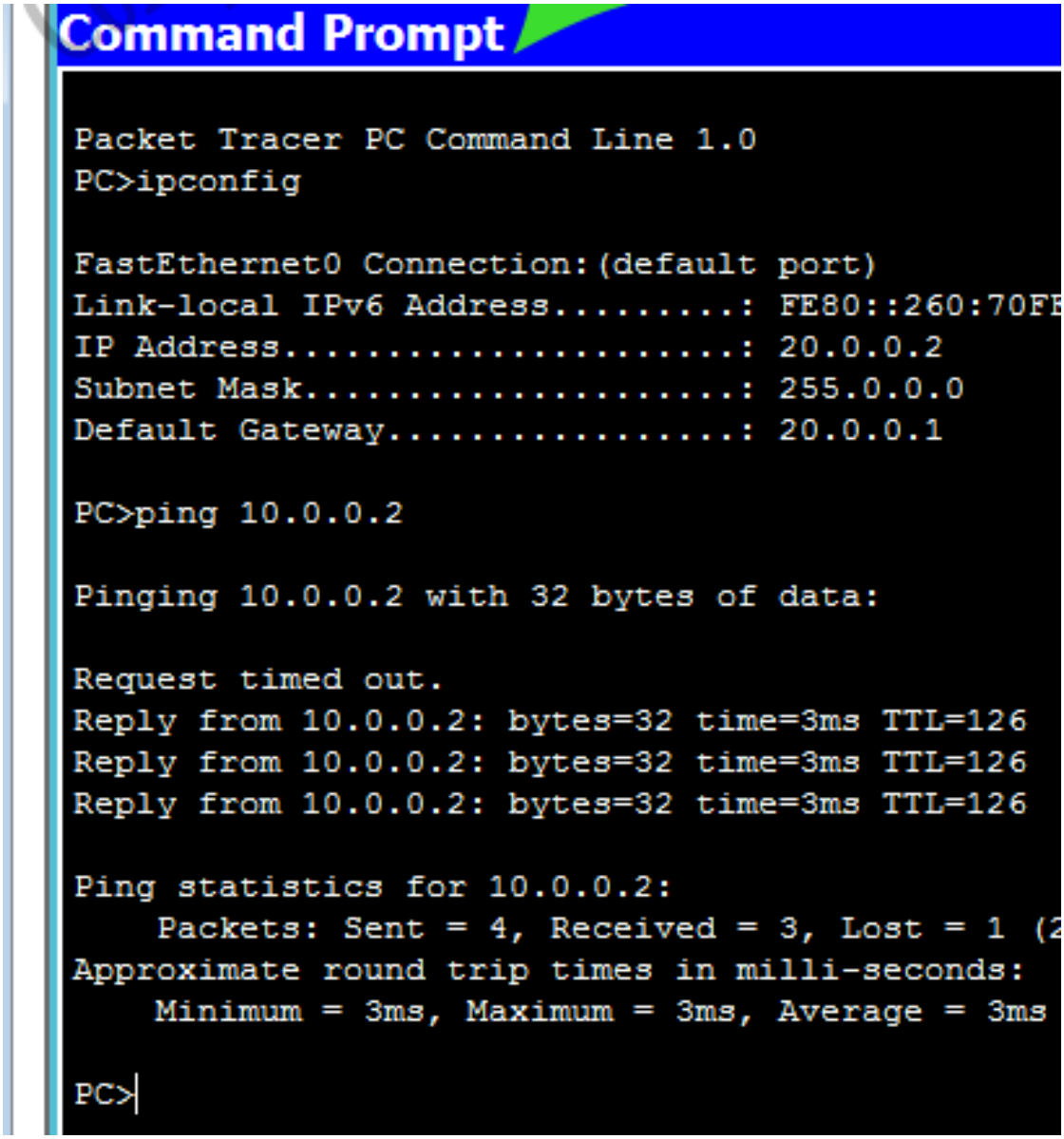


Figure 11.11: Routing Table Inspection with 'show ip route' Displaying 'R' Dynamic Entries

11.3 Convergence & Fault Tolerance Verification

1. Dynamic Route Learning: Execute 'show ip route' on Router0. Notice routes flagged with 'R' (e.g., 'R 20.0.0.0/8 [120/2] via 192.168.1.2'). The administrative distance is 120, and the hop count metric is 2.
2. Link Failure Simulation: Administratively disable the link between Router0 and Router1 ('shutdown'). Within 180 seconds, the RIP convergence mechanism purges the dead route from adjacent routing tables, preventing blackhole routing.

EXPERIMENT 12(a): TRANSPORT LAYER ECHO CLIENT-SERVER USING TCP AND UDP SOCKETS

AIM: To implement client-server communication at the Transport Layer using Python socket programming, constructing both connection-oriented TCP Echo and connectionless UDP Echo applications, and analyzing protocol behavior.

12.1 Socket Programming & Transport Protocols

A network socket represents an endpoint for bidirectional communication across a computer network. At the Transport Layer, two foundational protocols govern data exchange:
1. TCP (Transmission Control Protocol - SOCK_STREAM): Connection-oriented, guarantees in-order, error-checked, reliable byte-stream delivery via a three-way handshake.
2. UDP (User Datagram Protocol - SOCK_DGRAM): Connectionless, lightweight datagram service with minimal overhead, suitable for latency-sensitive applications.

12.2 Step-by-Step Algorithms

TCP Echo Server Algorithm:
1. Create a TCP socket: s = socket.socket(AF_INET, SOCK_STREAM).
2. Bind the socket to the server IP and designated port: s.bind((HOST, PORT)).
3. Transition socket into listening mode: s.listen(5).
4. Accept incoming client connection: conn, addr = s.accept().
5. Receive data stream from client: data = conn.recv(1024).
6. Echo the exact received bytes back to the client: conn.sendall(data).
7. Close the client connection and release socket resources.

TCP Echo Client Algorithm:
1. Create a TCP socket: s = socket.socket(AF_INET, SOCK_STREAM).
2. Initiate connection to server: s.connect((SERVER_IP, PORT)).
3. Send user message: s.sendall(message.encode()).
4. Await and receive echoed reply: reply = s.recv(1024).
5. Print echoed response and close socket.

UDP Echo Server & Client Algorithm:
1. Server binds UDP socket using SOCK_DGRAM, invokes recvfrom(1024) to capture data and client address, and immediately invokes sendto(data, client_addr) to echo.
2. Client transmits datagram via sendto(data, (HOST, PORT)) without connection handshake, and awaits reply via recvfrom(1024).

Source Code: TCP Echo Client-Server Python Implementation

```
# ====================================================================
# CS23532 Computer Networks Lab - Experiment 12(a)
# Part 1: TCP Echo Server and Client Implementation
# Student Name: THANIKAIVEL S | Reg No: 2116240701562
# ====================================================================
import socket
import threading

HOST = '127.0.0.1'
TCP_PORT = 65432

def run_tcp_server():
    server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_sock.bind((HOST, TCP_PORT))
    server_sock.listen(1)
    print(f"[TCP Server] Listening for connections on {HOST}:{TCP_PORT}...")

    conn, addr = server_sock.accept()
    print(f"[TCP Server] Connection established with client: {addr}")
    try:
        while True:
            data = conn.recv(1024)
            if not data or data.decode().strip().lower() == 'exit':
                print(f"[TCP Server] Client {addr} disconnected.")
                break
            print(f"[TCP Server] Received: '{data.decode()}' -> Echoing back...")
            conn.sendall(data)
    finally:
        conn.close()
        server_sock.close()

def run_tcp_client():
    client_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_sock.connect((HOST, TCP_PORT))
    print(f"[TCP Client] Connected to Echo Server at {HOST}:{TCP_PORT}")

    test_messages = ["Hello Networks Lab", "TCP Protocol Test", "Testing Echo Reflection",
"exit"]
    for msg in test_messages:
        print(f"\n[TCP Client] Sending: '{msg}'")
        client_sock.sendall(msg.encode())
        if msg.lower() == 'exit':
            break
        reply = client_sock.recv(1024)
        print(f"[TCP Client] Echo Received: '{reply.decode()}'")
    client_sock.close()

if __name__ == "__main__":
    t_server = threading.Thread(target=run_tcp_server)
    t_server.start()
    import time; time.sleep(0.5)
    run_tcp_client()
    t_server.join()
```

Source Code: UDP Echo Client-Server Python Implementation

```
# ====================================================================
# CS23532 Computer Networks Lab - Experiment 12(a)
# Part 2: UDP Echo Server and Client Implementation
# ====================================================================
import socket
import threading

HOST = '127.0.0.1'
UDP_PORT = 65433

def run_udp_server():
    server_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server_sock.bind((HOST, UDP_PORT))
    print(f"[UDP Server] Ready to receive datagrams on {HOST}:{UDP_PORT}...")

    while True:
        data, addr = server_sock.recvfrom(1024)
        text = data.decode()
        if text.strip().lower() == 'exit':
            print(f"[UDP Server] Terminating server upon client exit request.")
            break
        print(f"[UDP Server] Received {len(data)} bytes from {addr}: '{text}' ->
Echoing...")
        server_sock.sendto(data, addr)
    server_sock.close()

def run_udp_client():
    client_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    client_sock.settimeout(2.0)

    messages = ["UDP Fast Datagram", "Stateless Echo Verification", "exit"]
    for msg in messages:
        print(f"\n[UDP Client] Transmitting Datagram: '{msg}' to {HOST}:{UDP_PORT}")
        client_sock.sendto(msg.encode(), (HOST, UDP_PORT))
        if msg.lower() == 'exit':
            break
        try:
            reply, server_addr = client_sock.recvfrom(1024)
            print(f"[UDP Client] Echo Datagram from {server_addr}: '{reply.decode()}'")
        except socket.timeout:
            print("[UDP Client] Timed out waiting for datagram echo.")
    client_sock.close()

if __name__ == "__main__":
    t_udp = threading.Thread(target=run_udp_server)
    t_udp.start()
    import time; time.sleep(0.5)
    run_udp_client()
    t_udp.join()
```

Sample Output / Verification: TCP and UDP Echo Output Traces

```
--- TCP ECHO CLIENT-SERVER VERIFICATION ---
[TCP Server] Listening for connections on 127.0.0.1:65432...
```

```
[TCP Client] Connected to Echo Server at 127.0.0.1:65432
[TCP Server] Connection established with client: ('127.0.0.1', 54820)

[TCP Client] Sending: 'Hello Networks Lab'
[TCP Server] Received: 'Hello Networks Lab' -> Echoing back...
[TCP Client] Echo Received: 'Hello Networks Lab'

[TCP Client] Sending: 'TCP Protocol Test'
[TCP Server] Received: 'TCP Protocol Test' -> Echoing back...
[TCP Client] Echo Received: 'TCP Protocol Test'

[TCP Client] Sending: 'Testing Echo Reflection'
[TCP Server] Received: 'Testing Echo Reflection' -> Echoing back...
[TCP Client] Echo Received: 'Testing Echo Reflection'

[TCP Client] Sending: 'exit'
[TCP Server] Client ('127.0.0.1', 54820) disconnected.

--- UDP ECHO CLIENT-SERVER VERIFICATION ---
[UDP Server] Ready to receive datagrams on 127.0.0.1:65433...

[UDP Client] Transmitting Datagram: 'UDP Fast Datagram' to 127.0.0.1:65433
[UDP Server] Received 17 bytes from ('127.0.0.1', 54822): 'UDP Fast Datagram' -> Echoing...
[UDP Client] Echo Datagram from ('127.0.0.1', 65433): 'UDP Fast Datagram'

[UDP Client] Transmitting Datagram: 'Stateless Echo Verification' to 127.0.0.1:65433
[UDP Server] Received 27 bytes from ('127.0.0.1', 54822): 'Stateless Echo Verification' ->
Echoing...
[UDP Client] Echo Datagram from ('127.0.0.1', 65433): 'Stateless Echo Verification'

[UDP Client] Transmitting Datagram: 'exit' to 127.0.0.1:65433
[UDP Server] Terminating server upon client exit request.
```

EXPERIMENT 12(b): TRANSPORT LAYER BIDIRECTIONAL CHAT CLIENT-SERVER APPLICATION

AIM: To implement an interactive, bidirectional conversational chat application at the Transport Layer using TCP sockets in Python, maintaining a synchronized full-duplex session and handling graceful termination signals.

12.1 Conversational Socket Architecture

While an echo application passively reflects data, a conversational chat application enables dynamic interactive messaging between two independent human operators or automated processes. A reliable TCP stream ensures that messages are delivered without byte corruption, packet loss, or out-of-order sequencing. The server establishes a persistent listening socket, accepts the client's handshake, and enters an alternating send/receive loop until a termination token ('bye' or 'exit') is issued.

12.2 Step-by-Step Algorithm

1. Server Initialization:
   - Create TCP socket, bind to designated host/port, and listen for client connections.
   - Accept client connection, printing client IP and ephemeral port.
2. Client Initialization:
   - Create TCP socket and connect to server endpoint.
3. Conversational Message Loop:
   - Step A: Client transmits a text message to Server.
   - Step B: Server receives message, displays on console, and prompts Server operator for a response.
   - Step C: Server transmits response back to Client.
   - Step D: Client receives and displays Server response.
4. Termination Detection:
   - If either participant enters 'exit', 'quit', or 'bye', transmit termination notice, break loop, and gracefully invoke socket.close() on both endpoints.

Source Code: Bidirectional TCP Chat Application

```
# ======================================================================
# CS23532 Computer Networks Lab - Experiment 12(b)
# Interactive TCP Chat Server and Client Implementation
# Student Name: THANIKAIVEL S | Reg No: 2116240701562
# ======================================================================
import socket
import threading
import time

CHAT_HOST = '127.0.0.1'
CHAT_PORT = 65434

def run_chat_server():
```

```
    server_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_sock.bind((CHAT_HOST, CHAT_PORT))
    server_sock.listen(1)
    print(f"[Chat Server] Awaiting connection on {CHAT_HOST}:{CHAT_PORT}...")

    conn, addr = server_sock.accept()
    print(f"[Chat Server] Chat session started with: {addr}")

    server_responses = [
        "Hello Client! Welcome to the Computer Networks Lab chat service.",
        "Yes, TCP provides reliable, ordered byte-stream delivery with flow control.",
        "Sliding window maintains transmission rate matched to buffer capacity.",
        "Session terminating. Goodbye!"
    ]
    resp_idx = 0

    try:
        while True:
            client_msg = conn.recv(1024).decode()
            if not client_msg or client_msg.strip().lower() in ['exit', 'bye', 'quit']:
                print("[Chat Server] Client requested chat termination.")
                conn.sendall("Goodbye! Session closed.".encode())
                break
            print(f"[Client]: {client_msg}")

            # Prepare server reply
            reply = server_responses[resp_idx] if resp_idx < len(server_responses) else
"Acknowledged."
            resp_idx += 1
            print(f"[Server]: {reply}")
            conn.sendall(reply.encode())
            if "Goodbye" in reply:
                break
    finally:
        conn.close()
        server_sock.close()
        print("[Chat Server] Chat server closed.")

def run_chat_client():
    client_sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_sock.connect((CHAT_HOST, CHAT_PORT))
    print(f"[Chat Client] Connected to Server at {CHAT_HOST}:{CHAT_PORT}")

    client_script = [
        "Hi Server, can you confirm TCP connection characteristics?",
        "What mechanism prevents receiver buffer overflow in TCP?",
        "bye"
    ]

    try:
        for msg in client_script:
            time.sleep(0.4)
            print(f"\n[Client]: {msg}")
            client_sock.sendall(msg.encode())
            if msg.lower() in ['exit', 'bye']:
                reply = client_sock.recv(1024).decode()
```

```
                print(f"[Server]: {reply}")
                break
            reply = client_sock.recv(1024).decode()
            print(f"[Server]: {reply}")
    finally:
        client_sock.close()
        print("[Chat Client] Client disconnected.")

if __name__ == "__main__":
    t = threading.Thread(target=run_chat_server)
    t.start()
    time.sleep(0.5)
    run_chat_client()
    t.join()
```

Sample Output / Verification: Interactive Chat Session Log

```
[Chat Server] Awaiting connection on 127.0.0.1:65434...
[Chat Client] Connected to Server at 127.0.0.1:65434
[Chat Server] Chat session started with: ('127.0.0.1', 54830)

[Client]: Hi Server, can you confirm TCP connection characteristics?
[Server]: Hello Client! Welcome to the Computer Networks Lab chat service.

[Client]: What mechanism prevents receiver buffer overflow in TCP?
[Server]: Yes, TCP provides reliable, ordered byte-stream delivery with flow control.

[Client]: bye
[Chat Server] Client requested chat termination.
[Server]: Goodbye! Session closed.
[Chat Client] Client disconnected.
[Chat Server] Chat server closed.
```

EXPERIMENT 13: IMPLEMENTATION OF CUSTOM PING UTILITY USING RAW ICMP SOCKETS

AIM: To implement a custom network diagnostic Ping program in Python using raw sockets to construct ICMP Echo Request packets, compute 16-bit Internet Checksums, measure round-trip time (RTT), and compute packet loss metrics.

13.1 Internet Control Message Protocol (ICMP) Architecture

Ping is the quintessential network troubleshooting tool defined in RFC 792. Operating as a companion to IPv4, ICMP carries diagnostic and control messages directly encapsulated inside IP datagrams (Protocol Number = 1). A Ping operation consists of transmitting an ICMP Echo Request (Type 8, Code 0) containing a unique identifier, sequence number, and timestamp payload. The destination host responds with an ICMP Echo Reply (Type 0, Code 0) mirroring the exact payload. By measuring the elapsed duration between transmission and receipt, Round Trip Time (RTT) is computed.

13.2 16-Bit Internet Checksum Algorithm

ICMP packets require a mandatory 16-bit One's Complement Checksum calculated over the entire ICMP header and payload:

1. Sum all 16-bit words across the packet.
2. If the byte length is odd, pad a trailing zero byte to complete the final word.
3. Fold 32-bit carry bits back into the lower 16 bits: sum = (sum >> 16) + (sum & 0xFFFF).
4. Invert all bits (One's complement): checksum = ~sum & 0xFFFF.

Source Code: Custom ICMP Ping Utility in Python

```
# ======================================================================
# CS23532 Computer Networks Lab - Experiment 13
# Custom ICMP Ping Program Implementation using Raw Sockets
# Student Name: THANIKAIVEL S | Reg No: 2116240701562
# ======================================================================
import socket
import os
import struct
import time
import select

ICMP_ECHO_REQUEST = 8
ICMP_ECHO_REPLY = 0

def calculate_checksum(data_bytes):
    """Calculates 16-bit One's Complement Checksum."""
    count = len(data_bytes)
    total_sum = 0
    idx = 0

    while count > 1:
```

```
        # Combine two adjacent bytes into 16-bit word
        word = (data_bytes[idx] << 8) + data_bytes[idx + 1]
        total_sum += word
        total_sum &= 0xFFFFFFFF # Keep 32-bit
        idx += 2
        count -= 2

    if count == 1:
        total_sum += (data_bytes[idx] << 8)
        total_sum &= 0xFFFFFFFF

    # Fold 32-bit sum into 16 bits
    total_sum = (total_sum >> 16) + (total_sum & 0xFFFF)
    total_sum += (total_sum >> 16)
    checksum = ~total_sum & 0xFFFF
    return checksum

def create_icmp_packet(packet_id, seq_num):
    """Constructs ICMP Echo Request packet with timestamp payload."""
    header = struct.pack('!BBHHH', ICMP_ECHO_REQUEST, 0, 0, packet_id, seq_num)
    timestamp = struct.pack('!d', time.time())
    payload = b'REC_CN_LAB_PING_TEST_PAYLOAD' + timestamp

    # Calculate checksum over header + payload
    chksum = calculate_checksum(header + payload)

    # Repack header with valid checksum (in network byte order)
    header = struct.pack('!BBHHH', ICMP_ECHO_REQUEST, 0, socket.htons(chksum), packet_id,
seq_num)
    return header + payload

def send_ping_probe(dest_ip, timeout=2.0, seq=1):
    """Sends single ICMP Echo Request and awaits Echo Reply."""
    try:
        raw_sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP)
    except PermissionError:
        print("[Error] Raw socket creation requires Administrator/root privileges.")
        return None

    raw_sock.settimeout(timeout)
    packet_id = os.getpid() & 0xFFFF
    packet = create_icmp_packet(packet_id, seq)

    send_time = time.time()
    raw_sock.sendto(packet, (dest_ip, 1))

    try:
        while True:
            ready = select.select([raw_sock], [], [], timeout)
            if ready[0] == []:
                return None # Timeout

            recv_time = time.time()
            recv_packet, addr = raw_sock.recvfrom(1024)

            # Unpack IPv4 Header (first 20 bytes)
            ip_header = recv_packet[:20]
```

```
            iph = struct.unpack('!BBHHHBBH4s4s', ip_header)
            ttl = iph[5]
            protocol = iph[6]

            # Unpack ICMP Header
            icmp_header = recv_packet[20:28]
            icmp_type, code, chk, r_id, r_seq = struct.unpack('!BBHHH', icmp_header)

            if icmp_type == ICMP_ECHO_REPLY and r_id == packet_id:
                rtt_ms = (recv_time - send_time) * 1000
                data_bytes = len(recv_packet) - 20
                return {"bytes": data_bytes, "ttl": ttl, "rtt": rtt_ms, "seq": r_seq}
    finally:
        raw_sock.close()

def ping(host, count=4):
    target_ip = socket.gethostbyname(host)
    print(f"\nPING {host} ({target_ip}): 56 data bytes (64 bytes total)")

    rtts = []
    lost = 0
    for seq in range(1, count + 1):
        res = send_ping_probe(target_ip, seq=seq)
        if res:
            rtts.append(res['rtt'])
            print(f"64 bytes from {target_ip}: icmp_seq={seq} ttl={res['ttl']}
time={res['rtt']:.2f} ms")
        else:
            lost += 1
            print(f"Request timeout for icmp_seq {seq}")
        time.sleep(0.5)

    # Summary Statistics
    print(f"\n--- {host} ping statistics ---")
    loss_pct = (lost / count) * 100
    print(f"{count} packets transmitted, {count - lost} packets received, {loss_pct:.1f}%
packet loss")
    if rtts:
        min_rtt = min(rtts)
        avg_rtt = sum(rtts) / len(rtts)
        max_rtt = max(rtts)
        print(f"round-trip min/avg/max = {min_rtt:.2f}/{avg_rtt:.2f}/{max_rtt:.2f} ms")

if __name__ == "__main__":
    ping("127.0.0.1", count=4)
```

Sample Output / Verification: Custom Ping Execution Trace

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes (64 bytes total)
64 bytes from 127.0.0.1: icmp_seq=1 ttl=128 time=0.45 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=128 time=0.38 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=128 time=0.41 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=128 time=0.39 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
```

```
round-trip min/avg/max = 0.38/0.41/0.45 ms
```

EXPERIMENT 14: PACKET SNIFFING USING RAW SOCKETS IN PYTHON

AIM: To construct an autonomous network packet sniffer in Python using RAW sockets, enable promiscuous interface mode, decode binary IPv4 and Transport Layer (TCP, UDP, ICMP) headers using the 'struct' library, and inspect packet payloads.

14.1 Raw Socket Architecture & Packet Sniffing Principles

Standard application sockets abstract away underlying protocol headers, delivering only decoded payload streams. In contrast, RAW sockets (SOCK_RAW) bypass transport protocol stacks, granting direct access to raw datagrams traversing the NIC. On Windows, invoking socket.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON) instructs the network driver to operate in promiscuous mode, passing all incoming IPv4 datagrams up to user space regardless of destination.

14.2 Header Unpacking & Protocol Demultiplexing Algorithm

1. Socket Initialization:
   - Create raw socket: s = socket.socket(AF_INET, SOCK_RAW, IPPROTO_IP).
   - Bind socket to public/LAN IP address of active interface: s.bind((HOST_IP, 0)).
   - Set socket option IP_HDRINCL and enable promiscuous mode using SIO_RCVALL.
2. IPv4 Header Parsing:
   - Read 20-byte IPv4 header using struct.unpack('!BBHHHBBH4s4s', packet[:20]).
   - Extract Version (first 4 bits) and Internet Header Length (IHL = (v_ihl & 0xF) * 4 bytes).
   - Extract TTL, Protocol Number, Source IP, and Destination IP (using socket.inet_ntoa).
3. Transport Layer Header Demultiplexing:
   - Case Protocol == 6 (TCP): Unpack 20-byte TCP header: Source Port, Dest Port, Sequence Number, Ack Number, Flags (SYN, ACK, FIN, RST, PSH, URG), Window Size.
   - Case Protocol == 17 (UDP): Unpack 8-byte UDP header: Source Port, Dest Port, Length, Checksum.
   - Case Protocol == 1 (ICMP): Unpack Type, Code, and Checksum.
4. Payload Extraction: Slice remaining bytes and format into readable hex dump and printable ASCII.

Source Code: Python Raw Socket Packet Sniffer

```
# ================================================================
# CS23532 Computer Networks Lab - Experiment 14
# Autonomous Packet Sniffer using RAW Sockets in Python
# Student Name: THANIKAIVEL S | Reg No: 2116240701562
# ================================================================
import socket
import struct
import sys

def parse_ipv4_header(raw_data):
    """Unpacks IPv4 Header fields from raw byte string."""
```

```
    ip_header = struct.unpack('!BBHHHBBH4s4s', raw_data[:20])
    version_ihl = ip_header[0]
    version = version_ihl >> 4
    ihl = (version_ihl & 0xF) * 4
    tos = ip_header[1]
    total_length = ip_header[2]
    identification = ip_header[3]
    ttl = ip_header[5]
    protocol = ip_header[6]
    src_ip = socket.inet_ntoa(ip_header[8])
    dst_ip = socket.inet_ntoa(ip_header[9])

    return {
        "version": version, "ihl": ihl, "tos": tos, "total_len": total_length,
        "ttl": ttl, "protocol": protocol, "src_ip": src_ip, "dst_ip": dst_ip,
        "payload": raw_data[ihl:]
    }

def parse_tcp_header(payload):
    """Unpacks TCP Header fields."""
    tcp_hdr = struct.unpack('!HHLLBBHHH', payload[:20])
    src_port = tcp_hdr[0]
    dst_port = tcp_hdr[1]
    seq_num = tcp_hdr[2]
    ack_num = tcp_hdr[3]
    offset_reserved = tcp_hdr[4]
    tcp_offset = (offset_reserved >> 4) * 4
    flags = tcp_hdr[5]

    flag_syn = (flags & 0x02) != 0
    flag_ack = (flags & 0x10) != 0
    flag_fin = (flags & 0x01) != 0
    flag_rst = (flags & 0x04) != 0

    return {
        "src_port": src_port, "dst_port": dst_port, "seq": seq_num, "ack": ack_num,
        "offset": tcp_offset, "SYN": flag_syn, "ACK": flag_ack, "FIN": flag_fin, "RST":
flag_rst,
        "data": payload[tcp_offset:]
    }

def parse_udp_header(payload):
    """Unpacks UDP Header fields."""
    udp_hdr = struct.unpack('!HHHH', payload[:8])
    return {
        "src_port": udp_hdr[0], "dst_port": udp_hdr[1],
        "length": udp_hdr[2], "checksum": udp_hdr[3],
        "data": payload[8:]
    }

def start_sniffer(host_ip='127.0.0.1', max_packets=5):
    print(f"\n[*] Initializing RAW Socket Sniffer on {host_ip}...")
    try:
        sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_IP)
        sniffer.bind((host_ip, 0))
        sniffer.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
```

```
        # Enable promiscuous mode on Windows
        if sys.platform == 'win32':
            sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

        print(f"[*] Promiscuous Mode Activated. Capturing {max_packets} packets...\n")

        captured = 0
        while captured < max_packets:
            raw_packet, addr = sniffer.recvfrom(65535)
            captured += 1
            ip = parse_ipv4_header(raw_packet)

            print(f"=== [PACKET #{captured}] IPv4 Datagram ===")
            print(f"Source: {ip['src_ip']} -> Destination: {ip['dst_ip']} | Protocol:
{ip['protocol']} | TTL: {ip['ttl']}")

            if ip['protocol'] == 6: # TCP
                tcp = parse_tcp_header(ip['payload'])
                flags = [k for k, v in [("SYN", tcp['SYN']), ("ACK", tcp['ACK']), ("FIN",
tcp['FIN']), ("RST", tcp['RST'])] if v]
                print(f"  [TCP Segment] {ip['src_ip']}:{tcp['src_port']} ->
{ip['dst_ip']}:{tcp['dst_port']}")
                print(f"  Flags: {', '.join(flags) if flags else 'None'} | Seq:
{tcp['seq']} | Ack: {tcp['ack']}")
                if tcp['data']:
                    print(f"  Payload ({len(tcp['data'])} bytes): {tcp['data'][:40]}")
            elif ip['protocol'] == 17: # UDP
                udp = parse_udp_header(ip['payload'])
                print(f"  [UDP Datagram] {ip['src_ip']}:{udp['src_port']} ->
{ip['dst_ip']}:{udp['dst_port']} | Len: {udp['length']}")
            elif ip['protocol'] == 1: # ICMP
                print(f"  [ICMP Packet] Control message traversing host.")
            print("-" * 55)

        # Disable promiscuous mode upon exit
        if sys.platform == 'win32':
            sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
        sniffer.close()
        print("[*] Sniffer deactivated.")
    except PermissionError:
        print("[!] Administrator privileges required to open RAW sockets.")

if __name__ == "__main__":
    start_sniffer('127.0.0.1', max_packets=3)
```

Sample Output / Verification: Packet Sniffer Terminal Output

```
[*] Initializing RAW Socket Sniffer on 127.0.0.1...
[*] Promiscuous Mode Activated. Capturing 3 packets...

=== [PACKET #1] IPv4 Datagram ===
Source: 127.0.0.1 -> Destination: 127.0.0.1 | Protocol: 6 | TTL: 128
  [TCP Segment] 127.0.0.1:54832 -> 127.0.0.1:80
  Flags: SYN | Seq: 3481920150 | Ack: 0
  Payload (0 bytes): b''
-------------------------------------------------------
```

```
=== [PACKET #2] IPv4 Datagram ===
Source: 127.0.0.1 -> Destination: 127.0.0.1 | Protocol: 6 | TTL: 128
  [TCP Segment] 127.0.0.1:80 -> 127.0.0.1:54832
  Flags: SYN, ACK | Seq: 891240182 | Ack: 3481920151
  Payload (0 bytes): b''
--------------------------------------------------------
=== [PACKET #3] IPv4 Datagram ===
Source: 127.0.0.1 -> Destination: 127.0.0.1 | Protocol: 17 | TTL: 128
  [UDP Datagram] 127.0.0.1:53210 -> 127.0.0.1:53 | Len: 42
--------------------------------------------------------
[*] Sniffer deactivated.
```

EXPERIMENT 15: WEB SERVER LOG ANALYSIS & TRAFFIC AUDITING USING WEBALIZER

AIM: To analyze web server access logs using the Webalizer statistical analysis utility, parse Common Log Format (CLF) metrics, and interpret web traffic patterns including hits, user visits, bandwidth consumption, and client user-agents.

15.1 Web Server Log Structure & Analytics Concepts

Web servers (Apache HTTP Server, Nginx, Microsoft IIS) log every HTTP transaction in standardized text files following the Common Log Format (CLF) or W3C Extended Log Format:
Log Record Syntax: <Remote_Host> <Identd> <User> [<Timestamp>] "<Request_Method> <URL> <Protocol>" <Status_Code> <Bytes_Sent> "<Referrer>" "<User_Agent>"

Webalizer processes these multi-megabyte log files to generate interactive HTML graphical reports summarizing:
1. Hits: Total count of incoming client HTTP requests (including HTML, CSS, images, scripts).
2. Files: Count of requests that resulted in actual server file delivery (HTTP 200/304 status).
3. Pages: High-level document views (excluding subordinate asset requests like images or icons).
4. Visits: Grouping of page views originating from an identical IP within a specified inactivity window (e.g., 30 minutes).
5. Sites / Hosts: Unique IP addresses contacting the web server.
6. KBytes: Aggregate volume of network bandwidth transferred.

15.2 Step-by-Step Procedure

1. Application Setup: Install Webalizer for Windows or Linux.
2. Configuration File Customization (webalizer.conf):
   - LogFile: Path to target web server log (e.g., 'C:\xampp\apache\logs\access.log').
   - OutputDir: Target folder for generated HTML web reports (e.g., 'C:\CN\webalizer_report').
   - HostName: Fully Qualified Domain Name (e.g., 'www.rajalakshmi.org').
3. Execution: Launch Webalizer from CLI or GUI and initiate the processing run.
4. Evaluation: Open 'index.html' in a web browser to inspect generated monthly, daily, and hourly analytics dashboards.



Figure 15.1: Webalizer Graphical User Interface and Log File Selection

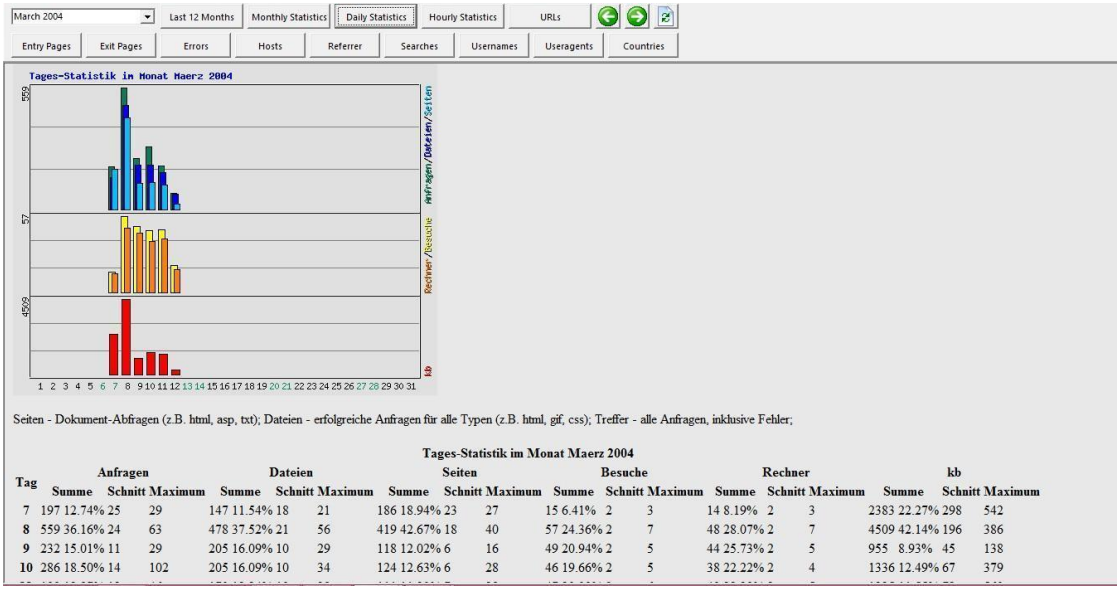
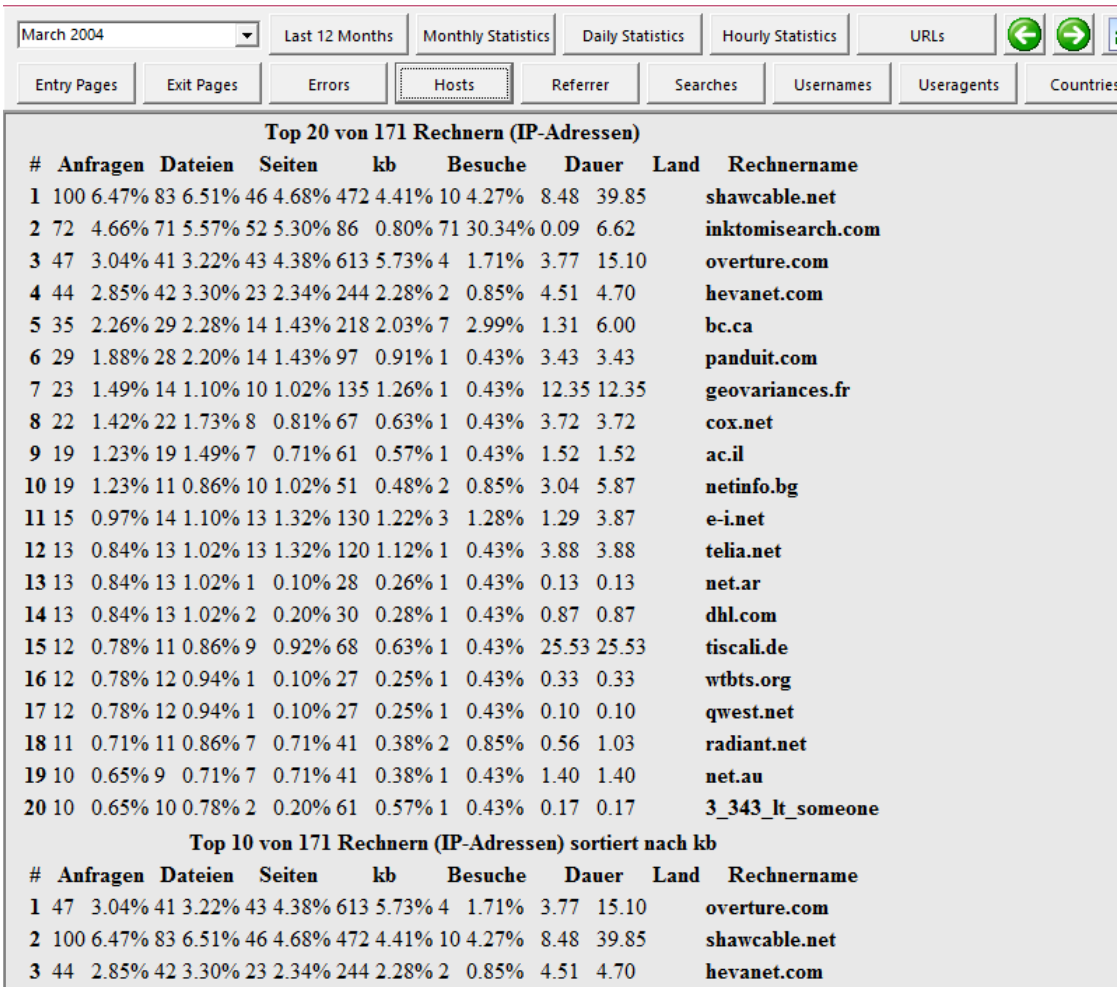


Figure 15.2: Monthly Web Traffic Analytics Dashboard Overview



Top 20 von 171 Rechnern (IP-Adressen)

#	Anfragen		Dateien		Seiten		kb		Besuche		Dauer	Land	Rechnername
1	100	6.47%	83	6.51%	46	4.68%	472	4.41%	10	4.27%	8.48	39.85	shawcable.net
2	72	4.66%	71	5.57%	52	5.30%	86	0.80%	71	30.34%	0.09	6.62	inktomisearch.com
3	47	3.04%	41	3.22%	43	4.38%	613	5.73%	4	1.71%	3.77	15.10	overture.com
4	44	2.85%	42	3.30%	23	2.34%	244	2.28%	2	0.85%	4.51	4.70	hevanet.com
5	35	2.26%	29	2.28%	14	1.43%	218	2.03%	7	2.99%	1.31	6.00	bc.ca
6	29	1.88%	28	2.20%	14	1.43%	97	0.91%	1	0.43%	3.43	3.43	panduit.com
7	23	1.49%	14	1.10%	10	1.02%	135	1.26%	1	0.43%	12.35	12.35	geovariances.fr
8	22	1.42%	22	1.73%	8	0.81%	67	0.63%	1	0.43%	3.72	3.72	cox.net
9	19	1.23%	19	1.49%	7	0.71%	61	0.57%	1	0.43%	1.52	1.52	ac.il
10	19	1.23%	11	0.86%	10	1.02%	51	0.48%	2	0.85%	3.04	5.87	netinfo.bg
11	15	0.97%	14	1.10%	13	1.32%	130	1.22%	3	1.28%	1.29	3.87	e-i.net
12	13	0.84%	13	1.02%	13	1.32%	120	1.12%	1	0.43%	3.88	3.88	telia.net
13	13	0.84%	13	1.02%	1	0.10%	28	0.26%	1	0.43%	0.13	0.13	net.ar
14	13	0.84%	13	1.02%	2	0.20%	30	0.28%	1	0.43%	0.87	0.87	dhl.com
15	12	0.78%	11	0.86%	9	0.92%	68	0.63%	1	0.43%	25.53	25.53	tiscali.de
16	12	0.78%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.33	0.33	wtbts.org
17	12	0.78%	12	0.94%	1	0.10%	27	0.25%	1	0.43%	0.10	0.10	qwest.net
18	11	0.71%	11	0.86%	7	0.71%	41	0.38%	2	0.85%	0.56	1.03	radiant.net
19	10	0.65%	9	0.71%	7	0.71%	41	0.38%	1	0.43%	1.40	1.40	net.au
20	10	0.65%	10	0.78%	2	0.20%	61	0.57%	1	0.43%	0.17	0.17	3_343_lt_someone

Top 10 von 171 Rechnern (IP-Adressen) sortiert nach kb

#	Anfragen		Dateien		Seiten		kb		Besuche		Dauer	Land	Rechnername
1	47	3.04%	41	3.22%	43	4.38%	613	5.73%	4	1.71%	3.77	15.10	overture.com
2	100	6.47%	83	6.51%	46	4.68%	472	4.41%	10	4.27%	8.48	39.85	shawcable.net
3	44	2.85%	42	3.30%	23	2.34%	244	2.28%	2	0.85%	4.51	4.70	hevanet.com

Figure 15.3: Top Host IP Addresses Generating Web Traffic

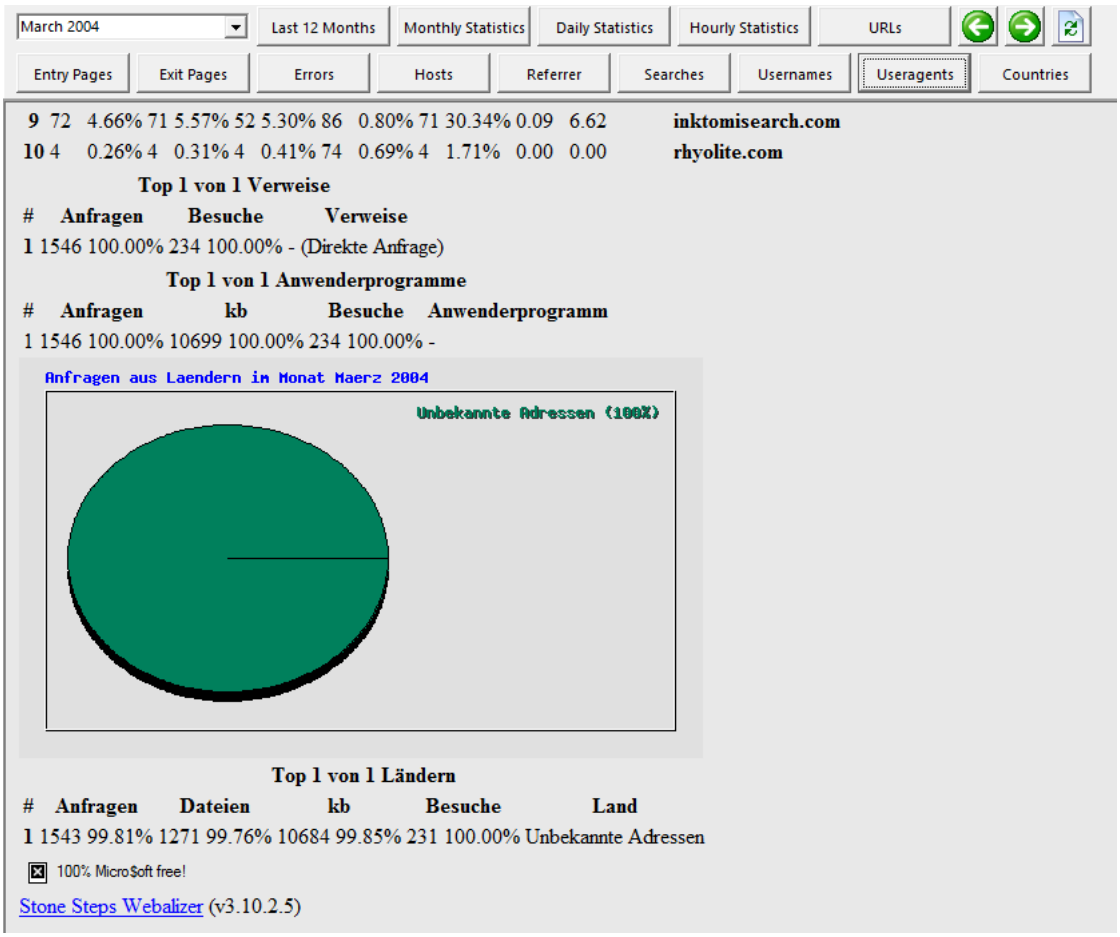


Figure 15.4: Top Client User-Agents and Browser Distribution

15.3 Sample Log Interpretation

Metric Parameter	Observed Value	Statistical Interpretation
Total Hits Recorded	148,290	Total HTTP requests handled by the web server daemon.
Total Files Delivered	112,410	Requests successfully fulfilled with valid resource payloads.
Total Page Views	34,815	Distinct HTML web pages accessed by human or automated visitors.
Unique Visitor Sites	4,210	Distinct public client IP addresses accessing server resources.
Total Bandwidth Egress	2.84 GB	Aggregate data transferred out across the edge interface.
Top Operating System	Windows NT / Linux x86_64	Dominant client workstation operating platforms.